Inside the REM Computer

Edition 2 — Expanded Offline Reference

REM Project

Contents

REM Quick Reference 1

REM Reference Alignment — Second Pass Notes 3

REM Reality Table 4

Canonical REM conventions for the examples 5

How to use this offline edition 6

Historical development path 7

Part 0 — Architecture and Platform Reference 8

REM Architecture Reference 10

REM Linux Boot Sequence 12

ISA, Platform, Kernel, and Userspace 13

What Is Implemented, Planned, or Unspecified 14

Developer Journeys 15

Preface 16

Part I — Becoming Productive on REM 21

Chapter 1 — Welcome to REM Application Development 22

Chapter 2 — Your First REM Development Session 29

Part I-A — The First Hour on REM 39

Chapter 2A — Your First REM Program 40

Chapter 2B — First Linux Syscall 41

Part II — Assembly Software Engineering 42

Chapter 3 — Writing Maintainable Assembly 43

Chapter 4 — Using libc, Syscalls and the Kernel 54

Chapter 4A — Writing a Syscall Wrapper 63

Chapter 4B — The C Runtime Boundary 64

Part III — Terminal Programming (The Missing Knowledge) 65

Chapter 5 — The Terminal Model 66

Chapter 6 — Terminal Modes and termios **75**

Chapter 7 — ANSI Terminal Control **86**

Chapter 8 — Keyboard Input Decoding **98**

REM breakpoint design **220**

Appendix A — REM Programming Checklist **259**

Appendix C — Test Program Template **266**

Appendix D — Makefile Template **268**

Appendix E — Terminal Escape Reference **270**

Appendix F — Debugging Decision Tree **273**

Appendix G — Current MyEmulator2 Toolchain Reference **275**

Appendix H — Verified Linux Syscall Reference **276**

Appendix I — Current Machine and Console Reference **278**

Appendix J — Regression Commands **279**

Appendix K — Android Offline Deployment **280**

Appendix L — Legacy Example Conversion **281**

Appendix M — First MyEmulator2 Assembly Program **282**

Appendix N — First Linux Syscall Program **283**

Appendix O — C and Startup Contract **284**

Appendix P — Current Debugging Recipe **285**

Appendix Q — Chapter-to-Repository Map **286**

Appendix R — Terminal ABI Notes **287**

REM Quick Reference

Keep this page beside the keyboard while working offline. It describes the current MyEmulator2 target; legacy MyEmulator 1.0 uses a different machine.

Item	Current value
Architecture	MyEmulator2 / `myemulator32`
Word and register size	32 bits
Address space	32-bit virtual and physical addresses
Endianness	Little-endian
Instruction width	Fixed 32-bit words, 4-byte aligned
Page size	4096 bytes
MMU	Two-level page walk: 10-bit directory, 10-bit table, 12-bit offset
Privilege modes	User and Supervisor
General registers	`r0`–`r15`; `r0` is hardwired zero
Stack and link	`r13/sp`, `r14/lr`
Function ABI arguments	`r1`–`r4`
Function ABI return	`r1` for 32-bit values
Linux syscall number/result	`r1`
Linux syscall arguments	`r2`–`r7` (up to six)
Executable format	ELF32 little-endian, initially static
Compiler/tool prefix	`myemulator2-elf-`
Assembler/linker	`myemulator2-elf-as`, `myemulator2-elf-ld`
Disassembler	`myemulator2-elf-objdump`
QEMU binary	`qemu-system-myemulator32`
QEMU machine	`myemulator32`
Default RAM	16 MiB
Reset words	`0x400` = SSP, `0x404` = PC
Console MMIO	`0xF0000000`–`0xF000000F`
Console receive IRQ	IRQ4
Timer IRQ	IRQ1

The function ABI and Linux syscall ABI are separate conventions that happen to reuse the same registers. A function receives its first arguments in `r1`–`r4` and returns a 32-bit value in `r1`; a syscall instead puts its number in `r1`, its arguments in `r2`–`r7`, and its result back in `r1`.

Boot

Current QEMU Linux path:

```
reset words
    → head.S
    → vector and stack setup
    → .bss clear
    → start_kernel()
    → MMU, timer, console, block device
    → init
    → shell
```

The future RIKMON firmware path inserts firmware and an ELF loader before the kernel. It must not be confused with the current direct QEMU development loader.

Development

```
source
    → assembler
    → object file
    → linker
    → ELF32 executable
    → Linux process
```

First checks

```
./toolchain/scripts/test-binutils.sh
python3 toolchain/scripts/test-linux-process.py
python3 toolchain/scripts/test-linux-minilibc.py
```

REM Reference Alignment — Second Pass Notes

This edition aligns the book against the current REM reference manuals.

Terminology and status corrections

The following distinctions are enforced:

- REM / MyEmulator2 refers to the current 32-bit target, not the legacy MyEmulator firmware target.
- Implemented features are separated from planned features.
- Host QEMU validation and Android Fold deployment validation are treated separately.
- Toolchain availability does not imply a complete in-guest development environment.

Current implementation baseline

The reference manual defines the current target as:

- 32-bit little-endian byte-addressed CPU.
- Fixed-width 32-bit instructions.
- ELF32 little-endian static executable workflow.
- GNU assembler/linker support.
- GCC integer and soft-float backend support.
- Linux entry, user ELF loading and process lifecycle support.

Known boundaries to preserve

The book must not imply that the following are complete:

- full Linux syscall coverage;
- dynamic linking;
- shared libraries;
- dynamic TLS;
- complete libc/POSIX coverage;
- final userspace ABI details;
- networking;
- complete in-guest package/development environment.

Fold deployment wording

Galaxy Fold execution remains a deployment target requiring device validation. Host QEMU success is not equivalent to phone execution.

REM Reality Table

Area	Current status
CPU architecture	Implemented and under active development
Linux kernel support	Implemented/tested according to project status
Toolchain	Available in the REM environment
Host QEMU	Development and regression environment
Galaxy Fold deployment	Requires device validation
Native applications	Development phase
REMEDIT	Example development environment project

This review draft distinguishes completed capabilities, tested capabilities, and planned work.

Canonical REM conventions for the examples

The detailed chapters below retain the original book's worked examples, but current REM conventions take precedence wherever an older example uses different names or registers.

The target is a 32-bit little-endian architecture using ELF32 static executables. The ordinary function ABI uses `r0` as constant zero, `r1`–`r4` for arguments, `r1` for a 32-bit return value, `r13` as the stack pointer, and `r14` as the link register. The stack is downward growing and 16-byte aligned at call boundaries.

Linux syscalls use a separate userspace convention: `r1` contains the syscall number and return value, while arguments use `r2`–`r7`. The currently exercised process path uses `clone`, `execve`, and `wait4`; the complete syscall ABI is still under development. Applications using the hosted C environment should use the project's statically linked MyEmulator2 musl toolchain. Dynamic linking and complete POSIX coverage remain future work.

When a chapter says a facility is planned, that does not make it a requirement for the first REMEDIT milestone. Host QEMU validation and Galaxy Fold deployment must be recorded separately.

How to use this offline edition

This book contains examples from earlier REM development stages. Before copying an example, identify its target:

Target	Tools and execution model
Legacy MyEmulator 1.0	`tools/myasm`, raw firmware image, separate 8-bit/16-bit architecture
MyEmulator2 bare metal	`myemulator2-elf-as`, `myemulator2-elf-ld`, ELF32 or derived raw image
MyEmulator2 Linux	MyEmulator2 GCC/musl or minilibc, Linux ELF, QEMU Linux machine

The current MyEmulator2 assembler uses GNU syntax: lowercase mnemonics are canonical, registers are `r0` through `r15`, memory operands use `displacement(base)`, and `#` begins a comment. Older examples using names such as `MOV`, `LOAD`, `STORE`, or bracketed addresses are explanatory pseudocode unless the surrounding chapter identifies a legacy target.

The authoritative local references are:

```
docs/MYEMULATOR2_ASSEMBLY_PROGRAMMING_MANUAL.md
docs/MYEMULATOR_2_ABI.md
docs/MYEMULATOR_2_ARCHITECTURE.md
docs/MYEMULATOR_2_LINUX_PORT.md
toolchain/README.md
docs/console.md
android/README.md
```

When a result matters, record the command used, the target being tested, and whether the result came from host QEMU or a physical Fold.

Historical development path

REM did not begin as a finished 32-bit Linux architecture. Its development path matters because the repository still contains tools and examples from each stage:

Flow: Python 8-bit computer → emulated CPU and firmware → MyEmulator2 32-bit QEMU target → ELF32 assembler, linker, and GCC → Linux kernel and userspace bring-up

Historical examples are useful for understanding why a subsystem exists, but they are not automatically current implementation instructions. The target table and the legacy conversion appendix identify which examples can be assembled for MyEmulator2 today.

Part 0 — Architecture and Platform Reference

The chapters that follow teach software construction. This part fixes the machine model those chapters assume. It is a map of the layers, not a second instruction manual. The architecture and assembly manuals remain the final authority for encoding and instruction behavior.

REM Architecture Reference

MyEmulator2 is a 32-bit, little-endian, byte-addressed architecture. Values, addresses, and instruction words are 32 bits. Instructions are fixed-width, stored little-endian, and 4-byte aligned. Sequential execution advances the program counter by four.

Programmer-visible state

State	Meaning
`r0` / `zero`	hardwired zero; writes are discarded
`r1`–`r12`	general registers
`r13` / `sp`	banked user or supervisor stack pointer
`r14` / `lr`	link register used by `jal`
`r15`	general temporary register
`PC`	program counter
`SR`	carry, overflow, interrupt priority, and privilege state

There are User and Supervisor modes. `SR.S` selects the mode. `r13` refers to the user stack pointer in User mode and the supervisor stack pointer in Supervisor mode. Supervisor software can access both through system-register operations; User software cannot directly access the supervisor stack.

The initial function ABI uses `r1`–`r4` for the first four arguments, `r1` for a 32-bit return value, `r5`–`r12` as callee-saved registers, `r15` as a caller-saved temporary, and `r14` as the link register. The stack grows downward, is 16-byte aligned at call boundaries, and has no red zone.

Exceptions and interrupts

The CPU has vectors for illegal instructions, privilege violations, alignment faults, page-not-present and protection faults, divide by zero, arithmetic overflow, system calls, breakpoints, NMI, double fault, and IRQ1–IRQ7. Hardware builds an exception frame on the supervisor stack. Linux expands that into its software `pt_regs` frame before calling C code. General registers are not automatically preserved by hardware handlers.

The timer uses IRQ1. The current console receive path uses IRQ4. Interrupt priority is controlled by `SR.IPL`; priority 7 masks ordinary IRQs.

MMU model

With `MMCR.EN=0`, virtual addresses are physical addresses and page permissions do not apply. With the MMU enabled, translation uses a two-level, 4 KiB page walk:

```
virtual address
31             22 21             12 11                0
+----------------+----------------+--------------------+
| page directory | page table     | page offset        |
| index (10)     | index (10)     | (12)               |
+----------------+----------------+--------------------+
        |                |
        v                v
     PDE table  ------> PTE ------> physical 4 KiB page
```

`PTBR` points to a physical, 4 KiB-aligned 1024-entry page directory. PDE and PTE entries carry present, user, read, write, execute, accessed, and dirty bits. Effective permissions are the intersection of the

directory and table entries. A missing entry produces a not-present fault; an insufficient permission produces a protection fault. The MMU sets accessed and dirty bits on successful accesses. There are no huge pages or legacy paging modes.

MyEmulator2 is strongly ordered. Ordinary loads and stores become visible in program order. `CAS` is a naturally aligned, fully ordered 32-bit atomic compare-and-swap. There are no fence instructions in the current ISA.

ELF and deployment boundary

The canonical object and executable format is ELF32 little-endian with the project-local provisional machine identifier. Static executables are the initial ABI. Dynamic linking, shared libraries, dynamic TLS, and final aggregate/varargs ABI rules remain future work. A raw image produced by `objcopy -O binary` is a derived deployment artifact; it does not contain the ELF entry point.

REM Linux Boot Sequence

The current Linux boot path begins with QEMU publishing the initial stack and program counter through physical reset words at `0x400` and `0x404`. The CPU starts in Supervisor mode with translation disabled and fetches the first instruction physically.

The tracked sequence is:

```
reset words at 0x400 / 0x404
        |
        v
linux/arch/myemulator2/kernel/head.S:_start
        |
        +-- select the linked Linux stack
        +-- install the vector table in VBR
        +-- clear .bss
        v
myemulator2_start_kernel()
        |
        v
start_kernel()
        |
        +-- describe RAM to memblock
        +-- construct page tables and enable the MMU
        +-- register timer, block, and serial console
        +-- create the initial userspace process
        v
/sbin/init and the guest shell
```

The kernel is linked with a 4 KiB page size. The current machine normally has 16 MiB of RAM, while the kernel reads the configured RAM extent from the boot descriptor at `0x408`. The exact initramfs, root filesystem, and command line are deployment inputs, not CPU reset behavior.

ISA, Platform, Kernel, and Userspace

Keep these boundaries separate:

```
+------------------------------+
| REM applications             |
+------------------------------+
| static musl / minilibc       |
+------------------------------+
| Linux syscall and tty ABI    |
+------------------------------+
| Linux kernel                 |
+------------------------------+
| REM platform: console, timer,|
| block device, QEMU machine   |
+------------------------------+
| MyEmulator2 CPU ISA and MMU  |
+------------------------------+
```

An instruction belongs to the ISA. The `0xF0000000` console belongs to the platform. `termios`, ELF process startup, and syscall numbers belong to Linux userspace. QEMU emulates the platform; it does not turn an unimplemented guest feature into an ISA feature.

What Is Implemented, Planned, or Unspecified

The current implementation includes the 32-bit CPU/MMU, exception integration, ELF32 static toolchain, GCC integer and soft-float backend, Linux entry and user ELF loading, basic process life-cycle, serial console, timer, and ext4 block-device work. The syscall table, signals, complete libc/POSIX behavior, dynamic linking, networking, and a stable userspace ABI remain incomplete.

SMP is disabled in the current Linux configuration. USB, sound, networking, and virtual graphics are disabled in the current Android/QEMU deployment. Features not mentioned in the architecture contract should be treated as unspecified rather than assumed absent or present.

Developer Journeys

Bare-metal assembly

1. Build the `myemulator2-elf` tools.
2. Assemble and link an ELF32 program.
3. Inspect it with `readelf` and `objdump`.
4. Run it under the MyEmulator2 QEMU machine.
5. Add a focused regression test.

Linux application development

1. Build or install the MyEmulator2 GCC and static musl/minilibc path.
2. Produce a static ELF executable.
3. Boot the verified Linux kernel and root filesystem.
4. Exercise the program under host QEMU.
5. Repeat the deployment and persistence checks on the Fold separately.

The smallest maintained Linux C milestone is the minilibc regression. It builds `toolchain/examples/linux-minilibc.c` with the target compiler, links the REM startup and syscall wrappers, boots a disposable initramfs, and expects the strings `C minilibc reached` and `C syscall wrappers reached`. Run it with:

```
python3 toolchain/scripts/test-linux-minilibc.py
```

This is the offline equivalent of a first "Hello Linux" program. The larger static musl regression uses `toolchain/examples/linux-libc-regression.c` and exercises files, allocation, strings, time, and platform reporting.

Emulator and kernel development

1. Change one ISA, MMU, device, or kernel component.
2. Run the narrowest existing regression.
3. Run the CPU/system or Linux process suite.
4. Inspect QEMU traces and structured debug state when needed.
5. Record whether the result is host-only, guest-tested, or phone-tested.

Preface

Why this book exists

When building a new computer architecture, there is a natural progression.

The first milestone is making the processor execute instructions.

The second milestone is making the processor run an operating system.

The third milestone is making the machine useful.

That final step is where software developers enter the picture.

A CPU manual can tell you everything about the machine:

- the registers
- the instruction set
- the memory model
- the ABI
- the assembler syntax
- the system call interface

Those documents answer the question:

> "What can this computer do?"

They do not answer the question:

> "How do I build software for this computer?"

That is the purpose of this book.

The difference between a CPU manual and a programming environment

A CPU manual is written from the perspective of the hardware.

It describes the machine as the processor sees it:

```
Registers
Instructions
Memory
Exceptions
Interrupts
System calls
```

A programmer thinks differently.

A programmer needs to know:

```
How do I structure a program?

How do I debug it?

How do I handle input?

How do I display output?
```

```
How do I organise a large assembly project?

How do I build tools that make the next program easier?
```

There is a large gap between these two views.

A processor can execute instructions perfectly and still be a difficult computer to program.

The difference between an interesting CPU design and a usable computer is the software environment built around it.

This book exists to document that environment.

What REM already provides

REM has already passed the stage where every program must directly control the hardware.

The original challenge was proving that a new instruction set architecture could support a modern software environment.

The result is not a toy interpreter or a limited demonstration system.

REM provides a growing Linux application environment, with the verified subset identified in the Linux port documentation:

Flow: Application programs → Libraries → static musl or minilibc → Linux kernel → REM CPU architecture

The programmer working on REM is not starting from bare metal.

The machine already provides:

- a Linux kernel environment
- an executable format
- an assembler and linker toolchain
- a static C runtime path
- selected standard-library functionality
- the tested subset of Linux process and file operations

This changes the nature of the problem.

The question is no longer:

> "Can REM run code?"

The question becomes:

> "How do we write good software on REM?"

What the application programmer still needs to build

The existing REM documentation explains the machine.

This book explains the software engineering required to use that machine effectively.

The missing knowledge is the practical layer:

- how to organise a large assembly project
- how to divide software into modules
- how to test low-level components
- how to debug failures

- how to interact with the terminal
- how editors and development tools are actually built
- how to create software that makes future software easier to write

A programmer who has only the instruction manual can write a program.

A programmer with this book can build an environment.

The intended reader

This book assumes that the reader already understands programming.

It is not an introduction to:

- variables
- loops
- pointers
- basic C syntax
- assembly language concepts
- Makefiles
- compilation

The intended reader is someone who has written software before and is comfortable looking at assembly code.

However, REM introduces areas where even an experienced programmer may need guidance.

For example:

A programmer who has used `ncurses` may know how to create a terminal application:

```
move(10,20);
printw("Hello");
refresh();
```

But ncurses hides a significant amount of machinery:

- terminal modes
- cursor positioning
- escape sequences
- keyboard decoding
- screen updates

When writing a native REM application, some of those layers become visible.

This book explains those layers.

How to use this book alongside the REM manuals

The REM documentation is divided by purpose.

The architecture documentation remains the authoritative source for:

- instruction behaviour
- register definitions
- ABI details
- assembler syntax
- system calls
- CPU operation

This book deliberately does not duplicate that information.

Instead, it assumes those manuals are available and concentrates on the engineering decisions that happen above them.

A useful mental model is:

```
REM Architecture Manual
        |
        |  "How does the machine work?"
        |
        v
Inside the REM Computer
        |
        |  "How do I build useful software?"
        |
        v
Applications and tools
```

When a question is about an instruction, consult the architecture documentation.

When a question is:

> "How should I design this program?"

or:

> "How do I build a tool that other programmers can use?"

this book is the guide.

The philosophy of this book

The examples in this book follow one principle:

Build small things that prove assumptions.

A common mistake in systems programming is to begin with the final application.

For example, when building a text editor, it is tempting to immediately write:

- the screen handling
- the menus
- the file system
- the command system

That creates a large number of unknowns.

A better approach is:

```
Terminal test
      |
Keyboard test
      |
File test
      |
Buffer test
      |
Editor
```

Each layer proves the layer beneath it.

This approach is slower for the first hour and much faster over the lifetime of the project.

The example project

The running example throughout this book is REMEDIT: a native REM text editor and development environment.

The choice of an editor is deliberate.

A text editor touches almost every important area of systems programming:

- terminal handling
- keyboard input
- memory management
- file I/O
- data structures
- rendering
- process control
- integration with the toolchain

Building an editor is not about creating another text editor.

It is about creating a practical test of whether REM is a usable computer platform.

By the end of this book, the goal is not simply to have an editor.

The goal is to have a development environment that makes building the next REM application easier.

Part I — Becoming Productive on REM

Chapter 1 — Welcome to REM Application Development

1.1 The machine is finished; now build the ecosystem

There is a particular point in the life of any computer architecture where the nature of the work changes.

At the beginning, everything is about the processor.

Can it execute instructions?

Can it access memory?

Can it handle exceptions?

Can it boot an operating system?

These are architecture questions.

They are the questions answered by the REM architecture documentation.

However, once a processor can execute a real operating system and run normal programs, the interesting problems move upward.

The question changes from:

> "Can this CPU run software?"

to:

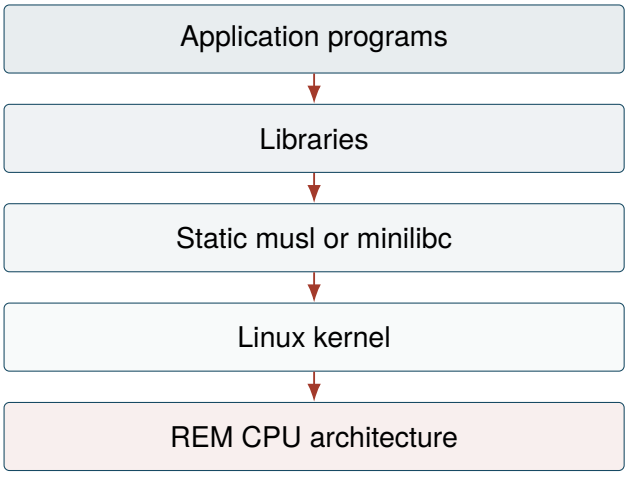
> "Can a programmer comfortably build software for this CPU?"

This is the point where REM has arrived.

The CPU is no longer the project.

The computer is the project.

A useful way to think about the REM software stack is:



Each layer exists to make the layer above it easier to use.

The CPU provides the fundamental operations.

The kernel provides the operating environment.

The C library provides common functionality.

Applications build on top of those foundations.

A programmer writing REM applications should normally think at the top of this stack.

You are not writing every instruction that controls the machine.

You are writing software that uses the machine.

That distinction is important.

1.2 Moving from CPU development to application development

When designing a processor, it is natural to focus on the lowest level.

You think about:

- registers
- instruction formats
- memory access
- pipeline behaviour
- exceptions
- privilege levels

These are the things that make a processor a processor.

Application development has a different set of concerns.

A programmer writing a text editor, compiler, database, or network service is more concerned with:

- How do I organise this program?
- How do I handle input?
- How do I store data?
- How do I recover from errors?
- How do I debug failures?
- How do I make this maintainable?

The processor matters, but it becomes the foundation rather than the focus.

A useful comparison is building a house.

The CPU is the foundation.

The operating system is the structure.

The libraries are the utilities.

The application is the house.

A good foundation is essential, but nobody lives in a foundation.

1.3 What you already have

One of the easiest mistakes when working with a new architecture is assuming that everything must be rebuilt.

That is true when bringing up a bare processor.

It is not true when building applications on a working Linux platform.

REM already has the important components required for normal software development.

The assembler: GAS

The GNU assembler is the first step in the software creation process.

It converts human-readable assembly source:

`source.s`

into machine code contained inside an object file:

`source.o`

The assembler understands:

- REM instruction syntax
- labels
- sections
- symbols
- relocations

The assembler answers the question:

> "How do I express a program in the REM instruction set?"

The linker: LD

The assembler does not create the final executable.

It creates pieces.

The linker combines those pieces.

For example:

Flow: main.o\terminal.o\buffer.o\library.o → linker → REM executable

The linker resolves:

- symbol references
- memory layout
- executable format
- library dependencies

Understanding the linker is one of the major steps from writing small programs to building real applications.

GCC backend

Although this book focuses heavily on assembly programming, REM also has compiler support.

The compiler provides another way of producing code for REM.

This matters because not every problem benefits from assembly.

A practical systems programmer chooses the right tool.

Assembly is excellent for:

- understanding the architecture
- performance-critical routines

- hardware interfaces
- low-level experimentation

A higher-level language is often better for:

- large amounts of application logic
- data processing
- user interfaces
- complex algorithms

A computer architecture becomes much more useful when it supports multiple ways of creating software.

---

libgcc

The compiler runtime provides low-level support functions required by generated code.

These routines handle operations that may not directly exist as single CPU instructions.

Examples include:

- arithmetic helpers
- compiler-generated support routines
- runtime operations

The important point is that application developers do not normally write these themselves.

They are part of the platform.

---

musl libc

The C library is where the Linux programming environment becomes familiar.

Instead of manually implementing common operations, applications can use standard interfaces.

Examples:

Memory:

```
malloc()
free()
memcpy()
```

Files:

```
open()
read()
write()
close()
```

Output:

```
printf()
```

Processes:

```
clone()
execve()
wait4()
```

The verified static musl/minilibc environment exposes the supported subset of these operations. The names and availability of other POSIX interfaces must be checked against the current Linux port.

The existence of a hosted C path is a major milestone for REM.

It means selected applications can use familiar Unix interfaces while the remaining syscall, libc, signal, and userspace ABI work continues.

BusyBox and the shell environment

A computer becomes useful when it can run programs that help create other programs.

The shell provides the interactive environment:

```
$ command
```

Utilities provide the basic tools:

- file management
- text processing
- scripting support
- system inspection

The exact set of available utilities may change as the REM environment develops, but the important concept remains:

The programmer should use existing tools wherever possible.

Do not spend a flight writing a replacement for `grep` unless writing `grep` is the objective.

1.4 What you do not need to rebuild

When working on a new CPU, there is a temptation to prove every layer yourself.

That instinct is useful during CPU bring-up.

It becomes a distraction during application development.

REM already has the lower layers.

Do not immediately rewrite:

---

malloc

Memory allocation is a solved problem for applications.

The question an application developer should ask is:

> “What memory does my program own, and when should it be released?”

not:

> “How do I create a heap allocator?”

---

printf

Formatted output is complicated.

It handles:

- number conversion
- formatting rules
- buffering
- locale considerations

If the goal is a terminal application, use the existing implementation.

Do not accidentally turn “write an editor” into “rewrite the C library.”

File handling

Files are a core operating system service.

The application developer needs to understand:

- opening files
- reading data
- writing data
- handling errors

The application developer does not need to recreate:

- filesystems
- block devices
- storage drivers

Process creation

A development environment will eventually need to run programs.

For example:

Flow: Editor → make → assembler → linker → executable

The operating system already provides the mechanism.

The application needs to use the verified process interfaces:

- `clone`;
- `execve`;
- `wait4`;
- pipes only where the current Linux environment provides them.

not redesign process management.

Standard utilities

A Unix-like environment is powerful because tools can be combined.

Before writing a new utility, ask:

> "Is this actually missing?"

or:

> "Am I avoiding learning how to use the existing tool?"

The best programmer is not the one who writes the most code.

It is the one who understands which code does not need to be written.

1.5 The first rule of systems programming

> Build small things that prove assumptions before building large things that hide failures.

This is perhaps the most important habit in low-level software development.

A large program contains many assumptions.

An editor assumes:

- the terminal works
- keyboard input works
- memory allocation works
- files work
- the display logic works
- the command system works

When something fails, the failure could be anywhere.

A small program removes uncertainty.

Instead of trying to debug the whole editor at once, build confidence step by step:

Flow: terminal test → keyboard test → file test → buffer test → editor

Each program answers one question.

Does output work?

Does input work?

Can I store text?

Can I save it?

Only then combine them.

This approach may feel slower initially.

It is faster overall because debugging time grows much faster than code size.

The purpose of this book is to follow that philosophy throughout.

Every major component will begin with a small working example before becoming part of the final REM development environment.

Chapter 2 — Your First REM Development Session

A new architecture creates a particular temptation.

You want to start writing something interesting immediately.

Perhaps a text editor. Perhaps a game. Perhaps a compiler.

Resist that temptation.

The first session on any new platform should not be about building the final application. It should be about removing uncertainty.

The objective is simple:

> Create a repeatable process where source code becomes a running program.

Once that process works, every future project becomes easier.

This chapter creates the foundation:

Flow: source code → assembler → object file → linker → executable / running program

If this pipeline is not reliable, everything built above it will eventually become difficult to debug.

2.1 Creating a project

Start with structure, not code

Small experiments often begin as a single file.

That is acceptable for learning.

It becomes a problem when the project grows.

A common failure mode in assembly projects is:

```
main.s
```

becoming:

```
main.s
|
+-- terminal code
|
+-- file handling
|
+-- editor logic
|
+-- keyboard decoder
|
+-- screen renderer
```

Eventually nobody knows which part of the program owns what.

The solution is to create boundaries early.

A reasonable REM application structure:

`remedit/`

src/ main.s terminal.s keyboard.s buffer.s editor.s

tests/ test_terminal.s test_buffer.s test_keyboard.s

build/ objects and temporary files

Makefile ~~~ ~~~

The exact names do not matter.

The principle does:

- source code has a home
- tests have a home
- generated files have a home

Source layout

A useful assembly project normally separates code by responsibility.

For REMEDIT:

`main.s`

The application entry point.

Responsible for:

- startup
- initialisation
- main loop
- shutdown

It should not contain the entire editor.

`terminal.s`

Responsible for communication with the terminal.

Examples:

`terminal_init()`

`terminal_clear()`

`terminal_move_cursor()`

`terminal_write()`

`terminal_restore()`

The rest of the program should not know about escape sequences.

`keyboard.s`

Responsible for converting terminal input into editor commands.

The terminal sends:

```
ESC [ A
```

The editor wants:

```
KEY_UP
```

That conversion belongs here.

`buffer.s`

Responsible for storing text.

The editor should not know whether the buffer uses:

- arrays
- linked structures
- gap buffers

The buffer module owns that decision.

`editor.s`

Responsible for editor behaviour:

- commands
- cursor movement
- inserting text
- deleting text
- modes

Test layout

Low-level software needs small test programs.

Do not wait until the entire application exists.

Example test layout:

```
tests/
├── test_terminal.s
├── test_buffer.s
└── test_keyboard.s
```

Each test has a focused question:

Test	Question
`test_terminal.s`	Can I initialise, use, and restore the terminal?
`test_buffer.s`	Can I insert, delete, and retrieve text?
`test_keyboard.s`	Can I decode characters, arrows, and function keys?

A test program is not wasted code.

It is documentation.

Six months later, a test program tells you:

"This assumption was important enough that somebody checked it."

Build output

Never mix generated files with source.

Bad:

```
src/
├── main.s
├── main.o
├── editor.o
└── remedit
```

Good:

```
src/
├── main.s
└── editor.s
build/
├── main.o
└── editor.o
bin/
└── remedit
```

This matters because a build directory can be deleted and recreated at any time.

The source directory should contain only things a programmer wrote.

2.2 The first program

The first program should be deliberately boring.

Do not start with:

- editor code
- graphics
- input handling
- complex libraries

The first program answers one question:

> Can I reliably create and execute a REM program?

Step 1 — Write assembly

Create:

```
src/main.s
```

The exact instructions depend on the REM ABI and assembler manual.

The important structure is:

Flow: entry point → perform simple operation → exit cleanly

The first program should:

- start
- perform one observable action
- terminate

For example:

- print text
- return a status code
- write a file

Step 2 — Assemble

The assembler converts:

`main.s`

into:

`main.o`

The object file is not yet a complete program.

It contains:

- machine code
- symbols
- relocation information

Think of it as a compiled component.

Step 3 — Link

The linker combines object files:

Flow: main.o\terminal.o\library objects → linker → executable

The linker resolves references.

For example:

Your code says:

```
call terminal_write
```

The assembler knows:

“There is a symbol called terminal_write.”

The linker knows:

“That symbol is located here.”

Step 4 — Execute

The operating system loads the executable.

This involves:

- reading the executable format
- creating process memory
- setting up the initial state
- transferring control to the entry point

At this point you have completed the entire software chain:

```
text file

assembler

object file

linker

ELF executable

Linux process

REM CPU executes instructions
```

2.3 Understanding the build pipeline

A programmer who only uses an IDE often sees “Build” and assumes it is one operation. It is actually a chain of independent steps:

Flow: source.s → assembler → object file → linker → ELF executable

Each stage can fail independently.

This matters because the error message tells you where to look.

Assembler failure

Example causes:

- invalid instruction
- incorrect operand format
- unknown symbol
- syntax error

Question:

> “Does the source describe a valid REM program?”

Linker failure

Example causes:

- missing object file
- undefined symbol
- incorrect library
- incorrect entry point

Question:

> "Do all the pieces of the program fit together?"

Loader failure

The executable exists, but Linux cannot start it.

Possible causes:

- incorrect executable format
- missing interpreter
- incorrect permissions
- incompatible binary

Question:

> "Can the operating system create this process?"

Runtime failure

The program starts but behaves incorrectly.

Possible causes:

- incorrect register usage
- bad memory access
- syscall mistake
- logic error

Question:

> "The machine ran my code. Did my code do the right thing?"

2.4 The role of Make

Make is not a compiler.

Make is a dependency manager.

It answers:

> "What needs rebuilding?"

A simple project can be understood as this build flow:

Flow: main.s / terminal.s → main.o / terminal.o → remedit

The dependencies are:

Target	Depends on
`remedit`	`main.o`, `terminal.o`
`main.o`	`main.s`
`terminal.o`	`terminal.s`

Targets

A target is something Make can build.

Example:

`all`

`test`

`clean`

A normal workflow:

`make`

Build the application.

`make test`

Build and run tests.

`make clean`

Remove generated files.

Incremental builds

The important feature of Make is that it does not rebuild everything.

If you change:

`terminal.s`

Make understands that:

`terminal.o`

must be rebuilt.

But:

`buffer.o`

does not need to change.

This becomes increasingly important as the project grows.

2.5 The first debugging checklist

When something fails, do not immediately inspect the application.

Find the layer where it failed.

---

1. Did the assembler fail?

Check:

```
source code
instruction syntax
labels
sections
```

The program has not been created yet.

---

2. Did the linker fail?

Check:

```
missing symbols
object files
libraries
entry points
```

The parts exist, but they cannot be combined.

---

3. Did the loader fail?

Check:

```
executable format
permissions
runtime dependencies
```

Linux cannot start the program.

---

4. Did the syscall fail?

Check:

```
arguments
registers
return values
permissions
```

The program reached the operating system, but requested something invalid.

---

5. Did the application fail?

Only now debug:

- algorithms
- data structures
- program logic

A useful rule:

> Always debug from the bottom upward.

Do not debug an editor if the terminal layer is broken.

Do not debug the terminal layer if the program cannot link.

Do not debug the program if the executable cannot start.

A systems programmer's greatest advantage is not knowing every answer.

It is knowing which question to ask first.

Part I-A — The First Hour on REM

Chapter 2A — Your First REM Program

This is the first complete bare-metal checkpoint. It performs a calculation and halts, so it needs no Linux, libc, filesystem, or terminal.

```
.text
.global _start
_start:
        li   r1, 5
        li   r2, 7
        add  r3, r1, r2
        beq  r3, r0, _start
        halt
```

Save it as `minimal.s` and run the target tools:

```
TOOL=.toolchain-install/bin/myemulator2-elf
$TOOL-as minimal.s -o minimal.o
$TOOL-ld minimal.o -o minimal.elf
$TOOL-readelf -h -S minimal.elf
$TOOL-objdump -d minimal.elf
```

Run the ELF under the current QEMU machine:

```
qemu-system-myemulator32 -machine myemulator32 \
  -display none -kernel minimal.elf
```

The program should reach `halt` with `r3 = 12`. If it does not, use the toolchain inspection and QEMU tracing recipes in Appendices G and P before building anything larger.

Chapter 2B — First Linux Syscall

The first Linux userspace milestone is a static assembly program that writes bytes and exits. The complete maintained source is:

```
sed -n '1,140p' toolchain/examples/linux-echo.S
```

Its essential convention is:

```
r1 = syscall number and result
r2-r7 = syscall arguments
```

For `write`, set `r1 = 64`, `r2 = fd`, `r3 = buffer`, and `r4 = length`, then execute `syscall 0`. For `exit_group`, set `r1 = 94` and the exit status in `r2`. The complete Linux boot fixture is exercised by the repository's Linux regression scripts; this is a guest userspace program, not a bare-metal `-kernel` image.

The worked wrapper and retry behavior are in Appendix N.

Part II — Assembly Software Engineering

Chapter 3 — Writing Maintainable Assembly

Assembly language gives the programmer complete control.

That is its greatest strength.

It is also its greatest danger.

A compiler forces certain disciplines onto a programmer:

- variables have types
- functions have signatures
- the compiler checks many mistakes
- the linker reports many inconsistencies

Assembly does not.

The processor does not know that a register contains:

- a pointer
- a counter
- a file descriptor
- a character
- a memory address

To the CPU, everything is simply bits.

The discipline must come from the programmer.

A useful assembly program is not a collection of instructions.

It is a collection of well-defined components with clear responsibilities.

3.1 Assembly is not just instructions

A beginner's assembly program often starts like this:

```
_start:

    instruction
    instruction
    instruction
    instruction
```

This works.

For a small test program, it may even be the correct approach.

The problem appears when the program grows.

A real application contains different types of work:

- terminal handling
- memory management
- file access
- input processing
- user interface

- data structures
- application logic

If all of this exists in one file, every change becomes dangerous.

The question changes from:

> "Can I write these instructions?"

to:

> "Can I change this program six months from now without breaking unrelated parts?"

That is software engineering.

Program structure

A well-designed assembly program has layers.

For REMEDIT:

Flow: editor → commands → buffer / keyboard / terminal → libc / syscalls → Linux kernel

Each layer has a purpose.

The editor should not know how terminal escape sequences work.

The terminal code should not know how text buffers are stored.

The buffer code should not know anything about the screen.

This separation reduces the number of things you need to think about at one time.

Example: a bad design

Imagine the editor loop contains:

```
read keyboard

if escape sequence:
    decode arrow key

move cursor

calculate screen position

write ANSI command

modify text buffer

save file
```

This appears simple because everything is in one place.

The problem appears later.

Suppose the terminal behaviour is wrong.

You now have to search through:

- keyboard code
- cursor code

- rendering code
- file code

The error could be anywhere.

Better design

Separate responsibilities:

```
keyboard.s

    read_key()
    decode_escape_sequence()


terminal.s

    move_cursor()
    clear_screen()
    write_text()


buffer.s

    insert_character()
    delete_character()
    get_line()


editor.s

    process_command()
    update_state()
    redraw()
```

Now a problem has a smaller search area.

3.2 Module design

A module is a boundary.

It defines:

- what other code is allowed to ask it to do
- what details remain private

For REMEDIT, the initial modules might be:

```
terminal.s

buffer.s

keyboard.s

editor.s
```

```
file.s

undo.s

syntax.s
```

The names are not important.

The boundaries are.

---

terminal.s

The terminal module owns communication with the terminal.

Other code should not contain raw escape-byte construction such as:

Flow: ESC → [→ 2 → J

or:

Flow: ESC → [→ 10 → ; → 20 → H

Instead:

```
call terminal_clear

call terminal_move_cursor
```

The terminal module hides the implementation.

If later you change:

- ANSI handling
- terminal size detection
- colour support

the rest of the editor does not need to change.

---

buffer.s

The buffer module owns text storage.

The editor should not care whether the buffer uses:

- an array of lines
- a gap buffer
- a linked structure
- a piece table

The editor asks:

```
insert character

delete character

get line

replace text
```

The buffer decides how.

This is particularly important in assembly because changing data structures can otherwise require changing code everywhere.

---

keyboard.s

The keyboard module converts raw terminal input into meaningful events.

The terminal sends:

```
ESC [ A
```

The editor wants:

```
KEY_UP
```

The conversion belongs in one place.

---

editor.s

The editor module controls application behaviour.

It knows:

- current mode
- cursor position
- current file
- user commands

It should not know:

- how bytes arrive from the keyboard
- how characters are physically displayed
- how memory is allocated

3.3 Function contracts

In a high-level language, a function declaration tells the compiler a lot.

For example:

```c
int insert_character(
    Buffer *buffer,
    int position,
    char value
);
```

The compiler knows:

- parameter count
- parameter types
- return type

Assembly does not have that information.

You must provide it.

Every important function should have a contract.

Example:

```
# buffer_insert_character
#
# Purpose:
#     Insert one character into a text buffer.
#
# Inputs:
#     r1 = buffer address
#     r2 = line number
#     r3 = column number
#     r4 = character
#
# Outputs:
#     r1 = success/failure
#
# Modified registers:
#     r5-r8
#
# Memory ownership:
#     buffer owns allocated storage
#
# Errors:
#     invalid position
#     allocation failure
#
```

This may appear excessive when writing the function.

It becomes invaluable later.

Inputs

Document exactly what enters the function.

Questions:

- Which registers contain arguments?
- Are pointers valid?
- Is memory already allocated?
- Are values ranges checked?

Bad:

```
r1 contains something
```

Good:

```
r1 = pointer to Buffer structure
r2 = zero-based line number
r3 = character position
```

Outputs

Document what the caller receives.

Examples:

Success:

```
r1 = 1
```

Failure:

```
r1 = 0
```

or:

```
r1 = negative error code
```

The important thing is consistency.

Registers modified

This is one of the most important pieces of information in assembly.

A function that silently destroys registers creates difficult bugs.

Example:

Before:

```
r5 = pointer to current file
```

Call function.

After:

```
r5 = temporary calculation
```

The caller fails later.

The problem may appear nowhere near the original mistake.

Document:

```
Preserved:

r1-r4

Destroyed:

r5-r8
```

Memory ownership

Pointers create responsibility.

If a function allocates memory:

Who frees it?

If a function receives a pointer:

Who owns it?

Example:

```
buffer_create()

allocates memory
```

```
ownership:
    Buffer object owns storage

caller must:
    call buffer_destroy()
```

Without ownership rules, assembly programs eventually become impossible to reason about.

3.4 Calling conventions

A calling convention is the agreement between functions.

It answers:

- Where are parameters placed?
- Where is the return value placed?
- Which registers can be changed?
- How is the stack used?

Without a calling convention, every function call becomes a negotiation.

Stack usage

The stack normally stores temporary information:

- return addresses
- local variables
- saved registers
- temporary data

A function should have a predictable stack frame.

Conceptually:

```
Before call:

caller stack


After entry:

+----------------+
| local data     |
+----------------+
| saved registers|
+----------------+
| return address |
+----------------+
```

The exact REM ABI rules belong in the architecture documentation.

The important programming rule is:

> Every function must leave the machine in the state expected by its caller.

Register preservation

A common convention is to divide registers:

Caller-saved

The caller assumes these may change.

Before calling another function:

```
save anything important
call function
restore if needed
```

Callee-saved

The function promises to preserve them.

The function must:

```
save register

use register

restore register

return
```

The exact REM register rules are defined by the ABI.

The programmer must follow them consistently.

A program where every function invents its own rules will eventually fail.

Interoperability with C

One advantage of REM having libc support is that assembly does not have to exist in isolation.

A practical system may contain:

```
C application logic

        |

assembly performance-critical routines

        |

libc
```

For this to work:

- arguments must be passed correctly
- registers must be preserved
- stack rules must be followed

The C compiler and assembler are not separate worlds.

They are two ways of producing code for the same machine.

3.5 Assembly debugging discipline

The fastest assembly programmer is not the one who writes the fewest instructions.

It is the one who finds mistakes quickly.

Reduce failures

When something breaks, reduce the problem.

Bad approach:

```
REMEDIT crashes
```

```
debug 20,000 lines of code
```

Good approach:

```
buffer test fails
```

```
debug 100 lines
```

Isolate assumptions

Every layer has assumptions.

Example:

Terminal:

“write() displays characters.”

Keyboard:

“arrow keys arrive in this format.”

Buffer:

“line pointers remain valid.”

Build:

“linker produces executable.”

Test those assumptions directly.

Build test programs

Before integrating a component:

Create a small program.

Examples:

```
terminal_test

Can I:
    clear screen?
    move cursor?
    print text?

buffer_test

Can I:
    insert?
    delete?
    retrieve?

keyboard_test

Can I:
    decode arrows?
    decode function keys?
```

These programs are not throwaway code.

They become permanent documentation of how the system works.

The assembly programmer's rule

The processor gives you freedom.

The software engineer provides structure.

Good assembly is not clever assembly.

Good assembly is code where another programmer can understand:

- what this module does
- what this function expects
- what it changes
- why it exists

That discipline is what allows a single programmer to build something as large as a complete development environment.

Chapter 4 — Using libc, Syscalls and the Kernel

One of the advantages REM has over a simple CPU demonstration is that application programmers do not need to communicate directly with hardware for every operation.

A modern operating system exists to provide services.

The kernel manages:

- processes
- memory protection
- filesystems
- devices
- networking
- scheduling

The C library exists to provide a convenient interface to those services.

A REM application normally sits above both.

Understanding where each responsibility belongs is one of the most important skills in systems programming.

4.1 The three levels

The relationship between an application, the C library and the kernel can be represented as:

Flow: Application → libc → syscall interface → kernel

Each layer has a different purpose.

Application layer

This is the program you are writing.

Examples:

- REMEDIT
- assembler tools
- system utilities
- network programs

The application should normally describe **what it wants to achieve**, not the low-level mechanism required to achieve it.

For example:

A text editor wants to save a file.

The editor should think:

```
Save document
```

not:

```
Move disk head
Write filesystem blocks
Update metadata
Handle storage controller
```

The operating system exists to hide those details.

---

libc layer

The C library provides a programming interface to common operations.

Instead of every application implementing its own version of:

- memory allocation
- string handling
- formatted output
- file operations

the library provides tested implementations.

Example:

```c
printf("Hello\n");
```

The application does not need to know:

- how characters are converted
- how output is buffered
- how the terminal receives bytes

The library handles those details.

---

syscall layer

A system call is the controlled boundary between an application and the kernel.

An application cannot normally directly perform privileged operations.

For example:

- creating a process
- opening protected files
- allocating protected memory
- communicating with devices

must be requested from the kernel.

The syscall mechanism provides that request path.

Conceptually:

```
Application

    |
    | "Please open this file"

    v
```

```
syscall interface

    |
    | validate request

    v

kernel

    |
    | perform operation

    v

result returned
```

Kernel layer

The kernel is responsible for managing the machine.

It provides the environment that makes multiple programs possible.

Examples:

A program asks:

```
Allocate memory
```

The kernel decides:

```
Which physical pages?
Which virtual addresses?
Which permissions?
```

A program asks:

```
Start another process
```

The kernel decides:

```
Create process structures
Allocate resources
Schedule execution
```

The application does not need to understand the hardware details behind those operations.

4.2 When to use libc

The general rule:

> Use the highest-level interface that solves the problem.

If libc provides what you need, use it.

This makes programs:

- shorter
- easier to understand
- easier to maintain
- less dependent on kernel details

Memory allocation

Example:

```
void *buffer = malloc(size);
```

The application thinks:

“I need some memory.”

The library and kernel handle the details.

Behind the scenes this may involve:

Flow: malloc() → libc allocator → kernel memory request → virtual memory system

The application should normally not care about:

- heap layout
- memory pages
- allocation strategies

unless it is specifically writing a memory allocator.

String handling

Common operations:

```
strlen()

memcpy()

strcmp()

strcpy()
```

These are simple operations, but they contain many opportunities for mistakes.

Using the library gives you:

- consistent behaviour
- tested implementations
- easier code

For example, a text editor will manipulate thousands of strings.

The interesting problem is:

“How should the editor represent text?”

not:

“How do I write memcpy?”

File operations

A normal application uses:

```c
FILE *fp = fopen("file.txt", "r");
```

or:

```c
int fd = open("file.txt", flags);
```

The choice depends on the requirement.

A text editor may eventually want lower-level control:

- file descriptors
- explicit reads
- seeking
- error handling

However, the first question should always be:

“What behaviour do I need?”

not:

“How low can I go?”

Formatted output

Functions such as:

```c
printf()

sprintf()

snprintf()
```

are useful because converting values into text is surprisingly complicated.

Consider:

```c
printf("%08x", value);
```

The library handles:

- number conversion
- padding
- formatting rules

A systems programmer should understand how these work, but does not usually need to rewrite them.

4.3 When to use syscalls

Direct system calls are useful when the normal library interface is not enough.

They are also useful when learning how the system works.

However, they should be deliberate.

A syscall is not automatically better because it is lower level.

Terminal control

Terminal applications are one area where lower-level control becomes useful.

A text editor needs:

- immediate keyboard input
- cursor movement
- screen control

The terminal layer may need direct access to:

- file descriptors
- terminal attributes
- input modes

The application may require behaviour that is below the normal line-oriented interfaces.

Experiments

When bringing up a new architecture, small syscall experiments are valuable.

For example:

“Does writing to stdout work?”

A minimal test:

Flow: application → write syscall → kernel → terminal

This proves a complete path through the system.

Low-level tools

Some programs exist specifically to operate near the system boundary.

Examples:

- debuggers
- profilers
- system monitors
- hardware tools

These often need information or control not exposed through normal libraries.

Do not confuse understanding with implementation

A common mistake when learning a new architecture is:

“I need to understand this, therefore I need to rewrite it.”

Not necessarily.

You should understand:

- how malloc works

without necessarily writing:

- malloc

You should understand:

- how printf eventually writes bytes

without necessarily writing:

- printf

Knowledge and ownership are different things.

4.4 Understanding failures

When something goes wrong, the first question is not:

“What line of my program is broken?”

The first question is:

“Which layer failed?”

The stack is:

Flow: Application → libc → syscall interface → kernel → CPU

The debugging approach depends on the layer.

Application failure

Symptoms:

- wrong output
- incorrect calculations
- bad logic
- corrupted data

Examples:

The editor inserts text incorrectly.

The buffer loses a character.

The search routine returns the wrong location.

The operating environment may be working perfectly.

---

libc failure

Symptoms:

- unexpected library behaviour
- memory errors
- incorrect assumptions about library functions

Questions:

- Was the function used correctly?
- Were parameters valid?

- Was memory already corrupted?

Syscall layer failure

Symptoms:

The program reaches the kernel but the request fails.

Examples:

- incorrect arguments
- incorrect permissions
- invalid addresses
- wrong return value handling

A useful habit:

Always check syscall return values.

Never assume success.

Kernel failure

Symptoms:

Multiple programs fail in the same way.

Examples:

- process creation fails
- filesystem operations fail
- memory handling behaves incorrectly

A kernel problem normally affects more than one application.

CPU failure

The lowest layer.

Symptoms:

- impossible instruction behaviour
- incorrect register values
- memory corruption across unrelated programs
- failures that cannot be explained by software

This is where architecture debugging begins.

Debugging from the bottom upward

When debugging a REM application, work upward:

Flow: CPU → Kernel → Syscalls → libc → Application

Do not immediately blame the CPU.

A bug in a buffer routine can look like memory corruption.

A bad syscall argument can look like a kernel problem.

A terminal mode mistake can look like a display failure.

The skill is identifying the boundary where reality diverges from expectation.

The practical rule

Use the simplest layer that solves the problem.

Use libc for normal application work.

Use syscalls when you need control.

Use direct hardware interaction only when the problem actually requires it.

A good systems programmer understands every layer.

A good application programmer knows which layers they do not need to touch.

Chapter 4A — Writing a Syscall Wrapper

The function ABI and Linux syscall ABI use the same registers for different purposes. A wrapper must translate between them deliberately.

```
function wrapper arguments: r1-r4
Linux syscall number:       r1
Linux syscall arguments:    r2-r7
Linux syscall result:       r1
```

For example, a three-argument `write(fd, buffer, length)` wrapper receives the function arguments in `r1`, `r2`, and `r3`, moves them to syscall arguments `r2`, `r3`, and `r4`, loads syscall number 64 into `r1`, and invokes the syscall:

```
.text
.global rem_write
.type rem_write, @function
rem_write:
        add  r4, r3, r0       # length
        add  r3, r2, r0       # buffer
        add  r2, r1, r0       # file descriptor
        li   r1, 64           # write
        syscall 0
        ret                   # result remains in r1
```

The wrapper must preserve any callee-saved registers it uses. Treat the syscall argument/result registers as clobbered. A negative result is an error; it must not be interpreted as a successful byte count. The maintained wrappers in `toolchain/userspace/minilibc/syscall.S` show the same pattern for `read`, `openat`, `close`, `renameat`, and other currently exercised calls.

Chapter 4B — The C Runtime Boundary

A C program does not begin at `main`. The process enters at the ELF entry symbol, normally `_start`, and startup code prepares the C environment before calling the application.

Flow: Linux ELF loader → _start → libc or minilibc startup → main(argc, argv, envp) → exit_group(status)

The current minilibc `crt0-linux.S` is deliberately smaller: it calls `main`, moves the return value into the exit argument, and invokes `exit_group`. It does not promise a complete `argc/argv/envp` or `__libc_start_main` contract. Use the static musl startup path for applications that require the full hosted C interface supported by the current port.

The reproducible minilibc build and its disposable Linux boot test are in Appendix O and `toolchain/scripts/test-linux-minilibc.py`.

Part III — Terminal Programming (The Missing Knowledge)

Chapter 5 — The Terminal Model

A terminal is one of the biggest conceptual gaps for programmers who have always worked through libraries such as `ncurses`.

The reason is simple:

`ncurses` works very hard to make the terminal disappear.

It gives the programmer the impression that they have a screen:

```
move(10,20);
printw("Hello");
refresh();
```

It feels like a graphics API.

It is not.

A terminal application is fundamentally a program communicating through a stream of bytes.

Once you remove the library layer, the programmer must understand what is actually happening.

This chapter builds that mental model.

5.1 The mistake: thinking a terminal is a screen

The most common mistake is thinking:

Flow: Application → terminal interface → Screen

This is not how terminal applications work.

There is no framebuffer.

There is no memory location containing:

`character at row 10, column 20`

The application does not draw.

It sends instructions.

The real model is:

Flow: Application → bytes → Terminal emulator / pixels on display

The application sends characters and control sequences.

The terminal emulator interprets those bytes and decides what should appear.

The terminal emulator

A terminal emulator is a program that behaves like an old physical terminal.

Historically, a computer was connected to a separate hardware terminal:

Flow: Computer → serial cable → terminal → keyboard + screen

The terminal accepted character streams and displayed them.

Modern systems replace the physical terminal with software:

```
Terminal emulator
```

Examples:

- xterm
- GNOME Terminal
- Konsole
- terminal applications on Android

The concept is the same.

Your program is not talking directly to pixels.

It is talking to a terminal.

Character streams

A normal program writes output:

```
write(1, "Hello", 5);
```

The program has written five bytes.

It has not:

- moved a cursor
- changed a screen pixel
- drawn text

It has only produced:

```
H e l l o
```

The terminal receives those bytes and interprets them.

For ordinary characters:

```
H
```

means:

“Display the letter H at the current cursor position.”

But special sequences have different meanings.

For example:

```
ESC [ 2 J
```

means:

“Clear the display.”

Escape sequences

Terminal control is performed using special byte sequences.

The most common form is:

```
ESC [
```

This is called a CSI sequence:

```
Control Sequence Introducer
```

The terminal sees:

```
ESC [ command
```

and interprets the command.

Examples:

Clear screen:

```
ESC [ 2 J
```

Move cursor:

```
ESC [ row ; column H
```

Hide cursor:

```
ESC [ ? 25 l
```

Show cursor:

```
ESC [ ? 25 h
```

The exact byte values are:

```
ESC = 0x1B
```

So:

```
ESC [ 2 J
```

is actually:

```
1B 5B 32 4A
```

A terminal program is therefore a protocol implementation.

It sends the correct byte sequences and the terminal emulator responds.

5.2 The data path

The complete path is:

Flow: Application → write() / stdout → Linux terminal driver → Terminal emulator → Pixels

Each layer has a responsibility.

Application

Your program decides:

- what should be displayed
- where the cursor should move
- what colour should be used

Example:

```
Put cursor at line 10 column 20

Print "Hello"

Move cursor back
```

---

stdout

Programs normally write to file descriptor 1:

`stdout`

It does not matter whether that output goes to:

- a terminal
- a file
- another program through a pipe

The program is simply writing bytes.

Example:

```
program > output.txt
```

The same bytes that would appear on the terminal are now stored in a file.

Terminal driver

The Linux kernel provides the terminal interface.

It handles:

- input buffering
- terminal modes
- special character processing

This is where concepts such as:

- canonical mode
- raw mode
- echo

exist.

The kernel sits between your program and the terminal device.

Terminal emulator

The terminal emulator interprets output.

It understands:

`ESC [ 10 ; 20 H`

and changes the cursor position.

It understands:

`ESC [ 31 m`

and changes the text colour.

It converts the character stream into visible output.

Pixels

Eventually the user sees:

`Hello`

but the application never directly created those pixels.

The terminal emulator did.

5.3 What ncurses did for you

A programmer using `ncurses` normally works at a much higher level.

The library hides several difficult areas.

Understanding those hidden layers is essential when building REMEDIT.

Cursor movement

With ncurses:

```
move(10,20);
```

The library determines the required terminal command.

Without ncurses:

You send:

`ESC [ 10 ; 20 H`

Your terminal module should provide:

`terminal_move_cursor(row,column)`

The rest of the editor should not know how that works.

Screen buffering

A beginner implementation might do:

```
clear screen

draw everything again
```

This works.

It is also inefficient.

A real editor tracks:

- what changed
- what needs redrawing
- where the cursor should be

Ncurses maintains a virtual screen.

It compares:

```
desired screen

against

current screen
```

and sends only the necessary updates.

REMEDIT may initially use a simpler approach, but understanding the principle matters.

Keyboard decoding

The keyboard is another area hidden by ncurses.

The programmer sees:

```
KEY_UP
```

but the terminal actually receives bytes.

For example:

Arrow up:

```
ESC
[
A
```

The library converts:

```
ESC [ A
```

into:

```
KEY_UP
```

Your keyboard module will need to perform this job.

Terminal modes

A shell and an editor want different behaviour.

The shell wants:

```
type command
press Enter
execute
```

An editor wants:

```
press key
respond immediately
```

The terminal has modes controlling this behaviour.

Ncurses manages those modes.

REMEDIT must eventually manage them itself.

5.4 Building a minimal terminal library

The editor should not directly contain terminal commands.

Avoid:

```
editor.s

write ESC sequence

move cursor

write another ESC sequence
```

This spreads terminal knowledge everywhere.

Instead create a terminal abstraction.

Example:

```
terminal.s
```

provides:

```
terminal_init()

terminal_restore()

terminal_clear()

terminal_move()

terminal_write()
```

The editor calls:

```
terminal_move(10,20)
```

not:

```
send bytes:
ESC [ 10 ; 20 H
```

---

terminal_init()

Responsible for preparing the terminal.

Future responsibilities may include:

- saving original settings
- enabling raw input
- disabling echo
- configuring behaviour

---

terminal_restore()

This is one of the most important functions.

A program that crashes while the terminal is in a modified state can leave the user's terminal unusable.

Symptoms:

- typed characters do not appear
- Enter behaves strangely
- commands look broken

A good terminal application always restores the original state before exiting.

---

terminal_clear()

A simple wrapper around:

```
ESC [ 2 J
```

Possible implementation:

```
write:
    ESC
    [
    2
    J
```

The rest of the editor does not need to know the sequence.

---

terminal_move()

Responsible for cursor positioning.

Example:

```
terminal_move(5,10)
```

internally produces:

```
ESC [ 5 ; 10 H
```

---

terminal_write()

A controlled output path.

The editor should normally not scatter direct writes everywhere.

A central output function makes it easier later to add:

- buffering
- optimisation
- debugging output
- screen tracking

The first terminal test program

Before writing an editor, write a terminal test.

The purpose is not to build something useful.

The purpose is to answer:

“Do I understand the terminal?”

Test:

1. Can I clear the screen?
2. Can I move the cursor?
3. Can I write text at a position?
4. Can I change colour?
5. Can I restore the terminal?

Only after these work should the editor depend on the terminal layer.

The terminal is not a minor detail of an editor.

It is the foundation on which the entire user interface is built.

Chapter 6 — Terminal Modes and termios

The previous chapter introduced the idea that a terminal is not a screen.

It is a communication channel.

Your program writes bytes.

The terminal emulator interprets those bytes.

The same principle applies to input.

Your keyboard is not directly connected to your application.

The path is:

Flow: Keyboard → Terminal emulator → Linux terminal driver → Your program

The Linux terminal driver sits between the user and the application.

Its job is to make terminal input useful for normal programs.

For a shell, the default behaviour is excellent.

For an editor, it is usually wrong.

Understanding why requires understanding terminal modes.

6.1 Canonical mode

Canonical mode is the normal mode used by command-line programs.

It is designed around the idea of entering a complete line of text.

Consider a shell:

```
$ ls -la
```

The shell does not want to receive every individual keystroke.

When the user types:

```
l
s
(space)
-------

l
a
ENTER
```

the shell normally receives:

```
ls -la
```

as one unit.

The terminal driver collects the characters first.

The application does not see them immediately.

The flow is:

Flow: Keyboard → Terminal driver (waits for ENTER) → Application receives line

This is canonical input.

Why shells want canonical mode

Imagine writing a shell without terminal processing.

Every command would need to implement:

- line editing
- backspace handling
- delete handling
- history
- cursor movement
- command recall

The terminal driver provides a basic layer.

For example:

The user types:

```
abc
```

then presses backspace.

The terminal driver can handle:

```
remove previous character
```

before the shell ever receives the input.

The shell simply receives:

```
ab
```

This keeps simple programs simple.

The problem for editors

A text editor cannot work this way.

Imagine pressing the left arrow key.

The editor needs to know immediately.

The desired behaviour is:

Flow: User presses LEFT → Editor receives event → Cursor moves left

Canonical mode gives:

Flow: User presses LEFT → Terminal waits → Editor receives nothing

The editor cannot respond because the terminal driver is buffering input.

6.2 Raw mode

Raw mode reverses this behaviour.

The terminal driver stops trying to interpret input.

The application receives characters immediately.

The flow becomes:

Flow: Keyboard → Terminal driver → Application immediately

If the user presses:

```
a
```

the application receives:

```
0x61
```

immediately.

No Enter key is required.

Why editors need raw mode

An editor needs control over individual keystrokes.

Examples:

Character entry:

```
a
```

becomes:

```
insert 'a'
```

Arrow key:

```
ESC [ A
```

becomes:

```
move cursor up
```

Function key:

```
F5
```

becomes:

```
run build command
```

None of this works well with line buffering.

The editor needs the keyboard stream, not completed lines.

Raw does not mean “no processing at all”

The name is slightly misleading.

Raw mode does not mean:

“the hardware is connected directly to your program.”

The kernel still provides the terminal interface.

It simply disables many automatic behaviours.

A program can choose which processing it wants.

The important terminal settings are controlled through:

`termios`

6.3 The termios structure

Linux represents terminal settings using a structure:

```
struct termios {
    tcflag_t c_iflag;
    tcflag_t c_oflag;
    tcflag_t c_cflag;
    tcflag_t c_lflag;
    cc_t c_cc[NCCS];
};
```

The fields control different aspects of terminal behaviour.

For an editor, the important area is usually:

`c_lflag`

The local flags.

These control things such as:

- canonical input
- echo
- signal handling

ICANON — canonical input

When enabled:

`ICANON = ON`

the terminal waits for a complete line.

Example:

User types:

```
hello
```

Application receives:

```
hello
```

after Enter.

When disabled:

```
ICANON = OFF
```

the application receives:

```
h
e
l
l
o
```

as the characters arrive.

An editor normally disables ICANON.

6.4 Echo handling

Echo is another feature that is useful for shells but wrong for editors.

Normally:

```
Keyboard

   |

   v

Terminal driver

   |

   +-------> screen
   |
   +-------> application
```

The terminal displays characters automatically.

The user types:

```
hello
```

and sees:

```
hello
```

even before the application does anything.

This is echo.

Why echo breaks editors

An editor wants to control the display.

The editor decides:

- where the character appears
- whether syntax highlighting changes the colour
- whether the line needs redrawing

If echo remains enabled:

```
User presses A

terminal displays A

editor displays A
```

Result:

```
AA
```

The character appears twice.

Therefore an editor disables echo.

The editor becomes responsible for drawing the character.

ECHO flag

The terminal setting:

```
ECHO
```

controls this behaviour.

Enabled:

```
keyboard
    |
    v
terminal displays character
```

Disabled:

```
keyboard
    |
    v
application decides what happens
```

For REMEDIT:

```
ICANON = OFF
ECHO   = OFF
```

are two of the fundamental settings.

6.5 Signals

The terminal driver also handles special control characters.

These are useful in a shell.

They can be inconvenient in an editor.

Common examples:

Ctrl-C

Normally:

`Ctrl-C`

generates:

`SIGINT`

The kernel sends an interrupt signal to the foreground process.

For a shell:

```
run program
```

`Ctrl-C`

```
terminate program
```

is exactly what users expect.

For an editor:

The editor may want:

`Ctrl-C = cancel current operation`

rather than immediate termination.

Ctrl-Z

Normally:

`Ctrl-Z`

generates:

`SIGTSTP`

which suspends the process.

A shell uses this for job control:

```
run program
```

`Ctrl-Z`

```
return to shell
```

An editor may want a different meaning.

Ctrl-

Normally:

`Ctrl-\`

generates:

`SIGQUIT`

This can terminate a program and produce a core dump.

Useful for debugging.

Potentially surprising for an interactive editor.

The editor decision

A mature editor makes deliberate choices.

Questions:

Should Ctrl-C:

- terminate?
- cancel?
- copy?

Should Ctrl-Z:

- suspend?
- undo?

There is no universal answer.

The important point is:

The terminal driver has defaults.

The application chooses whether those defaults are appropriate.

6.6 Saving and restoring terminal state

This is one of the most important rules in terminal programming.

A terminal is shared state.

Your program changes it.

Your program must put it back.

The sequence is:

```
Start editor

    |

Save terminal settings

    |

Enable editor mode

    |

Run application

    |

Restore original settings
```

```
    |

Exit
```

Why restoration matters

Suppose REMEDIT does this:

```
Disable echo

Disable canonical mode

Start editing

Crash
```

The terminal may remain configured for the editor.

The shell now behaves strangely:

```
$ ls
```

The user sees:

```
$
```

but characters are not displayed correctly.

Enter may behave unexpectedly.

The terminal appears broken.

The problem is not the shell.

The terminal configuration was left modified.

Recovery command

Linux provides:

```
stty sane
```

which restores normal terminal behaviour.

A programmer should know this command.

It can rescue a broken terminal session.

However, a good application should not rely on the user fixing the damage.

Protecting against crashes

A robust terminal program should restore the terminal:

- during normal exit
- during error handling
- when interrupted by signals

A common design:

```
main()
    |
terminal_init()
    |
run editor
    |
terminal_restore()
    |
exit
```

and signal handlers ensure cleanup when something unexpected occurs.

The REM terminal abstraction

The rest of the editor should not manipulate termios directly.

Create a terminal module:

```
terminal.s
    terminal_init()
    terminal_restore()
    terminal_enable_raw()
    terminal_disable_raw()
```

The editor should simply request:

```
start interactive mode
```

and:

```
return terminal to normal mode
```

First terminal milestone

Before writing the editor, write a small terminal test program.

It should prove:

Test 1

Can I save the terminal settings?

Test 2

Can I enter raw mode?

Test 3

Does a key arrive immediately?

Test 4

Does echo stop?

Test 5

Can I restore the terminal?

Only after those work should the editor depend on terminal input.

A broken terminal layer will make every higher-level feature appear broken.

Fix the foundation first.

Chapter 7 — ANSI Terminal Control

In the previous chapters, we built the correct mental model:

A terminal is not a display device.

A program does not write characters into video memory.

It writes bytes.

Some of those bytes represent visible characters.

Some of those bytes are instructions to the terminal emulator.

Those instructions are called **escape sequences**.

Understanding ANSI terminal control is the point where a programmer moves from:

> "I can print text"

to:

> "I can control an interactive text user interface."

This is the foundation on which editors, pagers, debuggers and terminal-based tools are built.

7.1 Escape sequences — the protocol

An escape sequence is a special sequence of bytes that tells the terminal to perform an action.

The name comes from the first byte:

```
ESC
```

ASCII escape is:

```
0x1B
```

A common ANSI control sequence begins:

```
ESC [
```

The `[` character introduces a **Control Sequence Introducer** (CSI).

The general form is:

```
ESC [ parameters command
```

For example:

```
ESC [ 2 J
```

means:

```
Clear the screen
```

The bytes are:

```
1B 5B 32 4A
```

Breaking that down:

```
1B      ESC
5B      [
32      2
4A      J
```

The terminal receives these bytes and interprets them as a command rather than displaying them.

A simple experiment

Before writing a terminal library, write a tiny test program.

Do not start with an editor.

Start with:

```
write bytes
observe result
```

For example:

Send:

```
ESC [ 2 J
```

The terminal clears.

Send:

```
ESC [ 5 ; 10 H
```

The cursor moves.

The terminal is a protocol.

The fastest way to understand a protocol is to experiment with small messages.

Common ANSI control structure

Many commands follow this pattern:

Flow: ESC → [→ parameters → command letter

Examples:

```
ESC [ 10 ; 20 H
```

Move cursor.

```
ESC [ 31 m
```

Set foreground colour.

```
ESC [ ? 25 l
```

Hide cursor.

The terminal does not know anything about your application.

It only knows:

"I received a command sequence. I will perform that operation."

7.2 Cursor movement

A text editor lives and dies by cursor control.

The editor constantly needs to answer:

- Where is the cursor?
- Where should text appear?
- How do we redraw after scrolling?

There are two main approaches:

- absolute positioning
- relative positioning

Absolute positioning

Absolute positioning tells the terminal:

"Move to this exact location."

The command:

```
ESC [ row ; column H
```

Example:

```
ESC [ 10 ; 20 H
```

means:

```
Move to row 10, column 20
```

A terminal library might expose:

```
terminal_move(row, column);
```

Internally:

```
terminal_move(10,20)

        |

        v

ESC [ 10 ; 20 H
```

The editor does not need to know the protocol.

Why absolute positioning is useful

Imagine redrawing the screen.

The editor knows:

```
Line 1 starts here
Line 2 starts here
Line 3 starts here
```

It can simply say:

```
Move to line 1
draw text

Move to line 2
draw text

Move to line 3
draw text
```

This is easier to reason about than tracking every cursor movement.

Relative positioning

Relative commands move from the current position.

Examples:

Cursor up:

```
ESC [ n A
```

Cursor down:

```
ESC [ n B
```

Cursor forward:

```
ESC [ n C
```

Cursor backward:

```
ESC [ n D
```

Examples:

Move up five lines:

```
ESC [ 5 A
```

Move right ten columns:

```
ESC [ 10 C
```

Absolute versus relative movement

Both are useful.

Relative movement is efficient when:

- moving short distances
- responding to a key press

Example:

User presses:

```
LEFT
```

The editor may send:

```
ESC [ D
```

Absolute movement is easier when:

- redrawing a screen
- recovering from uncertainty
- positioning known content

A simple editor should favour correctness over micro-optimisation.

A few extra bytes sent to a terminal are usually irrelevant.

Cursor visibility

Editors often hide the cursor during large redraws.

Hide:

```
ESC [ ? 25 l
```

Show:

```
ESC [ ? 25 h
```

A typical redraw sequence:

```
hide cursor

draw screen

move cursor

show cursor
```

This reduces visual flicker.

7.3 Clearing and refreshing

A terminal screen is not a memory buffer.

You cannot simply write:

```
screen[10][20] = 'A'
```

Instead, you send commands that modify the terminal's current state.

Clearing the screen

The command:

```
ESC [ 2 J
```

clears the display.

However, clearing alone does not move the cursor.

A common sequence is:

```
ESC [ 2 J
ESC [ H
```

The second command:

```
ESC [ H
```

moves the cursor to the home position:

```
row 1
column 1
```

A terminal clear function usually does both.

Example:

```
terminal_clear()
```

internally:

```
clear screen

move cursor home
```

Clearing lines

Often an editor does not need to redraw the entire screen.

For example:

The user changes one line.

Redrawing everything works.

Redrawing only the changed line is better.

Clear current line:

```
ESC [ 2 K
```

Meaning:

```
Erase entire current line
```

Other options:

```
ESC [ K
```

Erase from cursor to end of line.

```
ESC [ 1 K
```

Erase from beginning of line to cursor.

Regions

Terminal protocols also allow more advanced control:

- scrolling regions
- inserting lines
- deleting lines

These are useful for large terminal applications.

A first REM editor does not need all of them.

Start with:

- cursor movement
- line clearing
- screen clearing

Build more only when needed.

Refresh strategy

A beginner implementation often does:

```
clear screen
```

```
draw everything
```

This is acceptable initially.

However, understand the cost.

A screen might contain:

```
80 columns
24 rows
```

That is:

```
1920 characters
```

Every redraw sends a large amount of data.

A mature editor tracks:

```
old screen
```

```
versus
```

```
new screen
```

and updates only changed areas.

This is what libraries such as ncurses do.

7.4 Colours

Colour is another terminal command.

It does not change the text.

It changes the terminal's current drawing attributes.

The basic format:

```
ESC [ value m
```

Foreground colours

Examples:

Black:

```
ESC [ 30 m
```

Red:

```
ESC [ 31 m
```

Green:

```
ESC [ 32 m
```

Yellow:

```
ESC [ 33 m
```

Blue:

```
ESC [ 34 m
```

Magenta:

```
ESC [ 35 m
```

Cyan:

```
ESC [ 36 m
```

White:

```
ESC [ 37 m
```

Background colours

Background colours use:

```
40-47
```

Examples:

Red background:

```
ESC [ 41 m
```

Blue background:

```
ESC [ 44 m
```

Resetting attributes

A program should not leave the terminal in an unexpected state.

Reset:

```
ESC [ 0 m
```

This returns:

- colour
- bold
- other attributes

to normal.

A terminal application should generally restore attributes before exiting.

Syntax highlighting

This is where colours become useful for REMEDIT.

The editor does not need to understand the entire assembler.

It only needs to classify text.

Example:

Source:

```
add r1, r2, r3  # add values
```

The lexer identifies:

```
ADD
```

as an opcode.

```
R1,R2,R3
```

as registers.

```
# add values
```

as a comment.

Then the renderer chooses:

```
opcode    -> colour 1
register  -> colour 2
comment   -> colour 3
```

The terminal only knows:

“change colour.”

The editor decides:

“what does this text mean?”

7.5 Terminal capabilities

The examples above assume a terminal understands ANSI sequences.

Most modern terminals do.

However, terminals are not identical.

Historically, Unix supported many terminal types:

```
VT100
xterm
linux console
ANSI terminal
```

Each had different capabilities.

The problem

Suppose your program sends:

```
ESC [ something
```

but the terminal does not understand it.

Possible results:

- nothing happens
- strange characters appear
- incorrect behaviour

A robust terminal application needs to know what the terminal supports.

TERM environment variable

Unix systems describe the terminal type using:

```
TERM
```

Example:

```
echo $TERM
```

might return:

```
xterm-256color
```

This tells applications:

"This terminal behaves like an xterm with colour support."

---

terminfo

Libraries such as ncurses use a database called:

```
terminfo
```

It contains terminal capabilities:

- cursor movement
- colours
- clearing
- scrolling
- special keys

Instead of hard-coding:

```
ESC [ 2 J
```

a portable program asks:

"What is the correct clear-screen sequence for this terminal?"

What REMEDIT should do

There are two approaches.

Simple approach

Assume:

```
ANSI-compatible terminal
```

Advantages:

- simple
- easy to debug
- perfect for controlled environments

Disadvantage:

- less portable

Full approach

Use terminal capability databases.

Advantages:

- works everywhere

Disadvantage:

- much more complexity

The practical REM decision

For the first REM editor:

Use a small ANSI terminal layer.

Support:

- cursor movement
- screen clearing
- line clearing
- colours
- cursor visibility

Do not attempt to recreate ncurses.

The goal is not to build a terminal library.

The goal is to build a useful development environment.

First ANSI milestone

Before adding editor code, prove:

`terminal_clear()`

`terminal_move()`

`terminal_write()`

`terminal_colour()`

`terminal_restore()`

Work correctly.

Once these primitives are reliable, the editor can treat the terminal as a simple output device.

That is the abstraction we want.

Chapter 8 — Keyboard Input Decoding

A terminal keyboard is not a collection of buttons that directly generate named events.

The editor does not receive:

```
UP ARROW
DELETE
HOME
F5
```

Those concepts exist at the application level.

The terminal provides something much simpler:

A stream of bytes.

The keyboard layer exists to translate those bytes into meaningful editor commands.

The separation is important:

```
Physical keyboard
|
v

Terminal emulator
|
v

Byte stream
|
v

Keyboard decoder
|
v

Editor commands
```

The editor should not know anything about escape sequences.

It should only know:

```
KEY_UP
KEY_DELETE
KEY_HOME
```

8.1 Characters versus keys

The first important concept is:

> A character and a key are not the same thing.

A character has a representation.

For example:

```
A
```

is:

```
ASCII 0x41
```

The editor can receive:

```
0x41
```

and understand:

“Insert the character A.”

Special keys are different.

Consider:

```
UP ARROW
```

There is no single ASCII character called:

```
UP ARROW
```

Instead, the terminal sends a sequence.

For example:

Flow: ESC → [→ A

The application receives three bytes:

```
0x1B
0x5B
0x41
```

The keyboard decoder must recognise:

```
ESC [ A
```

and convert it into:

```
KEY_UP
```

Why terminals work this way

Historically, terminals were separate hardware devices.

A keyboard did not send high-level events.

It sent characters over a serial connection.

The computer received:

```
bytes
```

not:

```
keyboard events
```

Modern terminal emulators preserve this model because Unix programs depend on it.

The benefit is simplicity:

A terminal is just a stream.

The disadvantage is that applications must decode special sequences themselves.

Libraries such as ncurses hide this complexity.

REMEDIT cannot assume that layer exists.

8.2 Escape sequences

Most special keys use escape sequences.

The first byte is:

```
ESC
```

which is:

```
0x1B
```

This tells the application:

“The next bytes are not ordinary text.”

Arrow keys

The common arrow sequences are:

Up

```
ESC [ A
```

Bytes:

```
1B 5B 41
```

Meaning:

```
KEY_UP
```

Down

`ESC [ B`

Bytes:

`1B 5B 42`

Meaning:

`KEY_DOWN`

Right

`ESC [ C`

Bytes:

`1B 5B 43`

Meaning:

`KEY_RIGHT`

Left

`ESC [ D`

Bytes:

`1B 5B 44`

Meaning:

`KEY_LEFT`

Why this matters

A simple implementation might read:

`one byte`

and immediately decide:

“this is a character.”

That fails.

If the byte is:

```
ESC
```

it might mean:

- the Escape key
- the beginning of an arrow key
- the beginning of a function key

The decoder must wait and examine the following bytes.

Function keys

Function keys are more complicated.

For example:

```
F1
```

may arrive as something like:

```
ESC [ 11 ~
```

The exact sequences depend on:

- terminal type
- emulator
- keyboard configuration

This is one reason terminal capability databases exist.

A first REM editor does not need every possible key.

Start with:

- arrows
- delete
- home
- end
- page movement
- function keys you actually use

8.3 Building a keyboard state machine

The correct way to decode terminal input is as a state machine.

Do not write:

```
if byte == ESC
    maybe arrow
else
    character
```

inside the editor loop.

Create a dedicated keyboard module.

States

A simple decoder:

```
NORMAL
```

ESC received

v ~~~

ESCAPE

```
|
```

```
| '[' received
```

```
v
```

CSI

```
|
```

```
| command decoded
```

```
v
```

NORMAL

```
---
```

```
# NORMAL state
```

```
Most input arrives here.
```

```
Examples:
```

a

```
becomes:
```

CHAR('a')

```
Control keys may also be handled here.
```

```
Examples:
```

Ctrl-S Ctrl-Q

```
---
```

```
# ESCAPE state
```

```
The decoder receives:
```

ESC

```
It cannot decide yet.
```

```
Possibilities:
```

Escape key pressed

```
or:
```

start of special sequence

```
The decoder waits for another byte.

---

# CSI state

The next byte is:
```

[

```
The sequence is now known to be a control sequence.

The decoder reads until it understands the command.

Examples:
```

ESC [A

```
becomes:
```

KEY_UP

ESC [3 ~

```
becomes:
```

KEY_DELETE

```
---

# Example decoder flow

Input:
```

ESC [A

```
Processing:
```

Read ESC

state = ESCAPE

Read [

state = CSI

Read A

command = KEY_UP

state = NORMAL

```
The editor never sees:
```

ESC [A

It receives:

KEY_UP

Timeouts and ambiguity

There is one subtle problem.

What if the user presses the Escape key?

The terminal sends:

ESC

The decoder does not know whether:

- Escape was pressed
- another byte is coming

For example:

ESC [A

starts with the same byte.

A real implementation uses a timeout.

Conceptually:

Receive ESC

wait briefly

another byte arrives?

yes:
    decode sequence

no:
    treat as Escape key

The timeout does not need to be perfect.

It only needs to feel responsive.

8.4 Mapping to editor commands

The keyboard decoder should stop at a clean boundary.

Bad design:

editor.s

if ESC if [if A move cursor

Now the editor knows terminal protocol.

Good design:

terminal input

    |

    v

keyboard decoder

    |

    v

editor command

Example event definitions

The keyboard layer produces:

KEY_UP

KEY_DOWN

KEY_LEFT

KEY_RIGHT

KEY_DELETE

KEY_HOME

KEY_END

KEY_PAGE_UP

KEY_PAGE_DOWN

CHARACTER

The editor handles:

if event == KEY_UP

move cursor upward

It does not care how that event was generated.

Why this abstraction matters

Today:

Flow: terminal emulator → ANSI escape sequences → keyboard decoder

Tomorrow:

Flow: Bluetooth keyboard → different input system → keyboard decoder

```
The editor does not change.

This is the same principle used throughout systems programming:

Hide unstable details behind stable interfaces.

---

# First keyboard test program

Before connecting this to the editor, build:
```

keyboard_test

```
It should display:
```

received:

CHAR ‘a’

KEY_UP

KEY_DOWN

KEY_LEFT

KEY_RIGHT

KEY_DELETE

```
Test:

- normal characters
- numbers
- Enter
- Backspace
- Escape
- arrow keys
- function keys

Only when this works should the editor consume keyboard events.

---

# The keyboard layer milestone

At the end of this chapter you should have:
```

Raw bytes

```
    |

    v
```

Keyboard decoder

```
    |

    v
```

Logical events

```
The editor should never know that:
```

UP ARROW

was actually:

ESC [A

That separation is what turns a collection of terminal tricks into a real application architecture.

Part IV — Building REMEDIT

Chapter 9 — Editor Architecture

A text editor appears simple from the outside.

The user sees:

- characters appearing on the screen
- a cursor moving
- files opening and saving

However, an editor is not primarily a screen program.

The screen is only the final output of a much larger system.

A useful way to think about REMEDIT is:

```
             User
              |
              v
          Keyboard
              |
              v
          Commands
              |
              v
          Document
              |
              v
          Renderer
              |
              v
          Terminal
```

```
The mistake many first implementations make is starting at the bottom:

> "How do I print characters on the screen?"

The better question is:

> "What information does the editor need to maintain, and how does that information
 ↪  change?"

The screen is a view of the editor's internal state.

It is not the editor itself.

---

# 9.1 The editor loop

Almost every interactive program has a central loop.

The editor is no different.

The basic structure is:

**Flow:** read → interpret → modify → render

This loop repeats until the user exits.

---

# Read

The first stage receives input.

The keyboard layer has already converted raw terminal bytes into events:
KEY_UP
KEY_DELETE
CHARACTER('A')
CTRL_S

The editor does not know:
ESC [ A

was received.

That problem has already been solved.

The editor sees:
KEY_UP

This is an important boundary.

---
```

Interpret

The editor decides what the event means.

A key does not always have one meaning.

For example:

CTRL_S

might mean:

save file

in normal editing mode.

A different mode might interpret it differently.

The editor contains the rules:

KEY_UP

|
v

move cursor upward

or:

CHARACTER('x')

|
v

insert x into document

Modify

The editor changes its internal state.

Examples:

User presses:

a

The editor does not immediately write:

a

to the terminal.

Instead:

Keyboard event

|

v

Insert character into document

|

v

Document changed

|

v

Screen needs updating

```
The document is the source of truth.

---

# Render

The renderer looks at the current editor state and displays it.

It decides:

- which lines are visible
- where the cursor should appear
- which colours to use
- what status information to show

The renderer does not decide what the document contains.

It only displays it.

---

# Why this separation matters

A beginner often creates:
```

keypress

|

print character

```
This works for a toy program.

It fails for an editor.

Consider:

User types:
```

hello

```
then presses:
```

LEFT

```
then types:
```

X

The editor cannot simply append characters to the screen.

It needs to know:

Current document:

hello

Cursor:

position 4

After insertion:

helXlo

The screen is merely updated afterwards.

9.2 Model versus display

This is the most important editor design concept.

The document is not the screen.

The screen is a representation of the document.

The wrong model

A beginner may think:

Screen contents = document

For example:

Line 1: Hello

is simply what is displayed.

This creates problems immediately.

What happens when:

- the file is larger than the screen?
- the user scrolls?
- part of the document is off-screen?
- the window is resized?
- syntax highlighting changes?

The screen cannot contain the entire document.

The correct model

The editor maintains a document:

Document

Line 1: Hello Line 2: World Line 3: REMEDIT Line 4: ... Line 5: ...

The terminal displays a window into that document:

Hello World REMEDIT

The user sees only part of the data.

The editor knows all of it.

The viewport

The connection between the document and the display is the viewport.

The viewport answers:

> "Which part of the document is currently visible?"

Example:

Document:

Line 1 Line 2 Line 3 Line 4 Line 5 Line 6 Line 7

Screen:

Line 4 Line 5 Line 6 Line 7

The viewport is:

first visible line = 4

When the cursor moves down:

Line 8

the document has not changed.

Only the viewport changes.

9.3 Editor state

The editor needs to maintain several independent pieces of information.

A useful first design:

EditorState

```
document

cursor

viewport

mode

selection

file information

Each represents a different concept.

---

# Cursor

The cursor represents the logical editing position.

It is not simply:
```
screen row screen column

```
because the document may be larger than the display.

A better representation:
```
cursor:
line column

```
Example:
```
Document:
Line 100
Cursor:
line = 100 column = 25

The renderer later decides where that position appears on the screen.

Screen cursor versus document cursor

These are different.

Document position:
```
line 100 column 25

```
Screen position:
```
row 12 column 25

because the viewport may start at:
```

line 88

```
The relationship is:
```

screen_row = document_line - viewport_start

```
Keeping these concepts separate prevents many bugs.

---

# Viewport

The viewport stores what portion of the document is visible.

Example:
```

viewport:

top_line = 50 left_column = 0

```
The renderer uses this:
```

Document:

50 51 52 53 54

Screen:

line 50 line 51 line 52 line 53

```
Scrolling changes the viewport.

It does not modify the document.

---

# Document

The document is the actual text being edited.

At this stage, do not worry about the final storage implementation.

Later chapters will discuss:

- simple line arrays
- gap buffers
- piece tables

The editor should interact with the document through operations:
```

insert character

delete character

insert line

delete line

retrieve line

```
The storage method remains hidden.
```

```
---

# Selection

A selection represents a range of text.

Examples:
```

select characters

select line

select block

```
Even if REMEDIT version one does not support visual selection, the architecture should
 ↪  leave room for it.

A selection might contain:
```

start:

line column

end:

line column

```
---

# Mode

Editors often have different operating modes.

Examples:

Simple editor:
```

NORMAL

```
and:
```

INSERT

```
A command key can mean different things depending on the current mode.

For example:

In insert mode:
```

a

```
means:
```

insert character

```
In command mode:
```

a

```
might mean:
```

append text

```
The editor state must know which interpretation applies.

---

# The REMEDIT state model

A simple initial structure:
```

EditorState

{ Document document;

```
Cursor cursor;

Viewport viewport;

Mode mode;

Selection selection;

FileInfo file;
```

}

```
The exact implementation in assembly will differ, but the concept remains.

---

# The complete editor loop

Putting everything together:
```

while running:

```
event = keyboard_read()

command = interpret(event)

modify(document, command)

update_viewport()

render()

Each stage has a responsibility.

---

# Why this architecture matters for REM

REM is a new computer architecture.

The temptation will be to optimise immediately:

- clever screen handling
- compressed data structures
- complex rendering
```

```
Avoid that initially.

The first goal is not the fastest editor.

The first goal is a correct editor whose parts can be understood.

A well-designed editor also becomes a demonstration platform.

Once REMEDIT exists, it can become:

- an assembly development environment
- a debugger front end
- a system monitor
- a test platform for new REM features

The editor is not just an application.

It is the first major piece of REM software infrastructure.

# Chapter 10 — Text Storage

The editor architecture introduced an important separation:
```

Document

|

v

Renderer

|

v

Terminal

```
The renderer does not care how text is stored.

The keyboard does not care how text is stored.

The command system does not care how text is stored.

Only the document layer owns that decision.

This is important because text storage is one of the places where editor design becomes
 ↪  a trade-off.

A text file looks simple:
```

Line one Line two Line three

```
but editing creates difficult operations:

- insert a character in the middle of a line
- delete characters
- insert new lines
- delete lines
- move large sections of text
- support undo
```

- handle large files

The storage method determines how easy those operations are.

10.1 The naive approach — single buffer

The simplest possible design is a single block of memory containing the entire file.

Example:

File:

Hello World

Stored as:

H e l l o [newline] W o r l d [newline]

Conceptually:

H e l l o [newline] W o r l d [newline]

The cursor is simply an offset:

position = 8

This is attractive because it is easy to understand.

For a first test program, it is excellent.

Inserting text

Suppose the file contains:

Hello World

The user inserts:

Beautiful

The buffer currently contains:

Hello World

The insertion point is:

Hello |World

The buffer must become:

Hello Beautiful World

That requires moving all data after the cursor:

Before:

Hello |World

After moving:

Hello | World

Insert:

Hello Beautiful World

```
For a small file, this is fine.

For a large file, repeated editing becomes expensive.

---

# Problems with a single buffer

## Problem 1 — Insertions are expensive

Every insertion may require moving a large amount of memory.

Example:
```

File size: 100 MB

Cursor position: beginning of file

```
Insert one character.

The editor may need to move almost:
```

100 MB

```
of data.

---

## Problem 2 — Deletions have the same problem

Deleting a character:
```

Hello World ^ delete

```
requires shifting everything after it.

---

## Problem 3 — Undo becomes complicated

Undo requires knowing:

- what changed
- where it changed
- what data was removed

The storage model does not naturally preserve history.

---
```

Problem 4 — Large files

A simple buffer assumes:

"The whole file fits comfortably in memory."

Modern systems usually have enough memory, but the design still limits scalability.

Why start with it anyway?

Because it is simple.

The purpose of a first implementation is not to win editor performance benchmarks.

The purpose is to prove:

- keyboard handling works
- commands work
- rendering works
- files can be loaded and saved

A simple buffer removes complexity.

That is often the correct engineering decision.

10.2 Line-based storage

A more natural editor representation is a collection of lines.

Instead of:

H e l l o [newline] W o r l d

store:

Line 1:

H e l l o

Line 2:

W o r l d

Conceptually:

Document

Line 1
Line 2
Line 3

This matches how humans think about text.

```
# Advantages of line storage

## Natural cursor handling

The editor already thinks in:
```

line column

```
A line-based structure matches this directly.

Example:
```

Cursor:

line = 20 column = 5

```
The document can directly locate:
```

Line 20 Character 5

```
---

## Simple rendering

The renderer needs:
```

visible lines

```
A line-based document provides exactly that.

Example:
```

Viewport:

line 100 line 101 line 102 line 103

```
The renderer requests:
```

give me lines 100-103

```
---

## Easy insertion of lines

Pressing Enter:

Before:
```

Hello World

```
After:
```

Hello

World

```
A line-based structure naturally supports:
```

split line insert new line

```
---

# Problems with line storage

It is not perfect.

A line may still be stored as a contiguous array.

Example:
```

Line:

Hello World

```
Insert:
```

Beautiful

```
Still requires moving characters inside that line.

---

Large documents also require managing:

- thousands of lines
- memory allocation
- line indexing

However, for a first editor, line storage is a very reasonable choice.

---

# 10.3 Gap buffers

A gap buffer is a classic editor data structure.

Many traditional editors use some variation of this idea.

The concept is simple:

Keep an empty space where the cursor is.

Example:
```

Before:

Hello World

```
Memory:
```

H e l l o _ W o r l d ^ gap

```
The underscore represents unused space.

The cursor sits in the gap.

---
```

```
# Inserting characters

Insert:
```

Beautiful

```
The editor fills the gap:
```

H e l l o B e a u t i f u l _ W o r l d

```
No large movement is required.

The next insertion is also cheap.

---

# Moving the cursor

The problem appears when the user moves far away.

Example:

Current gap:
```

Hello |World

```
User moves to:
```

Hello World|

```
The gap must move.

The editor shifts characters across the gap:
```

Before:

Hello _____ World

After:

Hello World _____

```
Moving the cursor has a cost.

---

# Advantages of gap buffers

## Fast typing

This is the important case.

Most editing involves:

- typing characters
- small cursor movements

The gap is already where typing happens.

---
```

```
## Simple implementation

Compared with more advanced structures, a gap buffer is straightforward.

It requires:

- memory block
- gap position
- gap size

---

## Good for assembly

This is important for REMEDIT.

A complicated data structure creates more assembly work.

A gap buffer has relatively simple operations:
```

move gap

insert character

delete character

```
---

# 10.4 Piece tables

Piece tables are a different approach.

They became popular in modern editors.

The idea:

Never modify the original file.

Instead:

- keep the original text
- store changes separately
- maintain a table describing the current document

---

# Example

Original file:
```

Hello World

```
Stored permanently:

Original buffer:

Hello World

```
User inserts:
```
```

Beautiful

```
The inserted text goes into an add buffer:
```

Add buffer:

Beautiful

```
The document becomes a list of pieces:
```

Original:

Hello

Add:

Beautiful

Original:

World

```
The file is represented by references.

---

# Advantages of piece tables

## Excellent undo support

Because the original data is never destroyed, undo becomes easier.

The editor can simply modify the piece list.

---

## Efficient large files

Large sections of text do not need copying.

The editor moves references instead.

---

## Natural history

The structure already represents changes.

This fits well with:

- undo
- redo
- version history

---

# Problems with piece tables

They are more complicated.

The programmer must manage:
```

```
- piece records
- buffers
- indexing
- merging pieces

For a first assembly editor, this complexity may not be justified.

---

# 10.5 Choosing REMEDIT version 1

The purpose of REMEDIT version 1 is not to compete with mature editors.

The purpose is to create:

- a usable REM development tool
- a learning platform
- a foundation for future tools

The first implementation should favour:

- simplicity
- visibility
- debugging ability

over:

- maximum performance
- handling gigabyte files
- advanced undo systems

---

# Recommended progression

A sensible development path:
```

Version 1

Simple line-based storage

    |

    v

Version 2

Gap buffer for active editing

    |

    v

Version 3

Piece table or advanced structure

```
Each stage teaches something.

---
```

```
# Why not start with the "best" design?

Because every additional feature creates another place to fail.

A piece table requires confidence in:

- memory management
- indexing
- rendering
- undo design

before you have even proved:

"Can I edit a file?"

---

# Recommended REMEDIT v1 structure

For the first working editor:
```

Document

```
array of lines
```

Line

```
length

character buffer
```

Cursor

```
line

column
```

Viewport

```
first visible line

first visible column


Operations:
```

insert character

delete character

split line

join lines

load file

save file

```
This is enough to create a real editor.

---

# The engineering rule

The best data structure is not always the most advanced one.
```

```
The best first data structure is the one that lets you prove the rest of the system.

A working simple editor teaches more than an unfinished sophisticated editor.

Build the foundation.

Optimise after the machine works.

# Chapter 11 — Editing Operations

The previous chapter discussed how text is stored.

This chapter deals with what the user actually does with that text.

An editor is not complicated because it displays characters.

Displaying characters is the easy part.

The difficult part is maintaining a consistent document while the user constantly
↪ changes it.

Every editing operation must update several things:

- the document
- the cursor position
- the viewport
- the undo history
- the display

A useful rule:

> The document changes first. The screen changes afterwards.

The terminal is only a representation of the current state.

---

# 11.1 Insert character

The most common editor operation is inserting a character.

The user presses:
```

a

```
The editor receives:
```

CHARACTER('a')

The editor must decide:

Where does this character belong?

The answer comes from the cursor.

Example:
```
```

```
Document:
```

Hello World

```
Cursor:
```

Hello |World

```
Insert:
```

X

```
New document:
```

Hello XWorld

```
The operation is:
```

insert character at cursor position

```
---

# The insert algorithm

For a line-based document:

Before:
```

Line:

```
H e l l o W o r l d
    ^
    cursor
```

```
The editor:

1. Finds the current line.
2. Finds the cursor column.
3. Makes space for one character.
4. Copies existing characters after the cursor.
5. Inserts the new character.
6. Moves the cursor forward.

Conceptually:
```

Before:

HelloWorld

After moving data:

Hello_ _World

Insert:

HelloXWorld

```
---

# Data structure considerations
```

```
The cost depends on the storage model.

## Simple array

A character insertion requires shifting data.

Example:
```

Insert near beginning:

A B C D E F

|

Move:

B C D E F

```
Cost:
```

O(n)

```
where `n` is the amount of text after the insertion point.

---

## Gap buffer

A gap buffer makes insertion efficient.

The cursor sits at the gap:
```

Hello_____World ^ cursor

```
Insert:
```

X

```
becomes:

HelloX____World

```
No large movement is required.

---

# Cursor movement after insertion

Most editors move the cursor after inserting.

Example:

Before:
```

Hello|

```
Insert:
```

X
```

```
After:
```

HelloX|

```
The cursor position changes:
```

column = column + 1

```
This seems obvious, but every operation needs clearly defined cursor behaviour.

---

# 11.2 Delete character

Deletion is the opposite operation.

There are two common delete operations:

- Backspace
- Delete key

They appear similar but behave differently.

---

# Backspace

Backspace removes the character before the cursor.

Example:

Before:
```

Hello World ^ cursor

```
Press backspace:
```

Hell World ^ cursor

```
The cursor moves left.

---

# Delete key

Delete removes the character under the cursor.

Example:

Before:
```

Hello World ^ cursor

```
Delete:
```

HelloWorld ^ cursor

```
The cursor does not move.

---

# Delete algorithm

For a simple line buffer:

Before:
```

H e l l o ^ delete

```
Remove the character:
```

H e l l _

```
Move remaining characters left:
```

H e l l

```
Update:

- line length
- cursor position
- undo history

---

# Edge cases

Deletion becomes interesting at boundaries.

## Beginning of line

Example:
```

|Hello

```
Backspace cannot remove a character.

Possible behaviours:

- do nothing
- join with previous line

Most editors choose joining.

---

## End of line

Example:
```

Hello| World

```
Backspace at the beginning of the next line:
```

Hello |World

may become:

HelloWorld

This operation is called:

join line

11.3 Split line

The Enter key is not just inserting a character.

A newline changes the document structure.

Before:

Hello World|

Press Enter:

Hello World
|

The document now contains two lines.

Split algorithm

Given:

Line:

Hello World

Cursor:

Hello |World

Create two lines:

First:

Hello

Second:

World

The cursor moves:

World|

The operation is:

split current line at cursor

```
# Why split is important

Almost every text editor operation eventually depends on this.

Examples:

- pressing Enter
- pasting multiline text
- formatting code
- inserting files

The line structure changes.

---

# Memory considerations

For a line-based editor:

Before:
```

Line array:

[Line 1] [Line 2] [Line 3]

```
Insert a new line:
```

[Line 1] [New Line] [Line 2] [Line 3]

```
The line table must expand.

Possible implementations:

Simple:
```

copy remaining line pointers

```
More advanced:
```

linked list dynamic array

```
For REMEDIT version 1:

A dynamic array is probably sufficient.

---

# 11.4 Join lines

Joining is the reverse of splitting.

Before:
```

Hello World

```
Cursor:
```

|World

```
Backspace:
```

HelloWorld

```
The two lines become one.

---

# Join algorithm

1. Locate current line.
2. Locate previous line.
3. Append current line contents.
4. Delete current line.
5. Move cursor to join point.

Example:

Before:
```

Line 1:

Hello

Line 2:

World

```
After:
```

HelloWorld

```
---

# Important decision

Should a space be inserted?

Different editors behave differently.

Possibilities:

No space:
```

HelloWorld

```
Space:
```

Hello World

```
Preserve exactly:
```

HelloWorld

```
The editor design must choose.

The important thing is consistency.

---
```

```
# 11.5 Cursor movement

Cursor movement seems simple:
```

LEFT RIGHT UP DOWN

```
but it creates many design decisions.

---

# Horizontal movement

Example:
```

Hello World ^

```
LEFT:
```

Hello World ^

```
RIGHT:
```

Hello World ^

```
Simple.

---

# Beginning and end of line

Common commands:
```

HOME END

```
Home:
```

|Hello World

```
End:
```

Hello World|

```
---

# Vertical movement

This is more complicated.

Suppose:
```

Line 1: Hello World

Line 2: Hi

```
Cursor:
```

Line 1 column 8

Move down.

Line 2 does not have column 8.

Where should the cursor go?

Options:

Hi|

or:

remember desired column 8

Most editors use the second approach.

Preferred column

The editor maintains:

cursor column

and:

preferred column

Example:

Move down:

Line 1:

Hello World ^ column 8

Move down:

Line 2:

Hi ^

Move down again:

Line 3:

abcdefghijk ^

The editor remembers the original intention.

Cursor state

A useful structure:

Cursor

line

```
column

preferred_column

This prevents jumping around while moving vertically.

---

# 11.6 Scrolling

The document may be larger than the screen.

The editor therefore needs a viewport.

Example:

Document:
```

Line 1 Line 2 Line 3 ... Line 1000

```
Screen:
```

Line 500 Line 501 Line 502 Line 503

```
The viewport stores:
```

top visible line = 500

```
---

# Keeping the cursor visible

When the cursor moves:
```

cursor line

versus

viewport range

```
must be checked.

Example:

Screen:
```

Lines 100-120

```
Cursor moves to:
```

Line 121

```
The editor must scroll.

New viewport:
```

Lines 101-121

```
---
```

```
# Horizontal scrolling

Long lines create another problem.

Example:
```

This is a very long line of assembly code..... ^ cursor

```
The screen cannot display everything.

The editor tracks:
```

left column

```
Example:
```

viewport.left = 40

```
The renderer displays from column 40 onwards.

---

# Redrawing after operations

Every editing operation follows the same pattern:
```

User input

```
|

v
```

Update document

```
|

v
```

Update cursor

```
|

v
```

Update viewport

```
|

v
```

Render screen

```
Do not update the terminal directly from editing code.

The renderer should be the only component that knows how to display.

---

# Testing editing operations

Before combining everything, test each operation.
```

```
Example:
```

buffer_test

```
Tests:
```

insert character

delete character

split line

join lines

move cursor

scroll viewport

```
A small failing test is easier to debug than a broken editor.

---

# The REMEDIT milestone

At the end of this chapter, the editor has the foundations of editing:
```

Keyboard

```
|

v
```

Commands

```
|

v
```

Document operations

```
|

v
```

Screen rendering

```
The editor is no longer just displaying text.

It understands text as data.

That distinction is what separates a terminal display program from a real editor.

# Chapter 12 — Undo and Redo

Undo is one of those features that appears simple from the outside.

The user presses:
```

Ctrl-Z

```
and the previous change disappears.
```

However, inside the editor, undo is not a display feature.

It is an architectural feature.

A text editor that was not designed with undo in mind usually has to be redesigned to support it properly.

The reason is simple:

The editor must remember what happened.

Not just:

What does the document look like now?

but:

How did the document get here?

12.1 Why undo is an architecture problem

A beginner implementation often looks like this:

keypress

|

v

modify document

|

v

display result

The document changes.

The previous state disappears.

For example:

Before:

Hello

Insert:

World

After:

Hello World

The editor now knows:

Current document:

Hello World

But undo requires:

What was the previous document?

Hello

The old information no longer exists.

The naive solution: save the whole file

The simplest undo system is:

Before every change:

copy entire document

After:

perform operation

Undo:

restore previous copy

This works.

It is also wasteful.

Example:

A user edits a 50 MB file.

They type:

a

The editor copies:

50 MB

They type another character.

Another:

50 MB

After a few minutes:

The undo history consumes enormous memory.

The better approach: remember operations

```
Instead of storing complete document states, store the changes.
```

```
The editor already knows:
```

Insert character Delete character Split line Join line

```
Undo is simply performing the opposite operation.
```

```
Example:
```

```
Normal operation:
```

Insert ‘A’

```
Undo:
```

Delete ‘A’

```
Normal operation:
```

Delete “hello”

```
Undo:
```

Insert “hello”

```
The editor stores the information required to reverse the action.
```

```
---
```

```
# Command-based architecture
```

```
A useful model is:
```

User action

```
  |
```

```
  v
```

Command object

```
  |
```

```
  v
```

Execute command

```
  |
```

```
  v
```

Store command history

```
Every editing action becomes a command.
```

```
Examples:
```

InsertText

DeleteText

ReplaceText

SplitLine

JoinLine

```
The command contains:

- what happened
- where it happened
- enough information to undo it

---

# 12.2 Command records

A command record is a description of one change.

In assembly, this is simply a data structure.

Example:
```

Command

```
type

location

data

length

The exact layout depends on the REM memory model.

The principle is more important than the implementation.

---

# InsertText

Example:

User types:
```

A

```
Document:

Before:
```

Hello

```
After:
```

HelloA

```
The command record stores:
```

Type:

INSERT_TEXT

Position:

line = 1 column = 5

Data:

“A”

```
To undo:
```

Delete character at line 1 column 5

```
---

# DeleteText

Example:

User deletes:
```

World

```
The command must store the removed data.

Record:
```

Type:

DELETE_TEXT

Position:

line = 1 column = 6

Data:

“World”

```
Undo:
```

Insert “World”

```
---

# ReplaceText

Replace is more complicated.

Example:

Before:
```

Hello World

```
Replace:
```

World

```
with:
```

REM

`The command stores both states:`

Old text:

World

New text:

REM

`Undo:`

replace REM with World

`---`

`# SplitLine`

`Before:`

Hello World

`After:`

Hello

World

`Command:`

Type:

SPLIT_LINE

Line:

1

Column:

5

`Undo:`

JOIN_LINE

`---`

`# JoinLine`

`Before:`

Hello

World

`After:`

HelloWorld

`The command stores:`

Original line boundary

Location

Removed newline

Undo restores:

Hello

World

Command execution

A useful design is:

execute(command)

    |

    v

document changes

    |

    v

push command onto undo stack

The command itself knows how to:

apply()

undo()

Conceptually:

Command

execute()

reverse()

12.3 Undo stack

Undo history is usually implemented as a stack.

A stack follows:

Last In, First Out

LIFO

Example:

User performs:

1. Insert A

2. Insert B
3. Delete C

History:

Delete C
Insert B
Insert A

Undo:

Remove:

Delete C

Apply reverse:

Insert C

Stack becomes:

Insert B
Insert A

Multiple undo operations

The user presses undo repeatedly.

Example:

Undo

Undo

Undo

The editor walks backwards through history:

Current state

|

v

Command 3 reversed

|

v

Command 2 reversed

|

v

Command 1 reversed

```
---

# How many commands?

The editor needs a policy.

Options:

## Unlimited

Advantages:

- simple conceptually

Disadvantages:

- memory grows forever

---

## Fixed number

Example:
```

Keep last 1000 commands

```
Advantages:

- predictable memory

Disadvantages:

- old history disappears

---

## Fixed memory size

Example:
```

Keep 64 MB of undo data

```
Advantages:

- better resource control

Disadvantages:

- more complex

---

For REMEDIT version 1:

A fixed command count or memory limit is probably sufficient.

---
```

```
# 12.4 Redo rules

Redo is often misunderstood.

Undo moves backwards.

Redo moves forwards again.

The editor therefore needs a second history.

Normally:
```

Undo stack

Redo stack

```
---

# Example

User types:
```

A B C

```
History:
```

Undo:

C B A

```
User presses undo:
```

Remove C

```
Now:
```

Undo:

B A

Redo:

C

```
Press redo:
```

Restore C

```
---

# The important rule

A new edit destroys the redo history.

Example:

Start:
```

ABC

Undo:

AB

Now type:

D

Result:

ABD

The old future:

ABC

is no longer valid.

The redo stack is cleared.

This rule simplifies the model.

Grouping commands

One usability improvement:

Not every keystroke should necessarily create a separate undo item.

Example:

User types:

hello

Should undo remove:

h

or:

hello

Most editors group continuous typing.

This requires a little more design.

A simple first version can treat each insertion as separate.

A later version can add grouping.

12.5 Memory management

Undo requires careful memory management.

Every command may contain data.

Example:

Delete:

Delete 5000 characters

The undo record must store:

5000 characters

Otherwise the deletion cannot be reversed.

Ownership rules

Every undo record must define:

Who owns this memory?

Example:

Command created

|

v

copies deleted text

|

v

undo stack owns copy

|

v

command removed

|

v

free memory

Without clear ownership rules, undo becomes a source of memory leaks.

Simple assembly design

A command record might contain:

Command:

type

line

column
length
data pointer
next pointer

The stack:
UndoStack
top -> Command | v Command | v Command

Testing undo

Undo should be tested separately.

Do not test only through the editor.

Create:
undo_test

Test:
insert
undo
check document
delete
undo
check document
replace
undo
check document

The key question:

> After any sequence of operations, can the editor return to the previous state?

The REMEDIT milestone

After implementing undo and redo:

The editor now has a history.

The architecture becomes:
Keyboard
|
v

Command

|

```
+------------+
|            |
v            v
```

Document Undo history

|

v

Renderer

```
Undo is not an optional extra feature.

It affects the design of the entire editor.

A good editor is not only a program that changes text.

It is a program that understands how text changed.


# Part V — Making It a REM Development Environment

# Chapter 13 — File Handling

A text editor without file handling is only a demonstration.

Displaying text proves that the terminal works.

Editing text proves that the data structures work.

Saving and loading proves that the application can become part of a real development
 ↪  environment.

File handling appears simple:
```

open file

read data

modify data

save file

```
However, a reliable editor must consider:

- file formats
- memory allocation
- permissions
- partial writes
- failures
- recovery after crashes

The filesystem is another boundary.

The editor should not care how the storage device works.
```

```
It should only care about the file interface provided by Linux.

The basic model is:
```

REMEDIT

|

v

libc / syscalls

|

v

Linux filesystem

|

v

Storage

```
---

# 13.1 Loading files

Opening an existing file is the first real interaction between REMEDIT and the outside
 ↪  world.

The operation is:
```

filename

|

v

open file

|

v

read contents

|

v

create document

```
---

# The editor workflow

When the user starts:
```

remedit program.s

```
the editor must:

1. Receive the filename.
```

2. Open the file.
3. Determine its size.
4. Read the contents.
5. Convert the contents into the internal document format.
6. Display the file.

Opening a file

The operating system represents an open file using a file descriptor.

Conceptually:

filename

|

v

kernel

|

v

file descriptor

The editor does not directly manipulate the disk.

It receives a handle:

fd = 3

and uses that for later operations.

Reading data

The simplest approach:

open file

read all contents

close file

For a small editor, this is perfectly reasonable.

Example:

File:

hello.asm

Contents:

MOV R1,R2 ADD R3,R4

Memory:

Document

Line 1: MOV R1,R2

Line 2: ADD R3,R4

```
---

# Converting file data into the document

The filesystem stores bytes.

The editor stores text structures.

These are not necessarily the same thing.

File:
```

M O V R 1 , R 2 [newline] A D D ...

```
Document:
```

Line array

```
|
+-- Line 1
|
+-- Line 2

The loading process converts:
```

byte stream

```
  |

  v
```

editor document model

```
---

# Line endings

One early decision:

How does the editor handle line endings?

Different systems use:

Unix:
```

LF

0x0A

```
Windows:
```

CR LF

0x0D 0x0A

```
A good editor normalises internally.

For example:

When loading:
```

CR LF

```
becomes:
```

LF

```
When saving:

the editor writes the chosen format.

---

# Large files

The simple approach:
```

load entire file into memory

```
works well initially.

Advantages:

- simple
- easy to debug
- fast editing

Disadvantages:

- large files consume memory

Advanced editors may use:

- memory mapping
- lazy loading
- paging

REMEDIT version one does not need this.

The first goal is reliability.

---

# File loading errors

Every file operation can fail.

Examples:

File does not exist:
```

ENOENT

```
Permission denied:
```
EACCES

```
Storage failure:
```
EIO

```
The editor should report:
```
Cannot open file: reason

```
not simply:
```
Error

```
Good error messages save debugging time.

---

# 13.2 Saving files

Saving appears easier than loading.

The naive approach:
```
open file
write contents
close file

However, saving is where reliability matters most.

The user has changed their work.

A failed save can destroy hours of effort.

The dangerous approach

Consider:
```
Open original file
Write new contents
Something fails

```
The result may be:
```
Original file corrupted

```
The editor has destroyed the only copy.

Never make the original file the first thing you overwrite.

```
```

```
# Safer saving

A common approach:
```

Create temporary file

Write new contents

Flush data

Rename temporary file

```
The process:
```

document

```
|

v
```

temporary file

```
|

v
```

rename

```
|

v
```

original filename

```
The rename operation is normally atomic.

The user sees either:

Old version

or:

New version

but not a half-written file.

---

# Save operation

Conceptually:
```

save()

```
create temp file

write document

close file

rename temp -> original

update editor state
```

```
---

# Tracking changes

The editor should know whether the file has changed.

State:
```

modified = false

```
After editing:
```

modified = true

```
After successful save:
```

modified = false

```
The UI can display:
```

program.s *

```
to indicate unsaved changes.

---

# 13.3 Temporary files

Temporary files support safe operations.

They are also useful for:

- recovery
- backups
- intermediate builds

---

# Temporary filename

Example:

Original:
```

main.s

```
Temporary:
```

.main.s.tmp

```
or:
```

main.s.REMEDIT.tmp

```
The important requirement:

Do not accidentally overwrite another file.
```

```
---

# Temporary file ownership

The editor must know:

Who owns the temporary file?

Example:
```

Create temp file

```
|

v
```

Editor owns it

```
|

v
```

Successful save

```
|

v
```

Rename and remove temporary state

```
If the editor exits unexpectedly:

the temporary file may still exist.

That can become useful for recovery.

---

# Atomic rename

The key idea:

Writing:
```

large new file

```
directly over:
```

existing file

```
is risky.

Instead:
```

write complete new version

then switch names

```
The rename is the point where the change becomes visible.

This technique is used by many reliable systems.
```

```
---

# 13.4 Crash recovery

A serious editor assumes crashes happen.

Possible causes:

- program bug
- power failure
- operating system problem
- hardware failure

The goal is not preventing every crash.

The goal is reducing lost work.

---

# Recovery files

Many editors periodically save recovery information.

Example:

Working file:
```

main.s

```
Recovery file:
```

.main.s.recovery

```
The recovery file contains enough information to restore:

- document contents
- cursor position
- undo information

---

# Simple recovery strategy

For REMEDIT version one:

A practical approach:
```

When file changes:

```
periodically write recovery copy
```

Example:

Every:
```

5 minutes

```
or after:
```
```

100 changes

```
---

# Recovery workflow

Startup:
```

Open main.s

Check:

Does recovery file exist?

```
If yes:
```

main.s was modified previously.

Recover?

```
The user decides.

---

# Recovery versus autosave

These are different.

Autosave:
```

replace original file automatically

```
Recovery:
```

preserve emergency copy

```
For a development tool, recovery is safer.

The programmer should remain in control of source files.

---

# File module design

The editor should not scatter file operations everywhere.

Create a file module:
```

file.s

```
Responsibilities:
```

file_open()

file_load()

file_save()

file_save_temp()

file_recover()

The editor calls:

save document

It does not need to know:

- filenames
- file descriptors
- temporary naming
- filesystem details

Testing file handling

Before connecting to the editor:

Create:

file_test

Tests:

create file

write data

read data

compare contents

rename file

delete file

Then test failure cases:

missing file

read-only file

invalid path

full storage

The REMEDIT milestone

After this chapter, the editor has crossed an important boundary.

It is no longer only a memory editor.

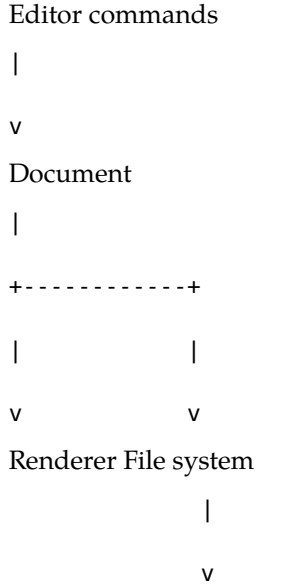
It is a development tool.

The architecture now looks like:

Keyboard

|

v




```
A real programmer does not only edit text.

They create files, modify files, build software and recover from mistakes.

Reliable file handling is what turns REMEDIT from an experiment into a tool.


# Chapter 14 — Search and Replace

Search is one of the first features that turns an editor from a text viewer into a
 ↪  useful development tool.

A programmer working on REM will spend a large amount of time searching for:

- instruction names
- labels
- function names
- error messages
- comments
- constants
- configuration values

A good search system is not complicated because finding text is difficult.

Finding text is easy.

The complexity comes from integrating search with:

- the document model
- cursor movement
- selection
- replacement
- undo history

The search system should follow the same architectural principle used throughout
 ↪  REMEDIT:
```

Search logic

|

v

Document model

|

v

Renderer

```
Search should not directly manipulate the screen.

It should operate on the document.

---

# 14.1 Literal search

The first search implementation should be literal text matching.

Given:

Document:
```

MOV R1,R2 ADD R3,R4 MOV R5,R6

```
Search:
```

MOV

```
Results:
```

Line 1, column 1

Line 3, column 1

```
No interpretation.

No regular expressions.

No language awareness.

Just:
```

Does this sequence of characters exist?

```
---

# Search operation

A simple search flow:
```

User enters search text

    |

    v

Search module scans document

|

v

Match found

|

v

Move cursor

|

v

Update viewport

|

v

Render

```
The search module does not draw anything.

It only reports:
```

match location

```
---

# Searching a line

The simplest case is searching within one line.

Example:

Line:
```

LOAD R1,VALUE

```
Search:
```

VALUE

```
Algorithm:
```

Start at beginning of line

Compare characters

Move forward

Stop when:

```
match found
```

or:

```
end of line reached
```

```
This is a straightforward character comparison.

---

# Searching the document

A document contains multiple lines.

Example:
```
Line 1 Line 2 Line 3

```
The search process:
```
Search line 1

```
|
```
v
```
Search line 2

```
|
```
v
```
Search line 3

```
The result contains:
```
line number

column number

```
The cursor can then move there.

---

# Search direction

Most editors support:

Forward search:
```
cursor —> end of file

```
Backward search:
```
cursor <— beginning of file

```
Forward search is simpler initially.

The search starts at:
```
current cursor position

```
and continues until:
```
end of document
```
```

If nothing is found:

Not found

Wrapping search

Many editors wrap around.

Example:

Cursor:

Line 90

Search reaches:

end of file

Instead of failing:

search from beginning

This requires a decision.

Should REMEDIT:

wrap automatically

or:

ask the user

For a first version, either is acceptable.

The important thing is consistency.

Search state

The editor should remember the last search.

Example:

SearchState

text

direction

case_mode

current_position

This allows:

Find next

without asking for the search text again.

14.2 Case handling

Case sensitivity is a design choice.

Consider:

MOV

and:

mov

Are they the same?

For assembly source, perhaps yes.

For programming languages with case-sensitive identifiers, perhaps no.

Case-sensitive search

The comparison is exact.

Example:

Search:

REM

Matches:

REM

Does not match:

rem

Case-insensitive search

The comparison normalises characters.

Example:

Search:

mov

Matches:

MOV Mov mov

```
The search algorithm compares:
```

uppercase(character)

```
or:
```

lowercase(character)

```
---

# The implementation question

Where should case handling happen?

Two options.

---

## Option 1 — Normalise during comparison

Example:
```

compare:

uppercase(a)

against:

uppercase(b)

```
Advantages:

- document remains unchanged
- simple

Disadvantages:

- slightly more processing

---

## Option 2 — Create normalised search data

Example:
```

Document:

MOV R1,R2

Search copy:

mov r1,r2

```
Advantages:

- faster repeated searching

Disadvantages:
```

- extra memory
- synchronisation problems

For REMEDIT version one:

Normalise during comparison.

The simplicity is worth more than the small performance difference.

14.3 Replacement

Replacement builds on search.

The operation:

Find text

Replace with different text

sounds simple.

However, replacement is actually a document modification.

That means it must obey all editor rules:

- update cursor
- update viewport
- create undo history
- maintain document consistency

Single replacement

Example:

Before:

MOV R1,R2

Search:

R1

Replace:

R5

After:

MOV R5,R2

The operation is:

delete old text

insert new text

Replace all

Example:

Before:
MOV R1,R2 MOV R1,R3 MOV R1,R4

Replace:
R1

with:
R5

After:
MOV R5,R2 MOV R5,R3 MOV R5,R4

The danger of replace all

A blind replacement can damage code.

Example:

Search:
A

Replace:
B

This may modify:
labels
comments
strings
identifiers

The editor does not know whether the text has meaning.

A literal replacement does exactly what was requested.

The user must decide whether that is appropriate.

Replacement strategy

A safe implementation:

Find first match

Show location

Ask confirmation

Replace

Continue

A faster mode:

Replace all

should clearly indicate:

number of replacements made

Search and replace in assembly source

REMEDIT will likely be used heavily for assembly.

Useful searches:

Find opcode:

LOAD

Find register:

R15

Find label:

main:

Find syscall:

write

The editor does not need to understand assembly.

It simply provides reliable text manipulation.

Syntax awareness comes later.

14.4 Interaction with undo

Search alone does not modify the document.

Replacement does.

Therefore replacement must integrate with undo.

```
---

# Single replacement

Example:

Before:
```

Hello World

```
Replace:
```

World

```
with:
```

REM

```
Undo record:
```

Type:

REPLACE_TEXT

Position:

line 1 column 6

Old:

World

New:

REM

```
Undo:
```

replace REM with World

```
---

# Replace all and undo

The question:

Should:
```

Replace all

```
create:
```

100 undo operations

```
or:
```

1 undo operation

```
Most editors choose one operation.

The user expects:
```

Ctrl-Z

```
to undo the entire command.
```

```
---
```

```
# Grouped undo command
```

```
Replace all can create a compound command.
```

```
Example:
```

Command:

REPLACE_ALL

Contains:

```
replacement 1
```

```
replacement 2
```

```
replacement 3
```

```
Undo:
```

reverse all replacements

```
This uses the command architecture from the previous chapter.
```

```
---
```

```
# Memory considerations
```

```
Replace all may create many changes.
```

```
Example:
```

```
A large source tree:
```

10,000 replacements

```
The undo system must store:
```

```
- locations
- old text
- new text
```

```
A practical solution:
```

```
Limit undo memory.
```

```
Possible policies:
```

Keep last N commands

```
or:
```

Keep up to X MB

```
---

# Search module design

The editor should isolate search functionality.

Example:
```
search.s

```
Functions:
```
search_find()
search_find_next()
search_find_previous()
replace_current()
replace_all()

```
The module communicates with:
```
Document
Cursor
Undo system

```
but not directly with:
```
Terminal

```
---

# Testing search and replace

Create:
```
search_test

```
Test:
```
find existing text
find missing text
case-sensitive search
case-insensitive search
replace one
replace many
undo replacement

```
---

# The REMEDIT milestone

After search and replace:
```

```
The editor now has the basic tools needed for real programming work.

A programmer can:
```

open source

navigate

edit

search

replace

undo mistakes

save

```
The next major step is making the editor understand the language being edited.

That is where syntax highlighting begins.


# Chapter 15 — REM Assembly Syntax Highlighting

Syntax highlighting is the first feature where REMEDIT begins to understand the
 ↪  language being edited.

However, an important distinction must be made:

The editor is not an assembler.

It does not:

- validate instructions
- calculate addresses
- resolve symbols
- generate machine code

That remains the responsibility of:
```

Source code

|

v

Assembler (GAS)

|

v

Object code

|

v

Linker

|

v

Executable

```
The editor only needs enough understanding to help the programmer read and navigate
 ↪  code.

This distinction keeps the design simple.

---

# 15.1 Lexer versus assembler

A common mistake when adding syntax highlighting is attempting to reuse the assembler.

The thought process is:

> "The assembler already understands REM assembly, so the editor should use the
 ↪  assembler."

This is usually the wrong approach.

The assembler has a different purpose.

---

# The assembler

The assembler must answer questions such as:
```

Is this instruction valid?

Does this register exist?

What machine code represents this instruction?

What address does this label refer to?

```
It needs complete knowledge of:

- instruction encoding
- ABI rules
- relocation
- object file format
- symbols

The assembler is a compiler tool.

---

# The lexer

A lexer only asks:
```

What kind of thing is this text?

```
Example:

Input:

MOV R1, 100      ; load value
```

The lexer identifies:

```
MOV
```

opcode ~~~

R1

```
register
```

100

```
number
```

; load value

```
comment
```

```
It does not decide whether:
```

MOV R1,100

```
is legal.

It only identifies categories.

---

# Why this separation matters

A syntax highlighter should still work while the programmer is writing broken code.

Example:

The user types:

MO
```

A good editor can already display:

```
MO
```

as probable instruction text.

The assembler cannot help because the instruction is incomplete.

The editor is helping the programmer write code.

It is not compiling code.

REMEDIT architecture

The flow should be:

```
Document
```

v ~~~

Lexer

|

v

Tokens

|

v

Renderer

|

v

Terminal colours

```
The lexer reads text.

The renderer decides how tokens appear.

---

# 15.2 Token types

The first REM assembly lexer does not need to understand everything.

A small number of token types provides most of the value.

Recommended initial types:
```

opcode

register

number

label

directive

comment

string

identifier

```
---

# Opcode

An opcode is an assembly instruction.

Examples:

MOV
ADD
SUB
LOAD
STORE
JMP
CALL
RET
```

The lexer compares a word against the instruction table.

Example:

Input:

```
ADD R1,R2,R3
```

Tokens:

```
ADD
```

OPCODE ~~~

R1

```
REGISTER
```

R2

```
REGISTER
```

R3

```
REGISTER

The renderer can display opcodes in a distinctive colour.

---

# Register

Registers are architecture-specific.

REM already has a defined register set.

Examples:
```

R0

R1

R15

SP

PC

```
The lexer needs a register table:
```

Register names:

R0

R1

R2

...

SP

PC

```
When a word matches:
```

REGISTER

```
is emitted.

---

# Number

Assembly contains many numeric forms.

Examples:

Decimal:

100
```

Hexadecimal:

```
0x1000
```

Binary:

```
0b1010
```

Character constants:

```
'A'
```

The lexer does not calculate the value.

It only identifies:

```
NUMBER
```

Label

Labels identify locations in code.

Example:

```
loop:
    ADD R1,R2,R3
    JMP loop
```

The lexer sees:

```
loop:
```

and emits:

```
LABEL
```

Important:

The lexer does not know the address.

That happens later during assembly.

Directive

Assembler directives begin with special syntax.

Examples:

```
.text
.data
.global main
.byte 10
.word 100
```

The editor can highlight these differently.

The lexer simply recognises:

```
DIRECTIVE
```

Comment

Comments are usually the easiest token.

Example:

```
ADD R1,R2,R3    ; add values
```

Everything after:

```
;
```

is:

```
COMMENT
```

The lexer does not inspect the comment contents.

Strings

Strings may appear in assembler data sections.

Example:

```
.ascii "Hello"
```

The lexer must recognise:

```
"Hello"
```

as one token.

Otherwise:

```
Hello
```

might incorrectly be treated as an identifier.

Token representation

The lexer produces records.

Example:

```
Token
```

type

start position

length

text reference ~~~

```
Example:

Input:

ADD R1,R2
```

Tokens:

```
TOKEN_OPCODE
```

ADD ~~~

TOKEN_REGISTER

```
R1
```

TOKEN_REGISTER

```
R2

The renderer uses the token type.

---

# 15.3 State machine design

Like the keyboard decoder, the lexer is easiest to implement as a state machine.

Do not attempt:
```

search for words

then guess meaning

```
Instead process the line from left to right.

---

# Lexer states

A simple assembly lexer:
```

START

v

WORD

+————————-+ | | v v

NUMBER IDENTIFIER

v

COMMENT

START state

The lexer is looking for the beginning of a token.

Possible characters:

Whitespace:

continue

Letter:

start word

Number:

start number

Comment marker:

start comment

WORD state

The lexer collects characters.

Example:

Input:

MOV

Characters:

M O V

The lexer builds:

MOV

Then checks:

Is this:

opcode? register? directive? identifier?

```
---

# NUMBER state

The lexer collects numeric characters.

Example:
0x1000

The lexer recognises:
0
x
1
0
0
0

and produces:
NUMBER

---

# COMMENT state

Once a comment marker is found:
;

everything until the end of the line is:
COMMENT

No further parsing is required.

This is an important optimisation.

---

# First REM lexer implementation

Do not attempt a complete assembler parser.

The first goal:

Make code readable.

Support:
opcode
register
number
```

label

directive

comment

```
That is enough to transform:

.global main

main:
    MOV R1,100
    ADD R2,R1,R3

    ; calculate result
```

into something much easier to read.

Integration with the renderer

The renderer receives:

```
line of text
```

The lexer produces:

```
tokens
```

The renderer outputs:

```
terminal commands
```

Example:

```
Opcode token
```

v ~~~

Set colour

```
|

v
```

Print text

```
---

# Performance considerations

Do not re-tokenise the entire file after every keystroke.

A simple first implementation may do this.

It is acceptable.

However, the better model is:
```

Edit line

|

v

Invalidate line tokens

|

v

Re-tokenise only changed lines

```
The document already knows which lines changed.

The lexer can use that information.

---

# Testing the lexer

Create:
```

lexer_test

```
Test input:

.global main

main:
    MOV R1,100      ; load value
```

Expected:

```
DIRECTIVE

LABEL

OPCODE

REGISTER

NUMBER

COMMENT
```

The REMEDIT milestone

After syntax highlighting:

The editor has become aware of the language.

The architecture now looks like:

```
Source text

|
```

v ~~~
Document
|
v
Lexer
|
v
Tokens
|
v
Renderer
|
v
Terminal

```
The assembler remains responsible for correctness.

The editor is responsible for helping the programmer think.

That is exactly the right division of responsibility.


# Chapter 16 — Building from the Editor

An editor becomes a development environment when it can perform the complete
↪  development cycle:
```

write code
|
v
build code
|
v
run or debug code

```
For REM, this is especially important.

The entire purpose of building REMEDIT is not just editing text.

It is creating a native environment where REM programs can be developed on REM itself.

The goal:
```

Edit assembly

|

v

Press F5

|

v

Assembler runs

|

v

Linker runs

|

v

Executable produced

The editor becomes the front end to the existing REM toolchain.

16.1 The F5 workflow

Many editors provide a build shortcut.

Examples:

- Emacs:

M-x compile

- IDEs:

Build button

For REMEDIT:

F5 = Build project

The user should not have to leave the editor.

Basic workflow

The sequence:

Flow: Save → clone → execve make → capture output → display errors

Each stage has a purpose.

Step 1 — Save

Before building:

The editor should save the current file.

Why?

Because the compiler works on files.

The compiler does not know about the unsaved text currently held in memory.

Example:

Editor contains:
main.s
MOV R1,100

Disk contains:
main.s
MOV R1,50

If the editor runs:
make

the assembler sees:
MOV R1,50

not:
MOV R1,100

The build command must operate on the current document.

Step 2 — clone

A running editor is a process.

The build command is another process.

The editor should not replace itself.

The relationship is:
REMEDIT
|
| clone()
|
+----------------+

```
                |
                v
            build process
```

The original editor continues running.

The child process prepares to run the build command.

On the current Linux port, this is implemented with the Linux userspace convention rather than a guessed desktop wrapper: put the syscall number in `r1`, put arguments in `r2`–`r7`, call the `syscall` instruction, and inspect the result in `r1`. The process launcher uses `clone` (220), `execve` (221), and `wait4` (260) in the currently exercised table. `execve` requires an executable path, an argument vector, and an environment vector. The parent must decode the `wait4` result and preserve the build exit status.

Why not simply run make directly?

Because:

make

would replace the editor process.

The editor would disappear.

The user could not return to editing.

The operating system process model exists specifically to allow this.

Step 3 — exec make

After clone:

The child process uses:

execve()

to replace itself with:

make

The flow becomes:

Child process

    |

    v

execve("/path/to/make", argv, envp)

```
    |
    v
```

make is now running

```
The editor process is unaffected.

---

# The complete process tree

During a build:
```

Linux kernel

```
  |
  +----------------+
  |                |
  v                v
```

REMEDIT make

```
                   |
                   v

              assembler

                   |
                   v

               linker

This is exactly how normal Linux development works.

REM is not inventing a special system.

It is providing a normal Unix development environment on a new architecture.

---

# Capturing build output

A build produces text:

Example:

Successful:
```

as main.s -o main.o

ld main.o -o main

```
Failed:
```

main.s:42: unknown instruction

```
The editor needs to capture this output.

---

# 16.2 Pipes

A pipe is a communication channel between processes.

The model:
```

Process A

|

v

pipe

|

v

Process B

```
For the build system:
```

make

|

v

pipe

|

v

REMEDIT

```
The build output becomes input for the editor.

---

# Standard output

Normally:
```

make

|

v

terminal

```
The user sees:
```

assembling... linking... error

```
With a pipe:
```

make

```
|

v
```

REMEDIT buffer

```
The editor can display:
```

Build Results

main.s:42 error: unknown opcode

```
---

# Standard error

Programs normally have two output streams:
```

stdout

stderr

```
stdout:

Normal output.

Example:
```

assembling main.s

```
stderr:

Errors and warnings.

Example:
```

main.s:42: invalid register

```
The editor should capture both.

---

# Pipe design

A simple build pipeline:
```

Create pipe

```
  |

  v
```

clone()

```
  |

  +----------------+
```

```
  |                |
  v                v
```

child parent

execve make read pipe

```
The child redirects:
```

stdout

stderr

```
    |
    v
```

pipe

```
The parent reads:
```

pipe

```
|
v
```

build output window

```
---

# Build output window

The editor should not overwrite the source file.

Instead:
```

Source
Build output error messages

```
The user can:

- inspect errors
- jump to source
- rebuild

---

# 16.3 Process control

Launching a process is only part of the problem.

The editor also needs to manage it.

---

# Waiting for completion
```

```
After starting:
```
make

```
the editor can wait:
```
waitpid()

```
The result:
```
Build complete
success

```
or:
```
Build failed
exit code 1

```
---

# Blocking versus asynchronous builds

Two approaches:

---

## Simple blocking build

The editor:
starts make
waits
shows result

Advantages:

- simple
- easy to debug

Disadvantages:

- editor freezes during build

For a first REMEDIT version:

This is acceptable.

---

## Asynchronous build

The editor:
starts make
continues editing
```

receives output later

```
Advantages:

- better user experience

Disadvantages:

- more complexity

Requires:

- process tracking
- event handling
- background output management

---

# Build status

The editor should know:
```

BUILDING

SUCCESS

FAILED

```
Example:

Status line:
```

Building...

```
then:
```

Build successful

```
or:
```

Build failed: 3 errors

```
---

# Terminating a build

Sometimes:
```

make

```
takes too long.

The user may want:
```

Ctrl-C

```
The editor may need to:

- send a signal
```

```
- terminate child processes
- clean up pipes

This is where the earlier terminal signal discussion becomes relevant.

---

# 16.4 Error navigation

A build environment becomes much more useful when errors are clickable.

Example compiler output:
```

main.s:42: unknown opcode MOVX

```
The editor should understand:
```

filename

line number

message

```
---

# Error parser

The editor can scan output.

Input:
```

main.s:42: error: unknown opcode

```
Extract:
```

file:

main.s

line:

42

```
Then provide:

Jump to error

---

# Error list

A useful design:

Build Errors

1. main.s line 42
2. memory.s line 17
3. linker error

Selecting an error:
```

move cursor to:

main.s

line 42

```
---

# The editor architecture now expands

Before:
```

Keyboard

|

v

Editor

|

v

Document

|

v

Renderer

```
After adding builds:
             Editor

                |

   +------------+------------+

   |                         |

   v                         v
```

Document Build system

```
                             |

                             v

                         make

                             |

                             v

                     assembler/linker

---
```

```
# REM development workflow

The final goal:
```

Open source file

    |

    v

Edit assembly

    |

    v

Press F5

    |

    v

Assembler runs

    |

    v

Linker creates executable

    |

    v

Run program

```
The editor becomes the place where REM software is created.

---

# Testing build integration

Create:
```

build_test

```
Test:
```

run successful command

capture output

capture errors

handle failure

parse error location

```
---

# The REMEDIT milestone

After this chapter:

REMEDIT is no longer just an editor.
```

```
It is a development environment.

It can:

- edit source
- understand assembly
- build programs
- display errors
- guide the programmer back to mistakes

This is the point where REM becomes self-hosting in spirit.

The machine is no longer just running software.

It is becoming capable of creating its own software.


# Part VI — Advanced REM Tools

# Chapter 17 — REM System Monitor

## Current status

The monitor described here is a planned REMEDIT project, except where a
specific diagnostic is identified as existing. The repository's `reminfo`
program reads the interfaces available in a particular Linux rootfs,
including `uname`, uptime, and selected `/proc` files. A disposable test
rootfs may not mount `/proc`, and a userspace monitor cannot read arbitrary
process registers without a kernel or debugger interface. Treat process,
register, and performance-counter displays as capability-dependent features.

A development environment is not complete when it can only create programs.

A programmer also needs tools to understand what those programs are doing.

Traditional Unix systems provide tools such as:
```

ps

top

vmstat

free

lsof

/proc

```
These tools answer questions:

- What programs are running?
- How much memory are they using?
- Are they consuming CPU?
- What resources do they have open?
- What is the system doing internally?

REM should eventually provide similar tools.

The purpose of a system monitor is not only convenience.
```

```
It is also a test of the architecture.

A system monitor exercises:

- process management
- memory management
- kernel interfaces
- scheduling
- hardware counters

If REM can support its own monitoring tools, the operating system environment is
 ↪  becoming mature.

---

# 17.1 The system monitor architecture

The system monitor sits above the kernel.

The architecture:
```

REM Monitor

  |

  v

System information interface

  |

  v

Linux kernel

  |

  v

REM CPU

```
The monitor should not directly inspect hardware.

It should ask the operating system:

"Tell me the current state."

---

# Information sources

A Unix-like system normally exposes information through:
```

system calls

/proc filesystem

kernel interfaces

```
The exact REM implementation depends on what the kernel provides.
```

```
The application should use the highest-level interface available.

---

# 17.2 Process monitoring

A process is a running program.
```

Examples:
remedit
shell
assembler
linker

```
The monitor should display:
```
PID
name
state
CPU usage
memory usage

```
Example:
```
PID NAME STATE
1 init running
20 shell sleeping
35 remedit running
42 make waiting

Process identifiers

Every running process has an identifier:
```
PID

```
Example:
```
PID 35

The kernel uses this value to identify the process.

Tools such as:
```
kill

```
use the PID:
```
kill 35

```
meaning:

"send a signal to process 35."

---

# Process states

A process is not always running.

Common states:
RUNNING
READY
SLEEPING
WAITING
STOPPED

Example:

A shell waiting for input:
SLEEPING

An assembler currently executing:
RUNNING

Understanding process states helps explain system behaviour.

---

# Process tree

Processes have relationships.

Example:
init
+– shell
   |
   +-- remedit

   |
   +-- make

          |
          +-- assembler

          |
          +-- linker

A useful monitor view:
```

PID PROCESS
1 init 10 shell 20 remedit 30 make 31 as 32 ld

```
This immediately shows how programs are related.

---

# 17.3 Memory monitoring

Memory is one of the most important parts of a new CPU architecture.

The monitor should display:
```
total memory
used memory
free memory
process memory

```
---

# Virtual memory view

Modern Linux programs do not normally use physical addresses directly.

The model is:
```
Application
 |
 v
Virtual address
 |
 v
MMU
 |
 v
Physical memory

The monitor should reflect this.

For a process:
```
Virtual memory:
code
data
heap
stack

```

```
```

```
# Process memory layout

A typical process:
```




```
The monitor can display:
```

PID 35 remedit

Code: 128 KB

Data: 32 KB

Heap: 512 KB

Stack: 64 KB

```
---

# Memory debugging

A system monitor becomes useful when debugging.

Example:

A program repeatedly allocates memory.

Monitor:
```

remedit

Heap:

100 KB

200 KB

300 KB

400 KB

```
The programmer sees:

"Something is leaking memory."

---

# 17.4 Register display

Because REM is a new CPU architecture, register inspection is especially valuable.

A monitor or debugger should eventually display:
```

Program Counter

Stack Pointer

General Registers

Status Registers

Example:

PC:

0x00401230

SP:

0x7FFF0000

R0:

0x00000000

R1:

0x00000042

Why registers matter

When debugging a normal application:

A programmer asks:

"What did my program calculate?"

On a new CPU:

They also ask:

"What did the processor actually do?"

Register inspection helps answer:

- Did the instruction execute?
- Did the value change?
- Did the function call work?
- Is the stack correct?

Accessing another process

A running program cannot normally inspect another process's registers.

The operating system must provide a mechanism.

Possible approaches:

debug syscall

ptrace-style interface

kernel debugger support

The system monitor does not invent this.

It uses the kernel interface.

```
---

# 17.5 Performance monitoring

A CPU architecture is not only about correctness.

Performance matters too.

The monitor can display:
```

CPU usage

instruction count

memory activity

system calls

```
---

# CPU usage

A simple measurement:
```

How much CPU time did this process receive?

```
Example:
```

PID CPU

10 2%

20 85%

30 13%

```
---

# Instruction counters

A REM-specific feature could be hardware counters.

Examples:
```

instructions executed

cache misses

branch events

exceptions

```
These are useful when improving the CPU itself.

---

# Example:

Compare two algorithms.

Program A:
```

instructions:

10,000

Program B:

instructions:

5,000

The monitor provides evidence rather than guesswork.

Designing the REM monitor

A simple first version:

remmon

Features:

process list

memory summary

CPU usage

register display

Interface:

REM System Monitor
PID NAME CPU MEM 1 init 0% 32K 5 shell
1% 64K 8 remedit 12% 512K

Assembly design considerations

A system monitor is a good assembly project because it exercises many areas:

- structures
- formatting numbers
- system calls
- terminal control
- refresh loops

However, do not begin with the display.

Build the data collection first.

Recommended implementation order

Phase 1 — Process listing

Prove:

Can I enumerate processes?

```
---

## Phase 2 — Memory information

Add:
```

memory usage

process sizes

```
---

## Phase 3 — Refresh display

Add:
```

update every second

redraw screen

```
---

## Phase 4 — Register/debug information

Requires deeper kernel integration.

---

# Testing

Create:
```

monitor_test

```
Test:
```

launch process

observe process

terminate process

observe removal

```
Memory test:
```

allocate memory

watch usage increase

free memory

watch usage decrease

```
---

# The REM milestone

After building a system monitor, REM gains an important capability:
```

```
It can observe itself.

The progression becomes:

**Flow:** CPU → Linux kernel → Applications → Development tools → Tools that analyse
 ↪  applications

This is the point where a computer architecture stops being only an execution platform.

It becomes an environment where programmers can understand, debug and improve the
 ↪  machine itself.


# Chapter 18 — REM Debugger Front End

## Target distinction

The existing `tools/mydebug` program and `docs/mydebug.md` describe the
legacy MyEmulator target, whose register widths and machine differ from
MyEmulator2. They are not automatically a Linux guest debugger. MyEmulator2
debugging currently depends on host QEMU debug facilities and project
experiments; a stable guest debugger or `ptrace` ABI is future work. Any
breakpoint or register example in this chapter must identify which of those
interfaces it uses.

A compiler and an editor allow a programmer to create software.

A debugger allows a programmer to understand software.

For a new CPU architecture, a debugger is not just a convenience.

It is one of the most important development tools.

When REM software fails, the programmer needs answers:
```

What instruction was executing?

What are the register values?

What memory was accessed?

How did execution get here?

```
A debugger provides visibility into the running machine.

---

# 18.1 The debugger architecture

A debugger has two parts:
```

Debugger front end

    |

    v

Debug interface

    |

    v

Kernel / CPU support

    |

    v

Running process

```
The front end is the user interface.

It provides:

- commands
- displays
- navigation
- source views

The lower layer provides:

- stopping execution
- reading registers
- modifying memory
- continuing execution

---

# The debugger is not the CPU

A common mistake is thinking:

> "The debugger reads the CPU directly."

Normally it does not.

The CPU executes instructions.

The kernel controls processes.

The debugger communicates through operating system mechanisms.

The path is:
```

Debugger

|

v

Kernel debug interface

|

v

Process state

|

v

REM CPU

```
---

# Debugging a program

The normal workflow:

Write code

|

v

Build program

|

v

Start debugger

|

v

Run program

|

v

Stop at interesting point

|

v

Inspect state

The programmer moves between:

- source code
- machine instructions
- CPU state

---

# 18.2 Breakpoints

A breakpoint tells the debugger:

"Stop execution here."

Example:

Assembly:

main:

    MOV R1,10

    ADD R2,R1,R3
```

```
    RET
```

The programmer sets:

```
break main
```

Execution:

Flow: Start program → Execute MOV → Breakpoint reached → CPU stopped

The debugger now has control.

Why breakpoints matter

Without breakpoints:

Flow: Program starts → Program runs → Program fails

The interesting moment may have already passed.

A breakpoint allows:

```
Stop before the failure

Inspect state

Continue carefully
```

Software breakpoints

The traditional method:

Replace an instruction temporarily.

Example:

Original:

```
1000:

ADD R1,R2,R3
```

Debugger changes memory:

```
1000:

BREAK
```

When execution reaches that address:

Flow: CPU exception → Kernel notified → Debugger takes control

The debugger later restores the original instruction.

Hardware breakpoints

Some CPUs provide dedicated breakpoint registers.

Example:

```
Breakpoint register 0:

Address = 0x1000
```

The CPU automatically stops when:

```
PC == 0x1000
```

Advantages:

- does not modify program memory
- useful for read-only code

Disadvantages:

- limited number of breakpoints

REM breakpoint design

Because REM is a new architecture, breakpoint support must be designed.

Questions:

Does REM provide:

- a breakpoint instruction?
- a debug exception?
- trap handling?
- debug registers?

A clean design might provide:

Flow: DEBUG exception → kernel → debugger

18.3 Register display

Once execution stops, the first question is usually:

“What does the CPU contain?”

The debugger should display:

```
Program Counter

Stack Pointer

General Registers

Status Flags
```

Example:

```
PC:

0x00401234


SP:

0x7FFF0000


R0:

0x00000000


R1:

0x0000000A


R2:
```

```
0x00000014
```

Why registers matter

Consider:

```
ADD R2,R1,R3
```

The programmer expects:

```
R2 = R1 + R3
```

The debugger allows verification:

Before:

```
R1 = 10

R3 = 20

R2 = 0
```

After:

```
R2 = 30
```

If the result is wrong:

The problem is either:

- instruction implementation
- register handling
- program logic

The debugger narrows the problem.

Register commands

A simple debugger interface:

```
registers
```

Output:

```
R0  00000000

R1  0000000A

R2  0000001E

PC  00401234

SP  7FFF0000
```

A future graphical interface can display the same information.

18.4 Memory inspection

Registers show the immediate CPU state.

Memory inspection shows the larger picture.

Useful questions:

```
What is stored at this address?

Did my stack change?

Was this buffer overwritten?
```

Memory dump

Example command:

```
dump 0x1000 64
```

Output:

```
1000:

48 65 6C 6C 6F 00 00 00

Hello...
```

The debugger can display:

- hexadecimal
- decimal
- ASCII

Stack inspection

The stack is critical in assembly.

Example:

```
Stack:

+----------------+

return address

+----------------+

saved registers

+----------------+

local variables

+----------------+
```

A debugger should make this visible.

Example:

```
stack

0x7FFF1000:

return = 0x401230

saved R1 = 10
```

Memory errors

Many assembly bugs are memory bugs.

Examples:

Wrong pointer:

`LOAD R1,[wrong address]`

Buffer overwrite:

`write beyond allocation`

Stack corruption:

`incorrect return address`

Memory inspection helps identify the damage.

18.5 Single stepping

Single stepping means:

Execute one instruction.

Stop.

Show the new state.

Repeat.

The workflow:

Flow: Instruction → CPU executes → Debugger stops → Display state

---

Source-level stepping

A modern debugger may show:

```
MOV R1,10
```

then:

`next`

and move to:

```
ADD R2,R1,R3
```

This requires:

- debug symbols

- source mapping

Instruction-level stepping

For a new CPU, instruction stepping is the foundation.

Example:

Current:

```
PC = 0x1000
```

Instruction:

```
MOV R1,10
```

After step:

```
PC = 0x1004
```

```
R1 = 10
```

The debugger shows exactly what happened.

Why single stepping matters for REM

When developing a new CPU:

You are debugging at several levels:

Flow: Application bug → Assembly bug → Compiler bug → CPU implementation bug

Single stepping allows checking:

"Did the CPU execute the instruction correctly?"

Debugger commands

A minimal REM debugger might provide:

```
run
```

```
continue
```

```
step
```

```
break address
```

```
registers
```

```
memory address
```

```
quit
```

Example session:

```
remdbg program
```

```
> break main
```

```
> run

Breakpoint reached

> registers

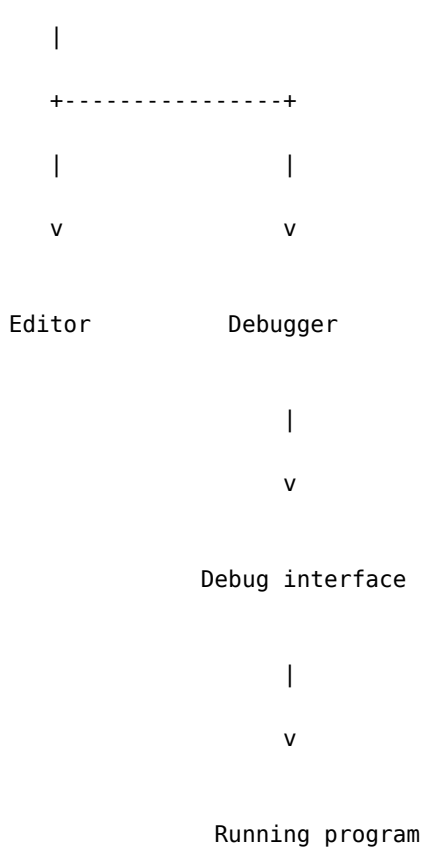
> step

> memory 0x1000
```

Integration with REMEDIT

Eventually REMEDIT can become the debugger front end.

The architecture:



The programmer can:

- edit source
- build
- run
- debug

without leaving the environment.

Debug information

To connect source code and machine instructions, the build process needs metadata.

Example:

Machine code:

```
0x401230
```

```
MOV R1,10
```

Source:

```
main.s line 5
```

The debugger needs a mapping:

```
Address
```

```
    |
```

```
    v
```

```
Source file + line
```

This is usually provided by debug symbols.

Testing the debugger

Start with the simplest possible program:

```
main:
    MOV R1,1
    ADD R1,R1,R1
    RET
```

Test:

```
break main
```

```
run
```

```
registers
```

```
step
```

```
registers
```

Verify:

```
R1 changes correctly
```

```
PC advances correctly
```

The REM milestone

After the debugger exists, REM gains the ability to inspect itself.

The development environment becomes:

```
Editor

    |

    v

Assembler

    |

    v

Linker

    |

    v

Program

    |

    v

Debugger

    |

    v

Understanding
```

A new CPU architecture without debugging tools is difficult to develop.

A new CPU architecture with its own debugger becomes a platform.

```
# Chapter 19 — Disassembly Tools

The current MyEmulator2 toolchain already provides target disassembly through
`myemulator2-elf-objdump`. This chapter therefore has an implementable first
milestone, unlike the planned monitor and guest-debugger features.

A debugger shows a program while it is running.

A disassembler shows a program when all you have is the machine code.

For a new CPU architecture, a disassembler is one of the first tools that proves the
 ↪  instruction set is understandable outside the emulator itself.

The complete software flow is:
```

Assembly source

```
  |
  v
```

Assembler

```
  |
  v
```

Machine code

```
  |
  v
```

Disassembler

```
  |
  v
```

Human-readable instructions

```
The assembler converts human language into machine language.

The disassembler performs the reverse process:
```

Machine language

```
  |
  v
```

REM assembly syntax

```
It will not recover the original source perfectly.

Comments, formatting, variable names and many labels are lost.

However, it can recover the instructions.

---

# 19.1 Why write a REM disassembler?

REM already has:

- GAS support
- LD support
- ELF executables
- an instruction set

The next natural tool is:
```

remdump

```
or:
```

remobjdump

```
A disassembler is useful for:
```

```
- debugging compiler output
- inspecting generated code
- verifying the assembler
- understanding binaries
- reverse engineering failures

---

# The disassembler architecture

The design:
```

ELF file

|

v

ELF reader

|

v

Code section

|

v

Instruction decoder

|

v

Assembly output

```
Each stage has a separate responsibility.

---

# 19.2 Reading ELF files

REM executables use ELF because Linux already understands this format.

ELF:
```

Executable and Linkable Format

```
is the standard structure used by Linux binaries.

A file contains:
```

ELF header

Program headers

Section headers

Sections

Symbol tables

Debug information

```
A disassembler does not need all of these initially.

The first version only needs:

- ELF header
- section table
- code section

---

# ELF header

The ELF header describes the file.

Important fields:
```

Magic number

Architecture

Entry point

Section table location

Number of sections

```
The first check:
```

Is this actually a REM executable?

```
The ELF magic bytes:
```

7F 45 4C 46

```
identify the file as ELF.

---

# Sections

An executable contains different sections.

Typical example:
```

.text

```
program instructions
```

.data

```
initialised data
```

.bss

```
zero-filled data

The disassembler normally starts with:
```

.text

```
because this contains executable instructions.

---

# Example

Executable:
```

hello

```
contains:
```

.text

0x1000: MOV R1,10 CALL print RET

.data

message: “Hello”

```
The disassembler reads only:
```

.text

```
and converts bytes back into instructions.

---

# Reading the section

The algorithm:
```

Open ELF

```
|

v
```

Read ELF header

```
|

v
```

Find section table

```
|

v
```

Locate .text

```
|

v
```

Read machine code bytes

```
The output is a byte stream:
```

01 10 00 0A 02 20 30 00 FF 00 00 00

```
The instruction decoder interprets those bytes.
```

```
---

# 19.3 Instruction decoding

The decoder is the heart of the disassembler.

The CPU already performs this operation:
```

Machine code

```
  |

  v
```

Instruction decoder

```
  |

  v
```

CPU operation

```
The disassembler performs the same first step.

However, instead of executing:
```

ADD R1,R2,R3

```
it prints:
```

ADD R1,R2,R3

```
---

# Opcode tables

The decoder requires a table.

Example:
```

Opcode

Mnemonic

Format

```
Example:
```

0x01

MOV

destination,source

0x02

ADD

destination,left,right

0x03

JMP

address

```
The table is shared knowledge between:

- assembler
- emulator
- disassembler

---

# Instruction decoding process

Example bytes:
```

02 01 02 03

```
Decoder:

Read first byte:
```

02

```
Lookup:
```

0x02 = ADD

```
Instruction format:
```

ADD register,register,register

```
Remaining bytes:
```

01 02 03

```
become:
```

R1,R2,R3

```
Output:
```

ADD R1,R2,R3

```
---

# Instruction formats

The decoder must understand every REM instruction format.

Examples:

Register format:
```

opcode
destination register
source register
source register

```
Immediate format:
```

opcode

register

constant value

Branch format:

opcode

address offset

Invalid instructions

The disassembler must handle unknown bytes.

Example:

FF FF FF FF

Possible output:

UNKNOWN 0xFFFFFFFF

Do not crash.

A disassembler is often used when investigating damaged programs.

Addresses

A useful disassembler shows addresses.

Example:

00401000:

MOV R1,10

00401004:

ADD R2,R1,R3

00401008:

RET

The address is important because:

- debuggers use addresses
- crash reports use addresses
- branch targets use addresses

19.4 Symbols and labels

Machine code contains addresses.

Humans prefer names.

Example:

Machine:

CALL 0x401000

Better:

CALL print_message

The linker creates symbol information.

The disassembler can use it.

Symbol table

The ELF symbol table contains:

symbol name

address

type

Example:

401000

main

401050

print_message

The disassembler can replace:

CALL 401050

with:

CALL print_message

19.5 Source mapping

A normal disassembler shows instructions.

A better one connects instructions back to source.

Example:

Assembly:

```
main:

    MOV R1,10
```

```
    ADD R2,R1,R3
```

Machine:

```
00401000:

MOV R1,10


00401004:

ADD R2,R1,R3
```

With debug information:

```
main.s:5

00401000:

MOV R1,10


main.s:6

00401004:

ADD R2,R1,R3
```

Debug information

The compiler and assembler can emit metadata.

This maps:

```
machine address

        |

        v

source file and line
```

The debugger uses this.

The disassembler can display it.

Why this matters for REM

A new architecture has a unique problem:

When something goes wrong, you need to know:

```
Did the assembler generate the wrong code?

Did the linker place it incorrectly?

Did the CPU execute it incorrectly?
```

The disassembler provides the middle ground.

Example:

Source:

```
ADD R1,R2,R3
```

Disassembly:

```
00400020:

ADD R1,R2,R3
```

If the output is wrong:

The assembler is suspect.

If the output is correct:

Look elsewhere.

Building remobjdump

A practical development order:

Phase 1 — Raw byte display

Read ELF.

Display:

```
.text bytes
```

Goal:

Prove ELF reading works.

Phase 2 — Decode instructions

Add:

```
opcode lookup

register decoding

immediate values
```

Goal:

Produce assembly output.

Phase 3 — Symbols

Add:

```
function names
```

```
labels
```

Goal:

Make output readable.

Phase 4 — Debug information

Add:

```
source filenames
```

```
line numbers
```

Goal:

Connect binary to source.

Testing the disassembler

Create a small program:

```
main:

    MOV R1,10

    ADD R2,R1,R1

    RET
```

Build:

```
assembler
```

```
linker
```

Then:

```
remobjdump program
```

Expected:

```
00400000:

MOV R1,10


00400004:

ADD R2,R1,R1


00400008:

RET
```

The disassembler should exactly match the original instructions.

The REM toolchain

At this point the REM development environment contains:

```
Editor

    |

Assembler

    |

Linker

    |

Executable

    |

Disassembler

    |

Debugger
```

The programmer can now move in both directions:

Source:

```
assembly
```

to:

```
machine code
```

and back:

```
machine code
```

to:

```
understandable instructions
```

That feedback loop is essential when creating software for a new CPU architecture.

A processor is not truly usable when it can execute instructions.

It becomes usable when programmers can understand, debug and improve the programs running on it.

```
# Part VII — Flight Projects

# Chapter 20 — The Two Flight Plan

Building software on an aeroplane is a different environment from normal development.

There is no second monitor.
```

There is no quick internet search.

There is no asking another developer:

> "Have you seen this bug before?"

The environment is deliberately constrained:

REM machine

-

assembler

-

linker

-

editor

-

documentation

The objective is therefore not to write the largest amount of code.

The objective is to create a sequence of small victories where every stage leaves a working system behind.

A senior systems programmer follows one rule:

> Never make the next layer depend on an unproven lower layer.

Before building an editor, prove the terminal works.

Before building the editor commands, prove the buffer works.

Before building the build system, prove the process model works.

The Two Flight Strategy

The project naturally divides into two journeys.

The outbound flight is about creating the editor foundation.

The return flight is about turning that editor into a REM development environment.

The split is deliberate.

The first flight produces something usable.

The second flight adds intelligence and integration.

Outbound Flight

```
## Objective

Build the minimum native REM editor.

By the end of this phase:
```
REMEDIT v0.1

```
should be able to:
```
start
display text
accept keyboard input
modify text
save a file
exit cleanly

```
This is already a significant milestone.

The machine has created a native tool.

---

# Phase 1 — Terminal Layer

## Goal

Teach REMEDIT how to control the terminal.

Before writing an editor, create the foundation.

Create:
```
terminal.s

```
Responsibilities:
```
terminal_init()
terminal_restore()
terminal_clear()
terminal_move()
terminal_write()
terminal_colour()

Why terminal comes first

A common beginner mistake is:
```
Write editor code
then discover the display is wrong
```

```
This creates confusing failures.

Instead:
```

Terminal test

    |

    v

Keyboard test

    |

    v

Editor

```
Each layer proves itself.

---

# Terminal knowledge required

A terminal is not a graphical display.

The editor writes characters and control sequences.

Example:

Move cursor:
```

ESC [y ; x H

```
Clear screen:
```

ESC [2 J

```
Change colour:
```

ESC [3 1 m

```
The terminal emulator interprets these sequences.

REMEDIT does not draw pixels.

It sends commands.

---

# Terminal test program

Before the editor exists, create:
```

terminal_test

```
It should prove:
```

clear screen

move cursor

print text

change colours

restore terminal

```
If this fails, stop.

Do not continue.

---

# Phase 2 — Keyboard Layer

## Goal

Convert raw terminal input into editor commands.

Create:
```

keyboard.s

```
The terminal does not send:
```

UP ARROW

```
It sends characters.

Example:
```

ESC [A

```
The keyboard layer converts this into:
```

KEY_UP

```
---

# Keyboard interface

The editor should not know about terminal escape sequences.

The editor should receive:
```

CHARACTER

KEY_UP

KEY_DOWN

KEY_LEFT

KEY_RIGHT

KEY_DELETE

KEY_HOME

KEY_END

```
The separation is important.
```

```
Later, the terminal code can change without rewriting the editor.

---

# Keyboard state machine

A simple design:

**Flow:** NORMAL → ESCAPE → CSI → FUNCTION KEY

Example:

Input:
```
ESC [A

```
Processing:
```
ESC
[
A
KEY_UP

Keyboard test program

Create:
keyboard_test

Verify:
letters
numbers
arrow keys
delete
escape
function keys

Phase 3 — Text Buffer

Goal

Create the document model.

Do not start with screen rendering.

The document is the real application.

The screen is only a view.
```
```

```
---

# First REMEDIT storage model

Use the simplest design first:
Document
|
+-- Line
+-- Line
+-- Line

Each line contains:
address
length
characters

---

# Required operations

The first buffer must support:
insert character
delete character
split line
join line

These operations form the foundation of editing.

---

# Buffer testing

Create:
buffer_test

Test without a terminal:
create document
insert text
delete text
split line
join lines
verify result

This is important.
```

```
A failing editor is difficult to debug.

A failing buffer test is simple.

---

# Phase 4 — Editor Core

Now combine:
```

Terminal

-

Keyboard

-

Buffer

```
Architecture:
```

Keyboard

|

v

Command processor

|

v

Document

|

v

Renderer

|

v

Terminal

```
---

# The editor loop

The heart of the editor:
```

read input

  |

  v

interpret command

```
  |
  v
```

modify document

```
  |
  v
```

update cursor

```
  |
  v
```

render screen

```
Keep this loop simple.

Do not mix:

- terminal control
- file handling
- editing logic

inside one large function.

---

# Outbound milestone

At the end of the outbound flight:

REMEDIT should be able to:
```

open file

move cursor

type text

delete text

save file

exit

```
The editor does not need:
```

syntax highlighting

build integration

debugger support

```
Those come later.

The foundation matters more.

---

# Return Flight
```

```
## Objective

Turn REMEDIT from an editor into a REM development environment.

The machine can already edit.

Now make it understand and build REM software.

---

# Phase 5 — Syntax Highlighting

## Goal

Make REM assembly readable.

Add:
```
lexer.s

```
The lexer identifies:
```
opcode

register

number

label

directive

comment

```
Example:

```
main:

    MOV R1,100

    ADD R2,R1,R3

    ; calculate value
```
The editor highlights the structure.

---

## Important rule

The lexer is not the assembler.

The assembler answers:

```
Is this valid?
```

```
What machine code does this create?
```

The lexer answers:

```
What kind of text is this?
```

The editor should still work while code is incomplete.
```

Phase 6 — Build Integration

Goal

Press one key and build.

The workflow:

Flow: Save → clone → execve make → capture output → display errors

---

F5 workflow

The user writes:

`main.s`

Presses:

`F5`

The editor:

1. Saves the file.
2. Starts the build process.
3. Captures output.
4. Displays errors.

Build output

Instead of:

`terminal full of compiler messages`

show:

```
+----------------------+
| Source               |
|                      |
|                      |
+----------------------+
| Build results        |
| main.s:42 error      |
+----------------------+
```

The editor becomes the development environment.

Phase 7 — REM Tooling

Once the editor exists, add native REM tools.

Possible projects:

```
remobjdump

remdbg

remmon
```

---

remobjdump

Purpose:

Understand binaries.

Features:

```
read ELF

decode instructions

show symbols

show source mapping
```

Useful for:

- debugging assembler output
- verifying the CPU
- investigating failures

---

remdbg

Purpose:

Understand running programs.

Features:

```
breakpoints

register display

memory inspection

single stepping
```

---

remmon

Purpose:

Understand the system.

Features:

```
process list

memory usage
```

```
CPU statistics

performance counters
```

Recommended order

The complete roadmap:

```
Outbound flight

1. Terminal

2. Keyboard

3. Buffer

4. Editor core


Return flight

5. Syntax highlighting

6. Build integration

7. REM tools

8. Debugging environment
```

Engineering discipline during flights

Limited environments reward discipline.

Maintain:

```
src/

tests/

build/

docs/
```

Every subsystem gets:

```
implementation

test program

documentation
```

End-of-session rule

Never stop with the system broken.

Good stopping point:

```
Keyboard decoder completed.
```

Good stopping point:

```
Insert character test passes.
```

Bad stopping point:

```
Halfway through rewriting the renderer.
```

Always return to a working checkpoint.

Final objective

The real goal is not a text editor.

The goal is the first stage of a self-hosting REM environment.

The progression:

```
REM CPU

    |

    v

Linux

    |

    v

Native assembler

    |

    v

Native editor

    |

    v

Native tools

    |

    v

REM develops REM
```

A CPU architecture becomes a computer when it can support the tools required to create the next generation of software.

```
# Appendices
```

```
---


# Part VIII — Worked REM Programs

The earlier chapters explain the design decisions. This part supplies the
small executable bridges that connect those decisions to the toolchain. Each
example is intentionally small. Copy it into a new directory, build it, and
change one thing at a time. The target compiler, assembler, linker, and QEMU
paths are the ones used by the repository tests.

## 21.1 A function with a stack frame

This function receives two values in `r1` and `r2`, preserves the callee-saved
`r5`, and returns the sum in `r1`.

.text
.global add_saved
.type add_saved, @function
add_saved:
        addi sp, sp, -16
        sw   r5, 0(sp)
        add  r5, r1, r0
        add  r1, r5, r2
        lw   r5, 0(sp)
        addi sp, sp, 16
        ret
.size add_saved, .-add_saved
```

Assemble it with `myemulator2-elf-as`, inspect it with `myemulator2-elf-objdump -dr`, and check that the stack returns to its entry value before `ret`. The frame remains 16-byte aligned at the call boundary.

21.2 C calling assembly

`add_two.S` contains the assembly function above without the save/restore work because it only uses caller-saved registers:

```
.text
.global add_two
.type add_two, @function
add_two:
        add r1, r1, r2
        ret
.size add_two, .-add_two
```

The C caller is:

```
extern int add_two(int, int);
int main(void) { return add_two(20, 22) == 42 ? 0 : 1; }
```

Build the pair with the target driver:

```
MYEMU_TARGET_GCC=.toolchain-install/bin/myemulator2-elf-gcc \
MYEMU_MUSL_PREFIX=.musl-install \
toolchain/scripts/myemulator2-musl-gcc -O2 -static main.c add_two.S -o c_calls_asm
```

The process exits with status zero when the C and assembly views of the ABI agree. This example deliberately uses no variadic function and no aggregate return.

21.3 Assembly calling C

Assembly can call a normal C function once the compiler-provided startup has run. This `main` calls `puts`; the first argument is in `r1` and the return value from `puts` is ignored.

```
.text
.global main
.type main, @function
.extern puts
main:
        addi sp, sp, -16
        sw   r14, 0(sp)
        lui  r1, %hi(message)
        ori  r1, r1, %lo(message)
        jal  puts
        lw   r14, 0(sp)
        addi sp, sp, 16
        li   r1, 0
        ret
.size main, .-main
.section .rodata
message: .asciz "assembly called C"
```

Build it with the same static musl driver. Do not call libc from a hand-written `_start` until the runtime startup contract has initialized the environment.

21.4 A raw syscall wrapper

This is a complete `write` wrapper using the Linux syscall convention. The C function arguments arrive in `r1`–`r3`; the kernel wants them in `r2`–`r4`.

```
.text
.global rem_write
.type rem_write, @function
rem_write:
        add r4, r3, r0
        add r3, r2, r0
        add r2, r1, r0
        li  r1, 64          # __NR_write
        syscall 0
        ret                 # r1 is bytes written or -errno
.size rem_write, .-rem_write
```

The immediate `0` on `syscall 0` is the instruction's diagnostic field. It is not the Linux syscall number; the number is in `r1`. Treat `r1 < 0` as failure and preserve any live caller values before the instruction.

21.5 A complete static libc program

```
#include <stdio.h>
int main(void) {
    puts("Hello from REM libc");
    return 0;
}
```

Build, inspect, and run it through the repository's Linux fixture:

```
MYEMU_TARGET_GCC=.toolchain-install/bin/myemulator2-elf-gcc \
MYEMU_MUSL_PREFIX=.musl-install \
```

```
toolchain/scripts/myemulator2-musl-gcc -O2 -static hello.c -o hello
myemulator2-elf-readelf -h -l hello
python3 toolchain/scripts/test-linux-native-gcc.py
```

The regression script supplies the kernel and disposable root filesystem. Its success criterion is the program's output followed by a zero exit status.

21.6 Raw terminal setup, one key, and restoration

This compact C program saves termios, enters one-byte input mode, reads one byte, and restores the terminal on every normal path.

```
#include <signal.h>
#include <stdio.h>
#include <termios.h>
#include <unistd.h>

static struct termios saved;
static volatile sig_atomic_t stop;
static void on_signal(int sig) { (void)sig; stop = 1; }
static void restore(void) { const char end[] = "\\x1b[?25h\\x1b[0m\\n";
    tcsetattr(STDIN_FILENO, TCSAFLUSH, &saved);
    write(STDOUT_FILENO, end, sizeof end - 1); }

int main(void) {
    struct termios raw;
    if (tcgetattr(STDIN_FILENO, &saved) < 0) return 1;
    raw = saved;
    raw.c_lflag &= ~(ICANON | ECHO | ISIG);
    raw.c_cc[VMIN] = 1; raw.c_cc[VTIME] = 0;
    if (tcsetattr(STDIN_FILENO, TCSAFLUSH, &raw) < 0) return 1;
    signal(SIGINT, on_signal); signal(SIGTERM, on_signal); signal(SIGHUP, on_signal);
    { const char prompt[] = "\\x1b[?25lPress a key\\r\\n";
      write(STDOUT_FILENO, prompt, sizeof prompt - 1); }
    unsigned char ch;
    while (!stop && read(STDIN_FILENO, &ch, 1) < 1) { }
    restore();
    return 0;
}
```

The handler only records a flag. Terminal calls happen after control returns to the main loop, where they are safe to perform.

21.7 A keyboard escape decoder

The decoder below recognizes ordinary bytes, Escape, and the four arrow sequences. It uses `VMIN=0` and `VTIME=1` while waiting after `ESC`, so an isolated Escape becomes an Escape event after a short tenth-of-a-second wait.

```
#include <poll.h>
#include <stdio.h>
#include <unistd.h>

enum key { KEY_CHAR, KEY_ESC, KEY_UP, KEY_DOWN, KEY_RIGHT, KEY_LEFT };
static int next_key(unsigned char *value) {
    unsigned char b[3]; ssize_t n = read(0, b, 1);
    if (n != 1) return -1;
    if (b[0] != 0x1b) { *value = b[0]; return KEY_CHAR; }
```

```
    struct pollfd p = { .fd = 0, .events = POLLIN };
    if (poll(&p, 1, 100) <= 0) return KEY_ESC;
    if (read(0, b + 1, 1) != 1) return KEY_ESC;
    if (b[1] == '[' && read(0, b + 2, 1) == 1) {
        if (b[2] == 'A') return KEY_UP;
        if (b[2] == 'B') return KEY_DOWN;
        if (b[2] == 'C') return KEY_RIGHT;
        if (b[2] == 'D') return KEY_LEFT;
    }
    return KEY_ESC;
}
```

A production decoder extends this state machine for numeric CSI parameters, SS3 function keys, and partial reads. The important boundary is that the editor receives key events rather than raw terminal bytes.

21.8 Partial I/O and retry errors

A `read` or `write` may transfer fewer bytes than requested. This helper writes the complete buffer, retrying only `EINTR` and reporting all other errors.

```
#include <errno.h>
#include <unistd.h>
ssize_t write_all(int fd, const void *data, size_t length) {
    const char *p = data; size_t done = 0;
    while (done < length) {
        ssize_t n = write(fd, p + done, length - done);
        if (n > 0) { done += (size_t)n; continue; }
        if (n < 0 && errno == EINTR) continue;
        return -1;
    }
    return (ssize_t)done;
}
```

Raw assembly performs the equivalent test against the negative kernel result and compares it with the target's `-EINTR` value; libc has already translated the result to `-1` and set `errno`.

21.9 File copy and atomic save

This program shows the complete open/read/write/close path. A real editor writes a temporary file in the same directory, calls `fsync`, closes it, and then renames it over the original.

```
#include <fcntl.h>
#include <unistd.h>
#include <stdio.h>
int copy_file(const char *from, const char *to) {
    int in = open(from, O_RDONLY), out;
    char buf[4096]; ssize_t n;
    if (in < 0) return -1;
    out = open(to, O_WRONLY | O_CREAT | O_TRUNC, 0644);
    if (out < 0) { close(in); return -1; }
    while ((n = read(in, buf, sizeof buf)) > 0) {
        ssize_t off = 0;
        while (off < n) { ssize_t w = write(out, buf + off, n - off);
            if (w < 0) { close(in); close(out); return -1; } off += w; }
    }
    if (n < 0 || fsync(out) < 0 || close(out) < 0) { close(in); return -1; }
    if (close(in) < 0) return -1;
```

```
    return 0;
}
```

For the editor's replacement operation, use `openat` for the temporary file, write all bytes, `fsync`, close, and `renameat`. If durability is not required, the `fsync` policy should be stated explicitly rather than implied.

21.10 A multi-file assembly project

Directory layout:

```
rem-mini/
  main.S
  output.S
  Makefile
```

`main.S` calls `rem_output`, while `output.S` owns the string and write sequence:

```
.text
.global _start
.extern rem_output
_start:
        jal rem_output
        li  r1, 94
        li  r2, 0
        syscall 0
1:      j 1b
```

```
.text
.global rem_output
rem_output:
        lui r3, %hi(message)
        ori r3, r3, %lo(message)
        li  r1, 64
        li  r2, 1
        li  r4, message_end-message
        syscall 0
        ret
.section .rodata
message: .ascii "REM multi-file build\\n"
message_end:
```

The Makefile makes dependencies visible and keeps host tools out of the target build:

```
PREFIX ?= .toolchain-install/bin/myemulator2-elf-
AS = $(PREFIX)as
LD = $(PREFIX)ld
OBJDUMP = $(PREFIX)objdump

all: rem-mini.elf
rem-mini.elf: main.o output.o
    $(LD) -T toolchain/userspace/myemulator2-user.ld $^ -o $@
main.o: main.S
    $(AS) $< -o $@
output.o: output.S
    $(AS) $< -o $@
inspect: rem-mini.elf
    $(OBJDUMP) -dr $<
clean:
    rm -f *.o rem-mini.elf
```

```
.PHONY: all inspect clean
```

Run `make`, then `make inspect`. The linker error for a missing `rem_output` symbol is useful: it proves that the module boundary is real.

21.11 One debugging session

Start with a known ELF and capture its structure:

```
myemulator2-elf-readelf -h -l -s -r rem-mini.elf
myemulator2-elf-objdump -dr rem-mini.elf > rem-mini.dis
qemu-system-myemulator32 -M myemulator32 -kernel vmlinux \
  -nographic -d in_asm,cpu,nochain -D qemu.log
```

A trace block has an instruction address followed by decoded instructions and register state. Find the last address printed before the fault, then map it back to `rem-mini.dis`. If the address is not in a loadable segment, inspect ELF program headers and the linker script first. If the instruction is `.word` in the disassembly, check its opcode and reserved bits. If the fault is a load/store exception, check the effective address and alignment before changing the instruction decoder.

For a stopped host-side machine, add `-S -gdb tcp::1234` and connect with the matching target GDB if it is installed. The current target does not promise a complete guest `ptrace` debugger; QEMU trace plus `readelf/objdump` is the reliable baseline.

These examples are deliberately small enough to become regression fixtures. Once each boundary works, combine them into the REMEDIT architecture from the preceding chapters.

Appendix A — REM Programming Checklist

This checklist is intended to be used before, during and after writing REM software.

The purpose is not bureaucracy.

The purpose is to prevent spending hours debugging a problem caused by an assumption that was never tested.

A senior systems programmer works from proven foundations.

Before Starting a Project

Define the goal

Write down:

```
What does this program do?

What inputs does it receive?

What outputs does it produce?

What operating system services does it need?
```

Avoid starting with:

> "I will just write some code and see what happens."

Assembly makes mistakes expensive.

Check available services

Before implementing functionality, check whether REM already provides it.

Available:

```
Linux kernel

musl libc

system calls

BusyBox utilities

assembler

linker
```

Do not rewrite existing functionality without a reason.

Examples:

Do not write:

```
your own malloc
```

unless testing memory management.

Do not write:

```
your own printf
```

unless testing output routines.

Project Structure

Use a consistent layout:

```
project/
src/
module1.s
module2.s
tests/
module1_test.s
build/
object files
bin/
executables
docs/
notes
```

The structure matters because projects grow.

A pile of assembly files becomes impossible to maintain.

Module Checklist

Every module should define:

```
Purpose
Inputs
Outputs
Registers modified
```

```
Memory ownership

Error handling
```

Example:

```
terminal_write()

Purpose:
```

Write characters to terminal ~~~

Input:

```
    r1 = address

    r2 = length
```

Output:

```
    r1 = bytes written
```

Clobbers:

```
R2,R3

---

# Before Coding

Create a small test first.

Ask:
```

How do I know this works?

```
Examples:

Before writing:
```

terminal_move()

```
write:
```

terminal_test

```
Before writing:
```

buffer_insert()

```
write:
```

buffer_test

```
---

# During Coding

Build frequently.
```

```
Recommended cycle:
```

Change one thing

```
    |
    v
```

Build

```
    |
    v
```

Run test

```
    |
    v
```

Checkpoint

```
Do not accumulate hundreds of changes before testing.

---

# Debugging Checklist

When something fails:

Ask in order:
```

Did the assembler fail?

Did the linker fail?

Did the program start?

Did the loader fail?

Did the syscall fail?

Did the kernel fail?

Did the CPU execute incorrectly?

```
Do not immediately assume the CPU is wrong.

Most bugs are higher in the stack.

---

# Before Release

Verify:
```

Program starts

Normal input works

Invalid input handled

Errors reported

Resources released

Terminal restored

```
A program that works once is not necessarily finished.

---

# Appendix B — Assembly Module Template

Every assembly module should have a predictable structure.

Example:

/*
 * module_name.s
 *
 * Purpose:
 *
 *     Describe what this module provides.
 *
 *
 * Interface:
 *
 *     function_name()
 *
 *
 * Inputs:
 *
 *     r1 = ...
 *
 *
 * Outputs:
 *
 *     r1 = ...
 *
 *
 * Clobbers:
 *
 *     r5,r6
 *
 */

.section .text


.global function_name


function_name:

    /*
     * Function implementation
     */


    ret
```

Function Entry Rules

Every function documents:

Inputs

Where does data arrive?

Example:

```
r1 = buffer address

r2 = length
```

Outputs

Where does the result return?

Example:

```
r1 = result

 r1 = negative error result, where applicable
```

Register preservation

Define:

Preserved:

```
r5-r12 and any required frame state
```

Temporary:

```
r1-r4 and r15
```

The exact convention depends on the REM ABI.

The important rule:

Be consistent.

Stack Usage

Document stack frames.

Example:

```
Stack before:

caller data


After entry:

+----------------+
```

```
return address

+----------------+

saved registers

+----------------+

local variables

+----------------+
```

Never use stack memory without ownership.

Error Handling

Define failure behaviour.

Example:

Success:

```
r1 = 0
```

Failure:

```
r1 = -1
```

or:

```
carry flag set
```

The entire system must agree.

Appendix C — Test Program Template

Every important module should have a small standalone test.

Example:

```
tests/

    terminal_test.s

    buffer_test.s

    lexer_test.s
```

Test Structure

```
/*
 * test_module.s
 *
 * Verify module behaviour.
 */


.global main


main:

    /*
     * Setup test
     */


    /*
     * Call function
     */


    /*
     * Verify result
     */


    /*
     * Report PASS/FAIL
     */


    RET
```

Test Philosophy

A test should answer one question.

Good:

```
Does buffer_insert place a character correctly?
```

Bad:

```
Does the entire editor work?
```

Small tests isolate problems.

Test Output

Use a consistent format.

Example:

```
TEST buffer_insert

PASS
```

or:

```
TEST buffer_insert

FAIL

Expected:

ABC

Got:

ACB
```

Appendix D — Makefile Template

Even assembly projects benefit from make.

A simple REM project:

```
TARGET=myprogram


SRC=main.s terminal.s buffer.s


OBJ=$(SRC:.s=.o)


all: $(TARGET)


$(TARGET): $(OBJ)
    ld $(OBJ) -o $(TARGET)


%.o: %.s
    as $< -o $@


clean:
    rm -f $(OBJ) $(TARGET)


test:
    make -C tests
```

MyEmulator2 toolchain variant

The generic `as` and `ld` names in older examples refer to a configured cross-toolchain. With the current project installation, use the target tools explicitly:

```
TARGET := myprogram
AS ?= myemulator2-elf-as
LD ?= myemulator2-elf-ld
SOURCES := main.S terminal.S buffer.S
OBJECTS := $(SOURCES:.S=.o)

all: $(TARGET)

$(TARGET): $(OBJECTS)
    $(LD) $(OBJECTS) -o $@

%.o: %.S
    $(AS) $< -o $@

clean:
    rm -f $(OBJECTS) $(TARGET)
```

The target tools are installed under `.toolchain-install/bin` by the repository toolchain scripts. The complete inspection sequence is shown in the toolchain reference near the end of this book.

Make Concepts

Targets

A target is something to build.

Example:

```
all
```

or:

```
test
```

Dependencies

Example:

```
program

depends on:

main.o

terminal.o
```

If only one file changed, make rebuilds only that file.

Common commands

Build:

```
make
```

Clean:

```
make clean
```

Run tests:

```
make test
```

Appendix E — Terminal Escape Reference

REMEDIT treats the terminal as a byte-oriented protocol. The table below is the practical ANSI-compatible subset needed by a small full-screen editor. It is deliberately a stable working set rather than a promise that every terminal implements every extension.

Bytes first

`ESC` is not a word understood by the CPU. It is one byte: hexadecimal `0x1B`, written as `\\033` or `\\x1b` in C. `CSI` is the two-byte prefix `ESC [` (`1B 5B`). For example, clearing the screen sends four bytes:

Meaning	Byte values	C spelling
Clear screen	`1B 5B 32 4A`	`write(1, "\\x1b[2J", 4)`
Move to row 10, column 20	`1B 5B 31 30 3B 32 30 48`	`write(1, "\\x1b[10;20H", 8)`
Hide cursor	`1B 5B 3F 32 35 6C`	`write(1, "\\x1b[?25l", 6)`

In assembly, put those bytes in a read-only string and pass its address and length to the `write` wrapper. The terminal emulator interprets the bytes; the kernel only sees a write to file descriptor 1.

Notation and conventions

- `ESC` is byte `0x1b`; `CSI` means `ESC [`.
- Parameters are decimal and normally 1-based. An omitted movement count means one.
- Send the bytes with one or more `write()` calls. The terminal emulator, not the Linux kernel, interprets the sequence.
- Query sequences produce a response on the input stream, so raw input mode and a parser are required before using them.

Cursor movement

Purpose	Sequence	Meaning
Up	`CSI n A`	Move up `n` rows
Down	`CSI n B`	Move down `n` rows
Forward	`CSI n C`	Move right `n` columns
Back	`CSI n D`	Move left `n` columns
Next line	`CSI n E`	Move down and to column 1
Previous line	`CSI n F`	Move up and to column 1
Column	`CSI n G`	Set column to `n`
Position	`CSI row ; col H`	Set row and column (1-based)
Position (alias)	`CSI row ; col f`	Same as `H`
Row	`CSI n d`	Set row to `n`
Save cursor	`ESC 7` or `CSI s`	Save position and (usually) attributes
Restore cursor	`ESC 8` or `CSI u`	Restore saved position

For example, `ESC [ 10 ; 20 H` moves to row 10, column 20. A cursor position query is `CSI 6 n`; a compatible terminal replies with `CSI row ; col R`.

Erasing and editing the display

Purpose	Sequence	Meaning
Erase display below	`CSI 0 J`	Cursor through end of screen
Erase display above	`CSI 1 J`	Start of screen through cursor
Erase display	`CSI 2 J`	Entire visible screen
Erase line right	`CSI 0 K`	Cursor through end of line
Erase line left	`CSI 1 K`	Start of line through cursor
Erase line	`CSI 2 K`	Entire current line
Erase characters	`CSI n X`	Replace `n` characters with blanks
Insert lines	`CSI n L`	Insert `n` blank lines
Delete lines	`CSI n M`	Delete `n` lines
Insert characters	`CSI n @`	Insert `n` blanks at cursor
Delete characters	`CSI n P`	Delete `n` characters
Scroll up	`CSI n S`	Scroll content up `n` lines
Scroll down	`CSI n T`	Scroll content down `n` lines
Set scroll region	`CSI top ; bottom r`	Restrict scrolling to a region

A redraw normally starts with `CSI 2 J`, moves to `CSI 1 ; 1 H`, writes the visible rows, and finishes by placing the cursor at the logical edit point. For a single changed line, `CSI 2 K` followed by the replacement text avoids a full-screen redraw.

Cursor visibility and terminal state

Purpose	Sequence
Hide cursor	`CSI ?25l`
Show cursor	`CSI ?25h`
Reset attributes	`CSI 0 m`
Request cursor position	`CSI 6 n`

Always restore the cursor and text attributes on exit, including error paths. The terminal driver state (termios) and emulator state are separate; restoring termios alone does not restore colours or cursor visibility.

Text attributes and colours

Purpose	SGR sequence
Reset all	`CSI 0 m`
Bold	`CSI 1 m`
Dim	`CSI 2 m`
Italic (support varies)	`CSI 3 m`
Underline	`CSI 4 m`
Reverse video	`CSI 7 m`
Normal intensity	`CSI 22 m`
Italic off	`CSI 23 m`
Underline off	`CSI 24 m`
Reverse off	`CSI 27 m`
Standard foreground	`CSI 30–37 m`
Standard background	`CSI 40–47 m`
Bright foreground	`CSI 90–97 m`
Bright background	`CSI 100–107 m`
256-colour foreground	`CSI 38 ; 5 ; n m`
256-colour background	`CSI 48 ; 5 ; n m`
True-colour foreground	`CSI 38 ; 2 ; r ; g ; b m`

Purpose	SGR sequence
True-colour background	`CSI 48 ; 2 ; r ; g ; b m`

The standard foreground values are black, red, green, yellow, blue, magenta, cyan, and white in order. Extended colour modes are useful enhancements, but a portable editor should have a readable path using the standard eight colours.

Function and editing keys

These are common emulator encodings. Applications should accept the arrow forms first and treat function-key variants as terminal-dependent input.

Key	Sequence
Up / down / right / left	`CSI A` / `CSI B` / `CSI C` / `CSI D`
Home / end	`CSI H` / `CSI F`
Insert	`CSI 2 ~`
Delete	`CSI 3 ~`
Page up / page down	`CSI 5 ~` / `CSI 6 ~`
F1–F4 (common)	`ESC O P` / `Q` / `R` / `S`
F5–F12 (common)	`CSI 15~`, `17~`, `18~`, `19~`, `20~`, `21~`, `23~`, `24~`

A key decoder must handle a partial sequence: bytes can arrive one at a time, and escape-prefix timeout policy is part of the editor's input design.

`ESC` is ambiguous. It is both the Escape key and the first byte of every CSI or SS3 sequence. A decoder therefore enters an `ESCAPE` state, waits for the next byte for a short bounded interval, and emits Escape if no byte arrives. `ESC [` begins CSI sequences such as `ESC [ A`; `ESC O` begins the SS3 sequences commonly used by F1–F4 (`ESC O P` through `ESC O S`). Do not decide that an isolated `0x1B` is Escape until this small timeout has expired.

Minimal REMEDIT redraw set

A first editor can be useful with only these operations:

Job	Sequence
Clear and home	`CSI 2 J`, then `CSI 1 ; 1 H`
Move to an edit row	`CSI row ; 1 H`
Erase the rest of a row	`CSI 0 K`
Erase a complete row	`CSI 2 K`
Hide while drawing	`CSI ?25l`
Show after drawing	`CSI ?25h`
Return to normal attributes	`CSI 0 m`
Read cursor position	`CSI 6 n` and parse `CSI row ; col R`

This subset is enough to implement viewport redraw, status lines, cursor placement, and a clean shutdown before adding colours or scrolling regions.

Appendix F — Debugging Decision Tree

When something fails, follow the layers.

Do not guess.

Program Will Not Build

`make failed`

Check:

Flow: Assembler error? → Linker error? → Missing symbol? → Wrong library?

Program Builds But Will Not Start

Check:

`ELF format?`

`Architecture correct?`

`Loader available?`

`Permissions?`

Program Starts But Crashes

Check:

`Invalid memory access?`

`Bad register value?`

`Stack corruption?`

`Wrong syscall?`

Program Produces Wrong Result

Check:

`Input correct?`

`Algorithm correct?`

```
Register values correct?

Instruction implementation correct?
```

CPU Suspected

Only after higher layers are verified:

Check:

```
Instruction decoder

ALU operation

Memory access

Exception handling

Pipeline/state
```

Terminal Broken After Crash

Usually:

Terminal was left in raw mode.

Recovery:

```
reset
```

or:

restore terminal state manually.

The editor should always attempt:

```
terminal_restore()
```

on exit.

Final Rule

When debugging a new architecture:

Do not ask:

> “Why is the computer broken?”

Ask:

> “Which layer made the first incorrect assumption?”

The answer is usually much closer than expected.

Appendix G — Current MyEmulator2 Toolchain Reference

The reproducible target prefix is `myemulator2-elf`. After building the toolchain, a bare-metal assembly program can be inspected with:

Build the target binutils and QEMU components with:

```
./toolchain/scripts/fetch-binutils.sh
./toolchain/scripts/build-binutils.sh
./qemu/tools/build-myemulator.sh
```

```
TOOL=.toolchain-install/bin/myemulator2-elf

$TOOL-as program.s -o program.o
$TOOL-ld program.o -o program.elf
$TOOL-readelf -h -S -s -r program.elf
$TOOL-objdump -d program.elf
$TOOL-nm program.elf
$TOOL-objcopy -O binary program.elf program.bin
```

The canonical artifact is ELF32 little-endian. A raw binary is a derived deployment artifact and does not contain the ELF entry point. Use the ELF file when the loader supports it.

The ELF machine value is the project-local `EM_MYEMULATOR2 = 0xF2E2`; it is not an official allocation. The linker emits the target's `R_MYEMU_NONE`, `R_MYEMU_32`, `R_MYEMU_BRANCH13`, `R_MYEMU_JUMP26`, `R_MYEMU_HI20`, and `R_MYEMU_LO12` relocations. `readelf -h -r` is the quickest way to confirm that a file was produced for this target.

The target toolchain test is:

```
./toolchain/scripts/test-binutils.sh
```

The GCC backend is a separate test path. The initial compiler is freestanding/static; do not assume a host compiler or dynamic loader exists inside the guest.

Appendix H — Verified Linux Syscall Reference

The current Linux userspace convention is:

```
r1 = syscall number on entry; return value on exit
r2-r7 = syscall arguments
```

The `syscall` instruction performs the user-to-kernel transition. The currently exercised calls include:

Number	Call	Use in current tests
17	`getcwd`	current directory reporting
34	`mkdirat`	filesystem tests
35	`unlinkat`	filesystem tests
38	`renameat`	safe-save path
49	`chdir`	build working directory
56	`openat`	file access
57	`close`	file cleanup
62	`lseek`	file positioning
63	`read`	input and files
64	`write`	output and files
74	`fsync`	requested durability
79/80	`fstatat/fstat`	file metadata
94	`exit`	process termination
124	`sched_yield`	cooperative tests
160	`uname`	platform reporting
172	`getpid`	process reporting
221	`execve`	launch a build or program
220	`clone`	create a child process
260	`wait4`	collect child status
214	`brk`	allocator bootstrap
403	`clock_gettime`	project-local timing number; verify target headers

```
The table is a bring-up reference, not a promise of complete Linux syscall
coverage. Check the current Linux port and wrapper source before depending on
a call. Negative results are errors and must not be treated as valid byte
counts or pointers.

The current kernel dispatcher also defines these entries for the hosted
runtime and terminal work:

| Number | Call | Current role |
| ---: | --- | --- |
| 23/24 | `dup`/`dup3` | standard-descriptor setup |
| 25 | `fcntl` | descriptor flags |
| 29 | `ioctl` | tty and device control |
| 52/53 | `fchmod`/`fchmodat` | file permissions |
| 59 | `pipe2` | build-process pipes |
| 66 | `writev` | libc buffered output |
| 72/73 | `pselect6`/`ppoll` | descriptor waits |
| 81 | `sync` | filesystem flush |
| 93/94 | `exit`/`exit_group` | process termination |
| 101 | `nanosleep` | timed waits |
```

```
| 144/146 | `setgid`/`setuid` | identity setup |
| 154–157 | `setpgid`/`getpgid`/`getsid`/`setsid` | session management |
| 166 | `umask` | file creation mask |
| 173–177 | `getppid`/`getuid`/`geteuid`/`getgid`/`getegid` | identity queries |

These numbers come from `linux/arch/myemulator2/kernel/syscall.c`; the
dispatcher list and the generated libc headers are the authority if they
change. The six argument registers are always `r2` through `r7`, with unused
slots set to zero by a wrapper.

## Wrapper and error rules

Treat `r1` as clobbered by every syscall. The kernel may also overwrite the
argument registers, so a wrapper must preserve any live values in
callee-saved registers or on the stack before executing `syscall 0`. The
current wrappers preserve the normal function ABI and return the kernel's
value unchanged.

The kernel reports failure as a negative error number, for example `-9` for
`EBADF` or `-11` for `EAGAIN`. A raw assembly caller tests `r1 < 0`. The
libc wrapper translates that result to `-1` and stores the positive error
number in thread-local `errno`; C code therefore tests the return value and
then reads `errno`. Do not apply the libc translation twice.

The smallest raw write sequence is:

        li   r1, 64          # __NR_write
        li   r2, 1           # stdout
        # r3 = address of buffer
        li   r4, 4           # byte count
        syscall 0
        # r1 >= 0: bytes written; r1 < 0: -errno
```

Use the maintained `toolchain/userspace/minilibc/syscall.S` wrappers when writing C-callable functions; it contains the target-specific register moves and error convention for `read`, `write`, `openat`, `close`, and related calls.

Appendix I — Current Machine and Console Reference

The current Linux machine uses 16 MiB of default RAM. Physical reset words are at `0x400` (initial supervisor stack pointer) and `0x404` (initial PC). The MyEmulator2 console occupies `0xF0000000`–`0xF000000F`:

Offset	Meaning
0	RX/TX data byte
1	status: bit 0 RX ready, bit 1 TX ready
2	control: bit 0 RX interrupt enable
3	IRQ status: bit 0 RX pending

Console receive uses IRQ4. The architected timer uses IRQ1. The device moves raw bytes; line editing, ANSI interpretation, prompts, and command parsing belong to software above the device. Under Linux, the tty driver and termios sit between the application and this byte-oriented console.

Appendix J — Regression Commands

Use these commands as offline checkpoints:

```
./toolchain/scripts/test-binutils.sh
python3 toolchain/scripts/test-linux-process.py
python3 toolchain/scripts/test-linux-minilibc.py
python3 toolchain/scripts/test-linux-minilibc-shell.py
./tests/run-qemu-debug-tests.sh
```

A result should record the commit, target, command, exit status, and whether it ran under host QEMU or on the Fold. A host pass does not establish phone execution.

For the full userspace regression harness:

```
python3 toolchain/scripts/test-userspace.py \
  --rootfs /tmp/rem-userspace-rootfs-shell.ext4 \
  --output /tmp/rem-userspace-regression
```

The harness writes a report and preserves failures as failures. Do not turn a crash or timeout into a pass by substituting a different program.

Appendix K — Android Offline Deployment

The Fold runtime requires an ARM64 QEMU binary, a verified MyEmulator2 Linux kernel, and a verified ext4 root filesystem. Legacy `myemulator.img` files are not Linux root filesystems.

The package launcher uses the `myemulator32` machine and a serial console. Before launch, run its preflight check and keep checksum sidecars for the kernel and root filesystem. Validate persistence by creating a file in one boot, restarting normally, and reading it in the next boot.

The complete scripts and Termux setup are documented in:

```
android/README.md
android/TERMUX_DEPLOYMENT.md
android/DEPLOYMENT_CHECKLIST.md
```

Phone validation is complete only after the physical device boots the guest, shows the expected prompt, and passes the persistence check.

Appendix L — Legacy Example Conversion

Some detailed sections were written while the earlier REM instruction model was still in use. Convert those examples before assembling them for MyEmulator2:

Older explanatory form	MyEmulator2 form
`MOV r1, value`	`li r1, value` or the appropriate `add`/`addi` form
`LOAD r1, [address]`	construct an address, then `lw r1, 0(rbase)`
`STORE r1, [address]`	construct an address, then `sw r1, 0(rbase)`
`JMP label`	`j label`
`CALL label`	`jal label` or `call label` alias
`RET`	`jr r14` or `ret` alias
`R0–R15`	canonical documentation spelling `r0–r15`
semicolon comment	`#` comment in current GAS syntax

For a full 32-bit address, use the relocation pair:

```
lui r5, %hi(symbol)
ori r5, r5, %lo(symbol)
lw  r1, 0(r5)
```

Do not use the legacy `tools/myasm` examples as MyEmulator2 GAS input unless the example has been deliberately ported.

GAS data-layout patterns

These directives are sufficient for small tables and records in a native MyEmulator2 assembly module:

```
.section .rodata
message:        .ascii "REM\n"
message_end:

.section .data
.align 2
counter:        .word 0
pair:           .word 10, 20

.section .bss
.align 2
buffer:         .space 256
```

`.byte`, `.short`, and `.word` emit 8-, 16-, and 32-bit values. `.space` reserves storage in a loadable or BSS section. Use `.align 2` for ordinary 32-bit objects, as in the target examples. A structure is a sequence of fields at documented offsets, accessed with `lw`/`sw displacement(base)`. Use `%hi(symbol)` and `%lo(symbol)` for a full symbol address rather than assuming that a label fits in one immediate.

Appendix M — First MyEmulator2 Assembly Program

This is the smallest maintained bare-metal assembly check. It performs an observable calculation and halts; it does not depend on Linux or libc.

```
.text
.global _start
_start:
        li   r1, 5
        li   r2, 7
        add  r3, r1, r2
        beq  r3, r0, _start
        halt
```

Save it as `minimal.s`, then assemble and link it:

```
TOOL=.toolchain-install/bin/myemulator2-elf
$TOOL-as minimal.s -o minimal.o
$TOOL-ld minimal.o -o minimal.elf
$TOOL-readelf -h -S minimal.elf
$TOOL-objdump -d minimal.elf
```

Run it with the MyEmulator2 QEMU target:

```
qemu-system-myemulator32 -machine myemulator32 \
  -display none -kernel minimal.elf
```

The program should reach `halt` without an illegal-instruction, alignment, or page fault. To inspect the final registers, start QEMU paused with `-S` and use the current QEMU debug interface. The expected arithmetic result is `r3 = 12` immediately before the branch test.

Appendix N — First Linux Syscall Program

The repository's `toolchain/examples/linux-echo.S` is a complete Linux userspace syscall example. This reduced excerpt shows the essential write and exit sequence:

```
.section .text
.global _start
_start:
        li   r1, 64              # write
        li   r2, 1               # stdout
        lui  r3, %hi(message)
        ori  r3, r3, %lo(message)
        li   r4, message_end-message
        syscall 0

        li   r1, 94              # exit_group
        li   r2, 0
        syscall 0

1:      j 1b

.section .rodata
message:
        .ascii "Hello REM Linux\n"
message_end:
```

This uses the Linux syscall ABI: `r1` is the number, `r2`–`r4` are the first three arguments for `write`, and the result returns in `r1`. The `syscall` instruction itself has no C-style automatic preservation guarantee; wrappers must treat the argument/result registers as call-clobbered and preserve any callee-saved registers they use.

The complete maintained echo fixture also demonstrates `read` and retrying when the bootstrap console returns `-EAGAIN`:

```
sed -n '1,140p' toolchain/examples/linux-echo.S
```

Build and boot the Linux fixture through the repository's Linux test scripts; do not pass this userspace ELF to the bare-metal reset path.

Appendix O — C and Startup Contract

There are two hosted paths. The small minilibc path is the easiest first regression; the static MyEmulator2 musl path provides a larger C environment. Both are target-specific and neither implies dynamic linking.

The maintained minilibc build uses the following shape:

```
PREFIX=.toolchain-install
GCC=$PREFIX/bin/myemulator2-elf-gcc
AS=$PREFIX/bin/myemulator2-elf-as
LD=$PREFIX/bin/myemulator2-elf-ld

$GCC -ffreestanding -fno-builtin -nostdinc -O2 \
  -I toolchain/userspace/minilibc/include \
  -c toolchain/examples/linux-minilibc.c -o main.o
$AS toolchain/userspace/minilibc/crt0-linux.S -o crt0.o
$AS toolchain/userspace/minilibc/syscall.S -o syscall.o
$LD -Ttext=0x02000000 -o init.elf crt0.o syscall.o main.o
```

The regression script additionally builds the required string support, constructs a disposable initramfs, rebuilds Linux with that initramfs, and boots QEMU:

```
python3 toolchain/scripts/test-linux-minilibc.py
```

The current `crt0-linux.S` calls `main`, moves its return value into the Linux exit argument, and invokes `exit_group`. It is a minimal bootstrap startup and does not promise a complete `argc/argv/envp` or `__libc_start_main` contract. Programs that need those interfaces should use the supported static musl startup path rather than assuming the minilibc entry point supplies them.

For the hosted static-musl path, use the target compiler driver and sysroot when they have been installed:

```
myemulator2-linux-musl-gcc -O2 -static hello.c -o hello
```

The compiler supplies the musl startup objects and the REM user linker script. The maintained native-GCC regression builds a disposable Linux image and verifies the result with:

```
python3 toolchain/scripts/test-linux-native-gcc.py
```

If that driver is not present, use the minilibc commands above; do not substitute the host compiler, because its ABI and startup objects target a different machine.

Appendix P — Current Debugging Recipe

For a bare-metal ELF, inspect the file before running it:

```
TOOL=.toolchain-install/bin/myemulator2-elf
$TOOL-readelf -h -l -S -s -r program.elf
$TOOL-objdump -dr program.elf
```

For QEMU instruction tracing, use the target's trace options:

```
qemu-system-myemulator32 -machine myemulator32 \
  -kernel program.elf -nographic \
  -d in_asm,cpu,nochain -D qemu.log
```

QEMU's generic remote stub can also stop the machine for host-side inspection:

```
qemu-system-myemulator32 -machine myemulator32 -kernel program.elf \
  -S -gdb tcp::1234
```

Use `myemulator2-elf-gdb` only if the matching target GDB build and register description are installed. The project does not yet promise a stable guest debugger or `ptrace` ABI, so a connection that lacks register support is a tooling limitation rather than evidence that the ELF is invalid.

For the legacy MyEmulator debugger, use `tools/mydebug` only with the legacy `myemulator` machine and its documented register model. It is not a Linux MyEmulator2 guest debugger. MyEmulator2 register inspection currently uses the host QEMU debug facilities and project-specific handoff procedures; a stable guest `ptrace` ABI is future work.

When a Linux program fails, preserve the complete serial transcript and run the narrowest matching regression first:

```
python3 toolchain/scripts/test-linux-process.py
python3 toolchain/scripts/test-linux-minilibc.py
python3 toolchain/scripts/test-userspace.py \
  --rootfs /tmp/rem-userspace-rootfs-shell.ext4 \
  --output /tmp/rem-userspace-regression
```

Appendix Q — Chapter-to-Repository Map

Book area	Repository material
ISA and ABI	`docs/MYEMULATOR_2_ARCHITECTURE.md`, `docs/MYEMULATOR_2_ABI.md`
Assembly syntax	`docs/MYEMULATOR2_ASSEMBLY_PROGRAMMING_MANUAL.md`, `toolchain/examples/`
Toolchain	`toolchain/README.md`, `toolchain/scripts/test-binutils.sh`
Linux boot	`linux/arch/myemulator2/kernel/head.S`, `setup.c`, Linux port docs
Syscalls	`toolchain/userspace/minilibc/syscall.S`, `bits/syscall.h.in`
Console and termios	`docs/console.md`, Linux tty overlay, Chapters 5–8
QEMU debugging	`docs/qemu-debugging.md`, `docs/REM_LINUX_DEBUG_HANDOVER.md`
Android deployment	`android/README.md`, `android/TERMUX_DEPLOYMENT.md`

Appendix R — Terminal ABI Notes

The terminal chapters explain behavior; this appendix records the current Linux constants used by the target headers. The Linux syscall number for `ioctl` is 29 in the current asm-generic table. The common terminal requests are:

```
TCGETS = 0x5401
TCSETS = 0x5402
TCSETSW = 0x5403
TCSETSF = 0x5404
NCCS = 19
VMIN = 6
VTIME = 5
ICANON = 0x00002
ECHO = 0x00008
ISIG = 0x00001
IXON = 0x00400
```

Use the target's termios headers and `tcgetattr`/`tcsetattr` wrappers when they are available; use raw `ioctl` only when the wrapper path is absent and the target headers confirm the same structure layout. Save the complete `struct termios`, clear `ICANON` and `ECHO` for editor input, set `VMIN` and `VTIME` deliberately, and restore the saved structure on every normal exit path. These constants do not make a full tty or signal implementation available; the current Linux port remains capability-limited.

Raw mode and terminal size

The implementation sequence is:

```
struct termios saved, raw;
tcgetattr(STDIN_FILENO, &saved);
raw = saved;
raw.c_lflag &= ~(ICANON | ECHO | ISIG);
raw.c_cc[VMIN] = 1;
raw.c_cc[VTIME] = 0;
tcsetattr(STDIN_FILENO, TCSAFLUSH, &raw);

/* editor loop */

tcsetattr(STDIN_FILENO, TCSAFLUSH, &saved);
```

The exact structure layout belongs to the target headers. Keep `saved` alive until shutdown, and use one cleanup path for normal exit and error returns. Install handlers for `SIGINT`, `SIGTERM`, and `SIGHUP` that set a shutdown flag; the cleanup code then restores termios, shows the cursor, resets SGR attributes, and exits. A signal handler should not call the full renderer.

To obtain the viewport, issue `ioctl` request `TIOCGWINSZ` on the terminal file descriptor and read `ws_row` and `ws_col` from `struct winsize`:

```
struct winsize size;
if (ioctl(STDOUT_FILENO, TIOCGWINSZ, &size) == 0) {
    rows = size.ws_row;
    columns = size.ws_col;
}
```

If the query fails or returns zero, use a conservative fallback and redraw after a resize notification when the target's signal and tty support provide one. The terminal reference sequences control the display; `ioctl` and termios control the kernel-side terminal device, so both layers must be handled by an editor.